

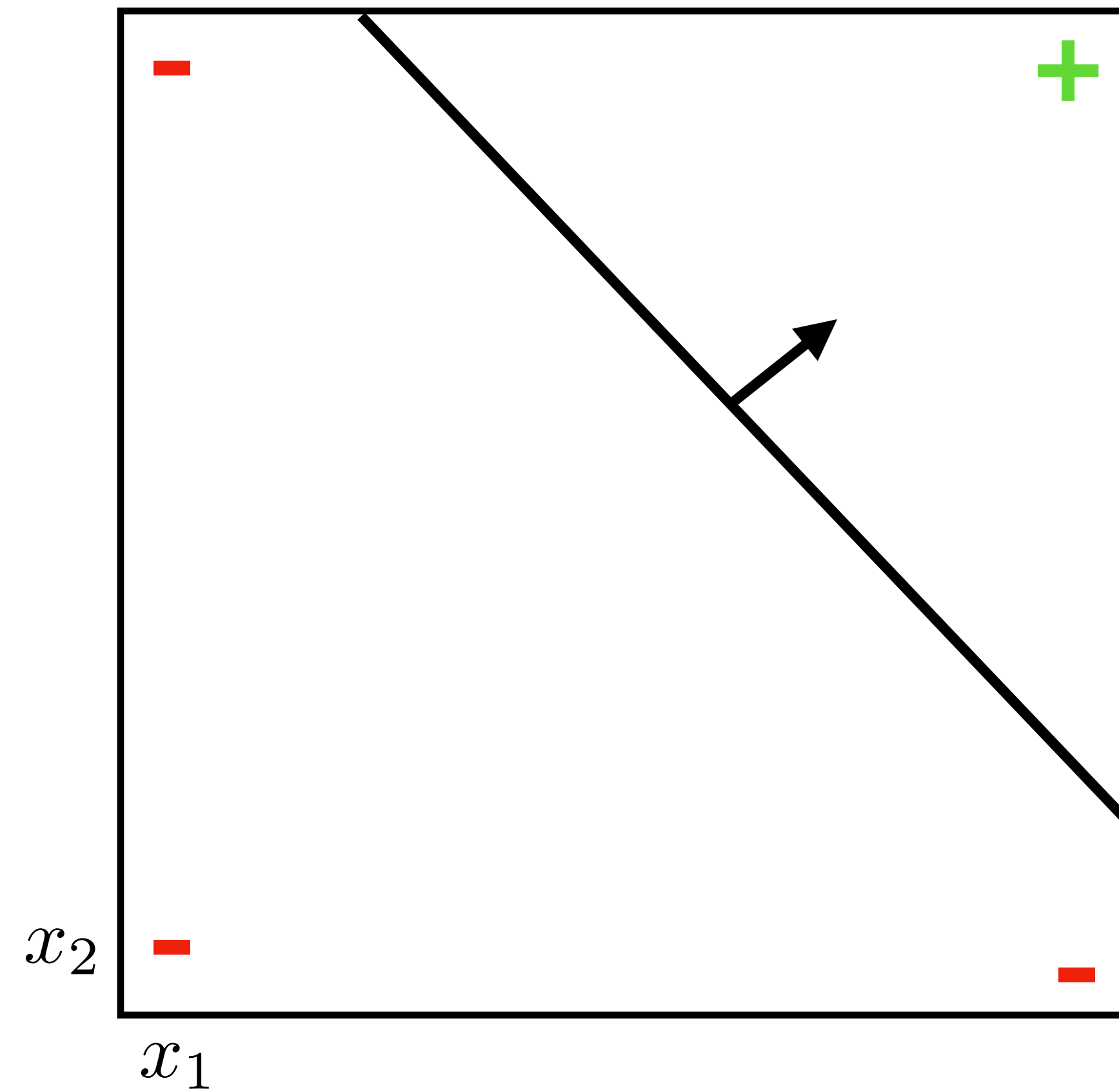
Lecture 7: Neural networks

- Section this week: machine learning
- Reading:
 - Szeliski 5.3
 - Goodfellow Deep Feedforward Networks
- Start thinking about project

Today

- Brief history of neural networks
- Computation in neural networks
- Multi-layer perceptrons (for PS4)
- Estimating gradients (to be continued next class).

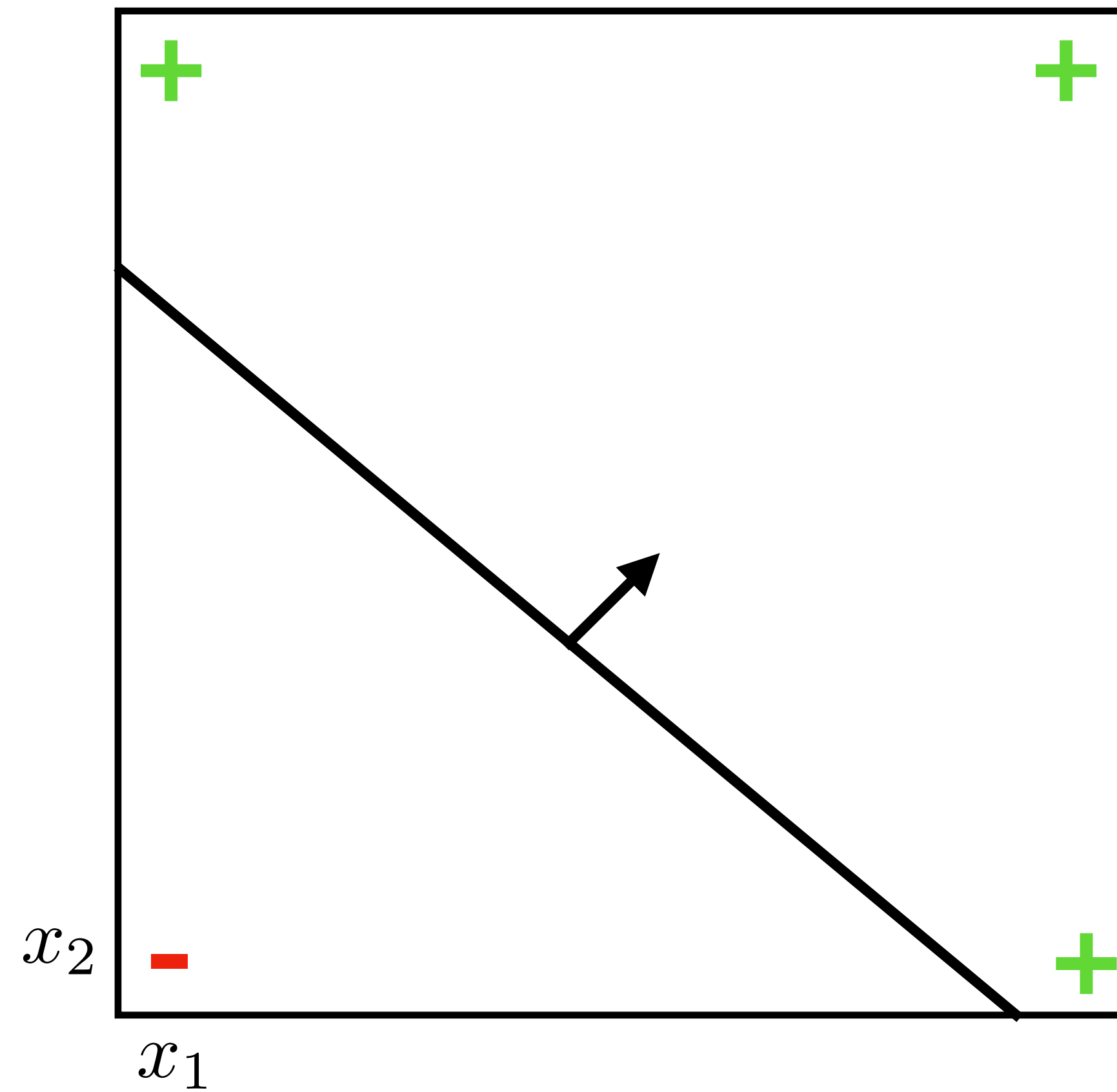
Recall: limitations to linear classifiers



		x_2	
		0	1
x_1	0	0	0
	1	0	1

AND

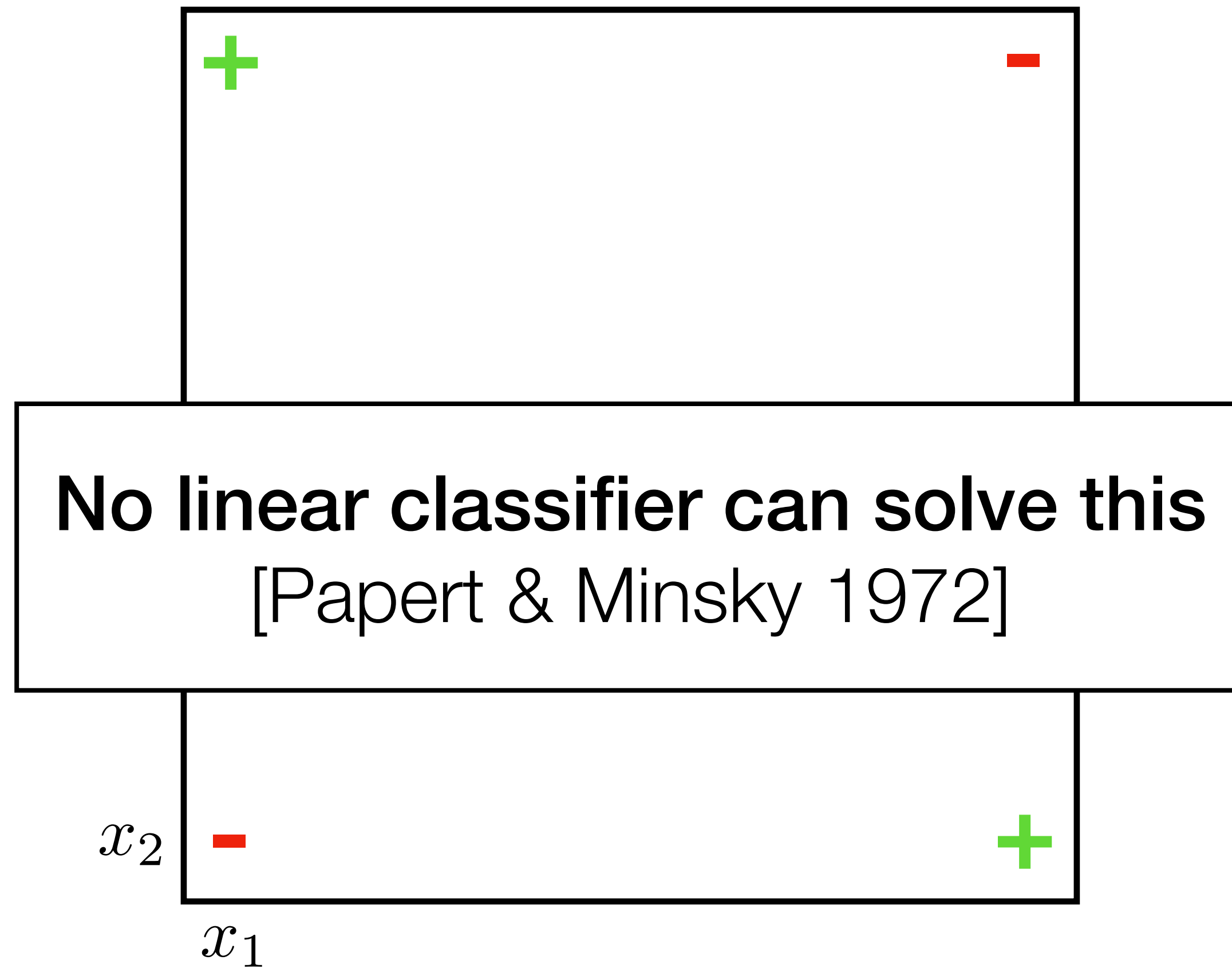
Limitations to linear classifiers



		x_2	
		0	1
x_1	0	0	1
	1	1	1

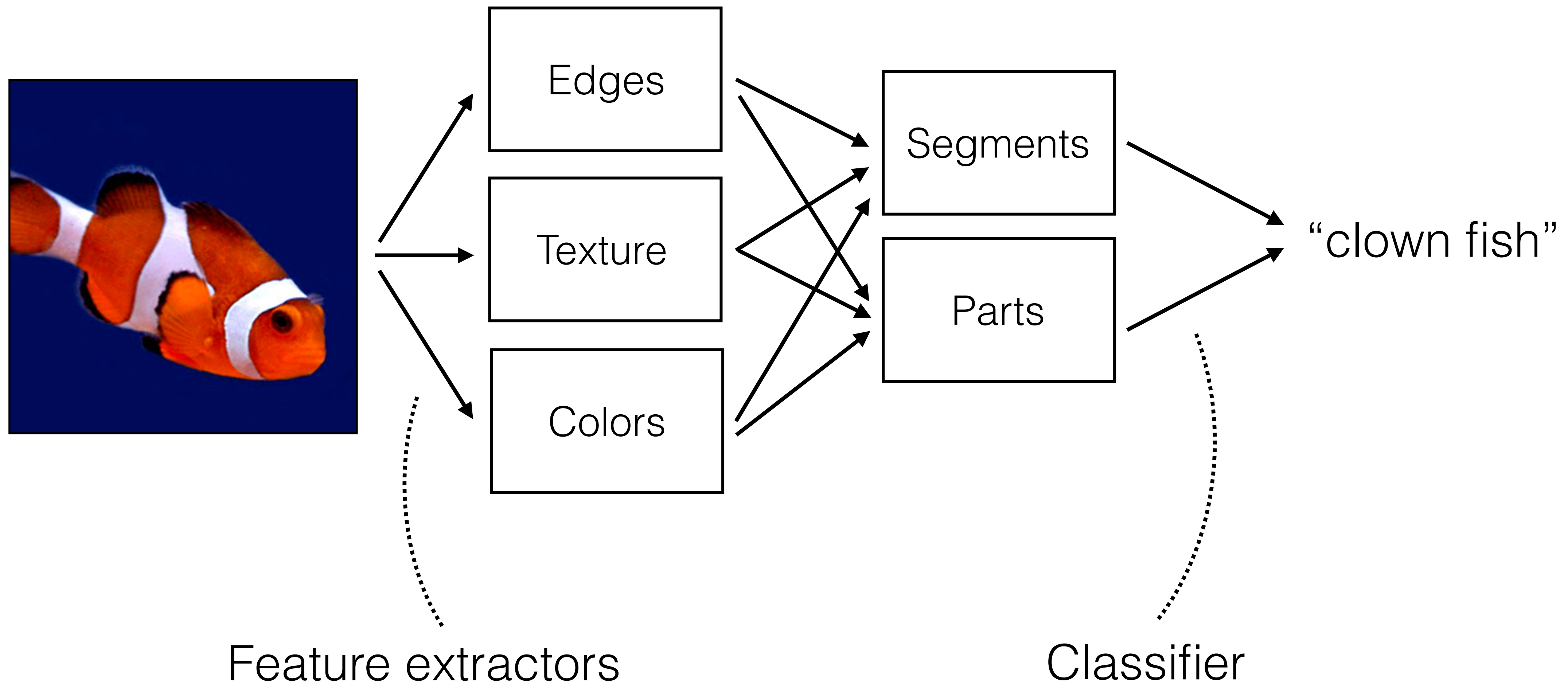
OR

Limitations to linear classifiers



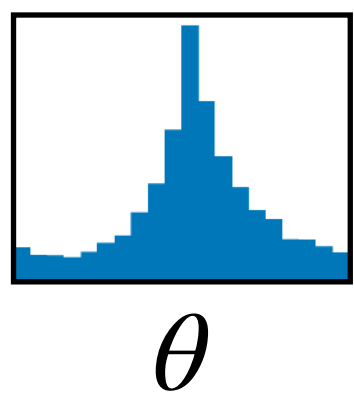
		x_2	
		0	1
x_1	0	1	0
	1	0	1
		XOR	

“Classic” recognition **without** neural nets

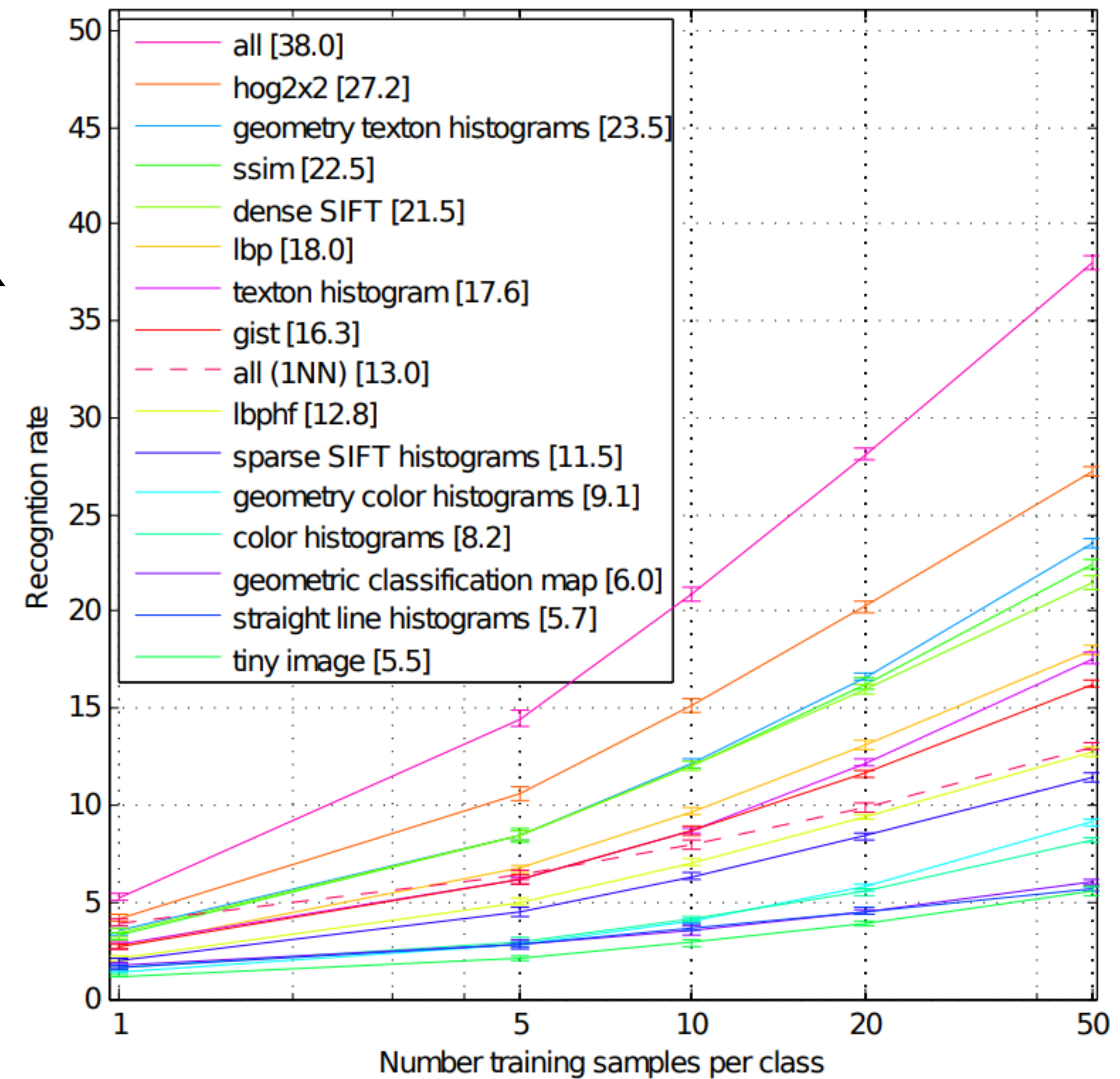


“Classic” recognition **without** neural nets

Hand-crafted features

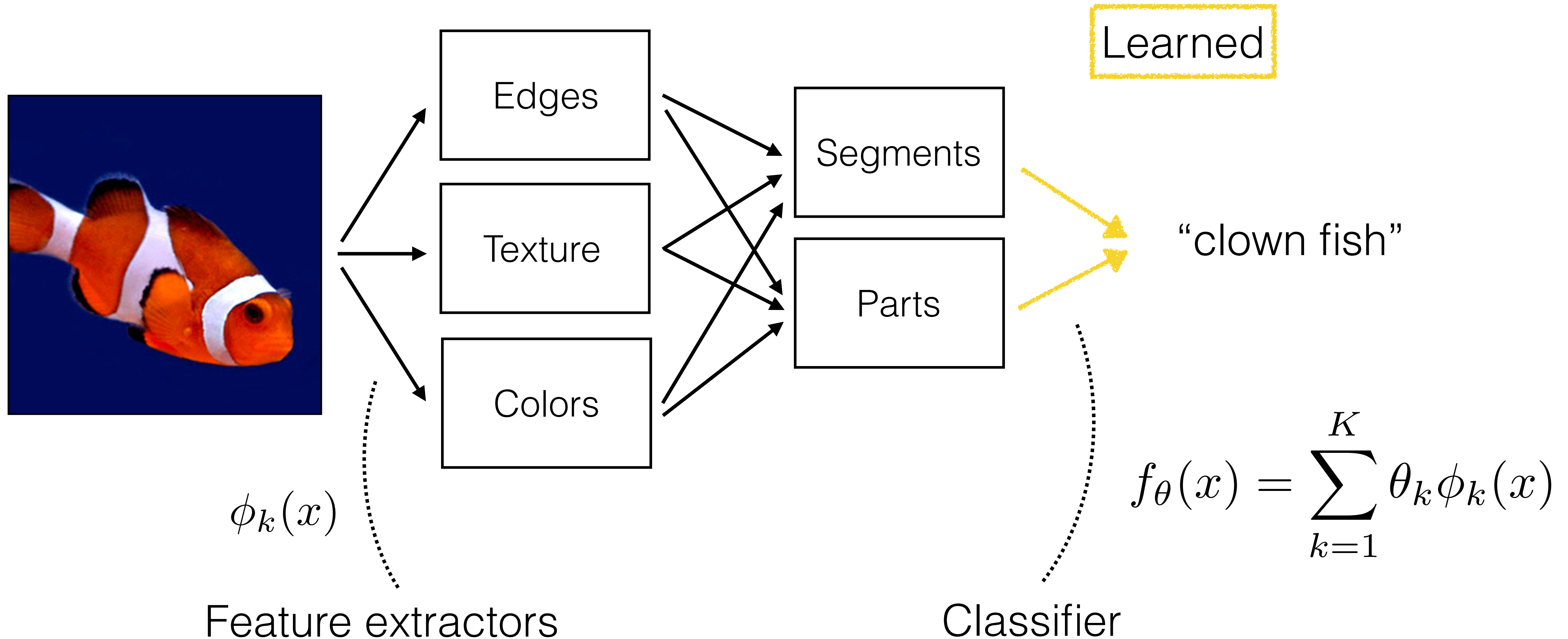


edge orientation
histogram

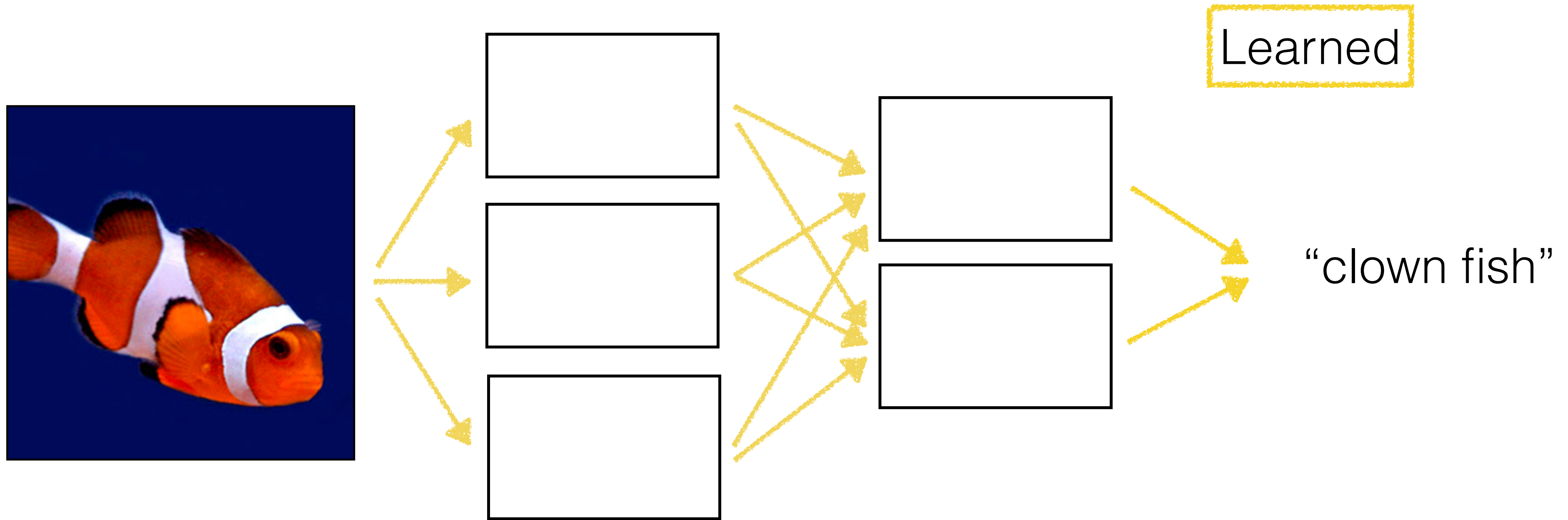


[Xiao et al., “SUN database”, 2010]

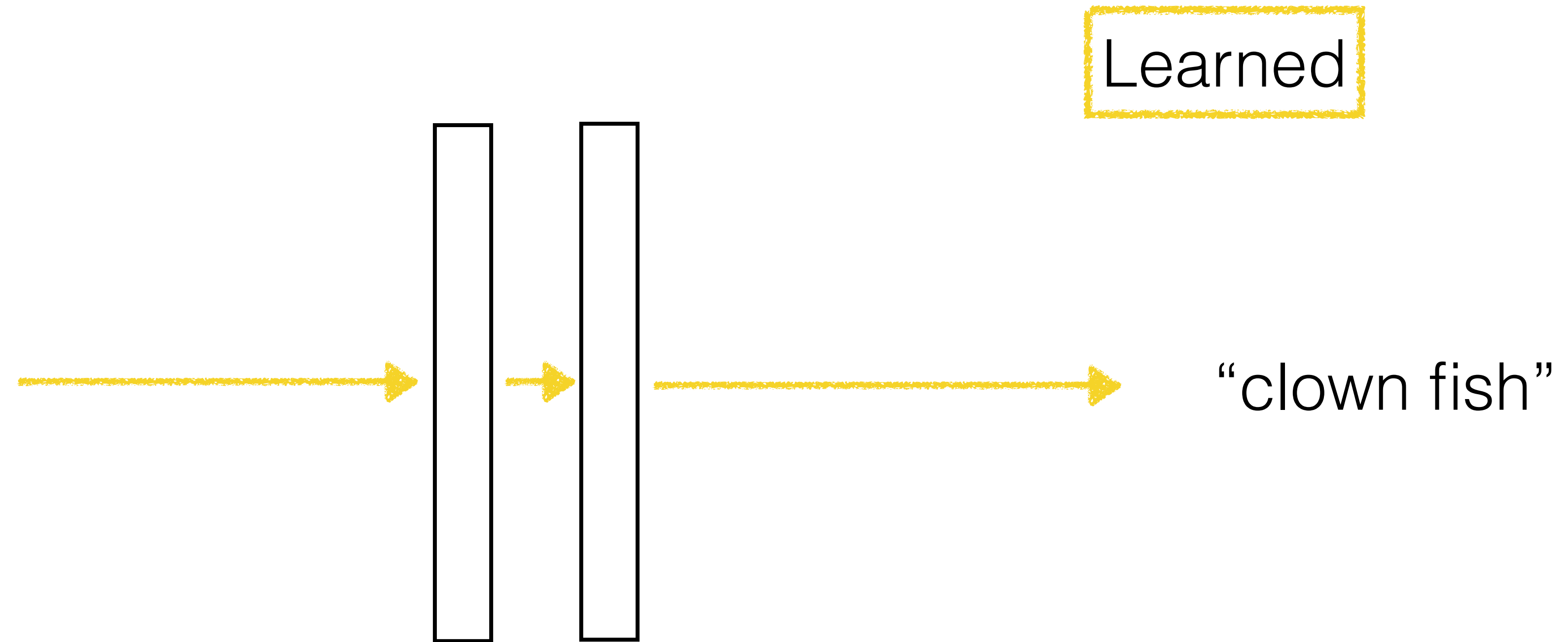
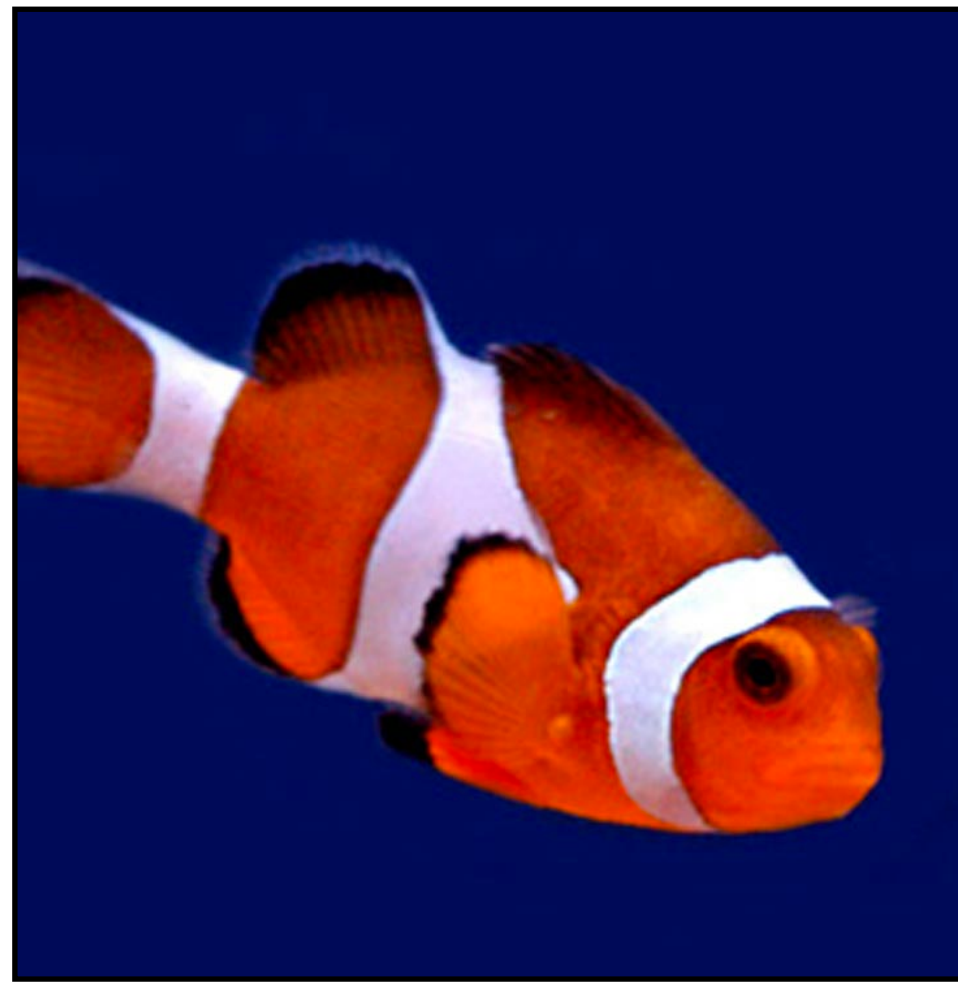
“Classic” object recognition **without** neural nets



Object recognition

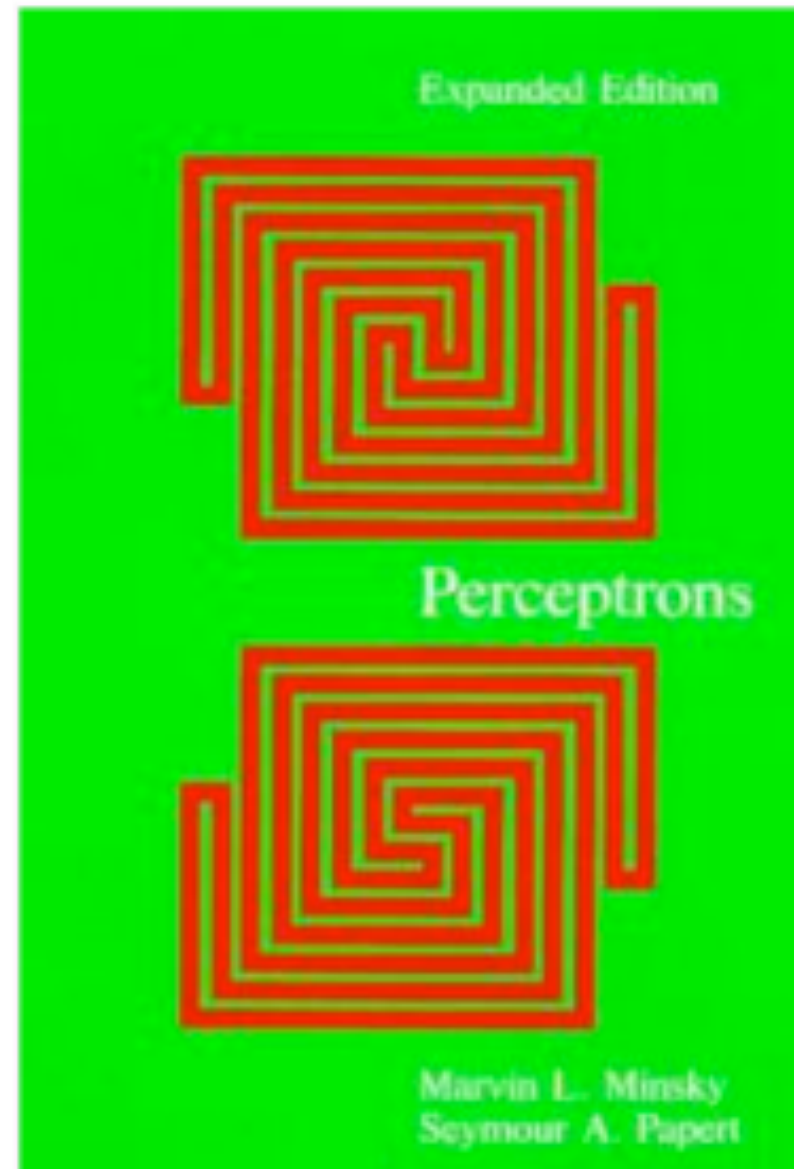


Object recognition



Neural net

Minsky and Papert, Perceptrons, 1972



FOR BUYING OPTIONS, START HERE

Select Shipping Destination

Paperback | \$35.00 Short | £24.95 |
ISBN: 9780262631112 | 308 pp. | 6 x
8.9 in | December 1987

Perceptrons, expanded edition

An Introduction to Computational Geometry

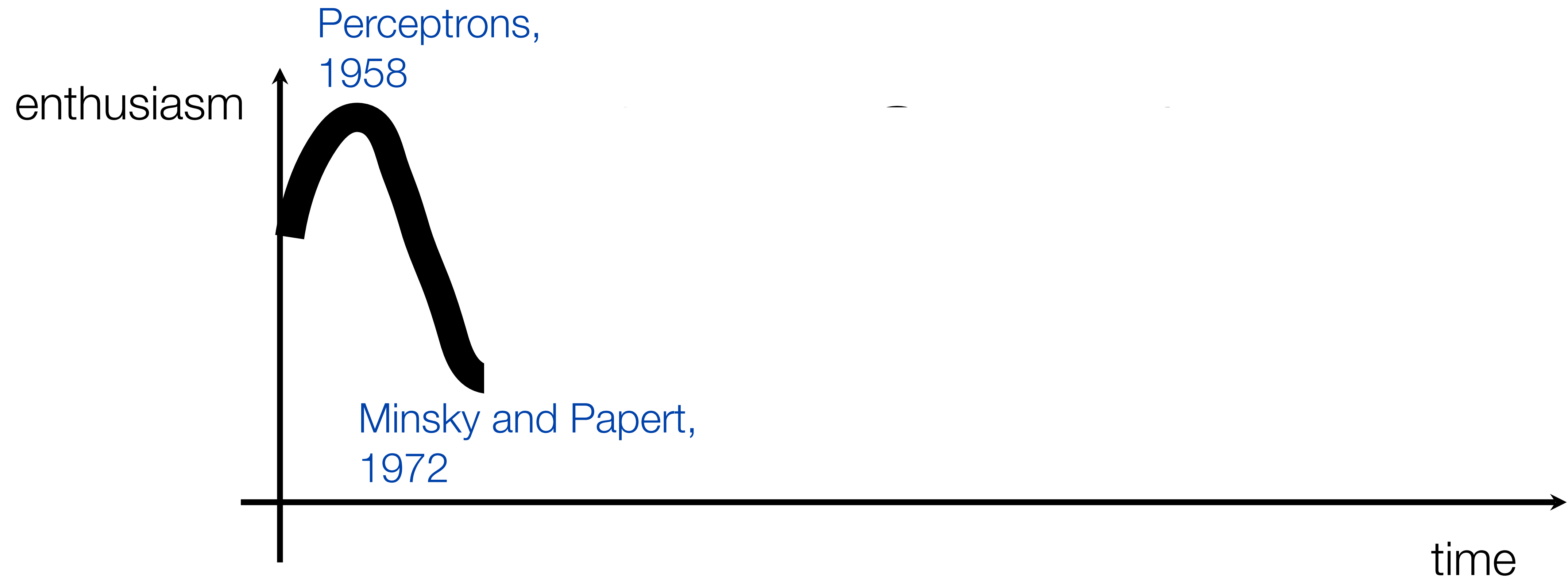
By [Marvin Minsky](#) and [Seymour A. Papert](#)

Overview

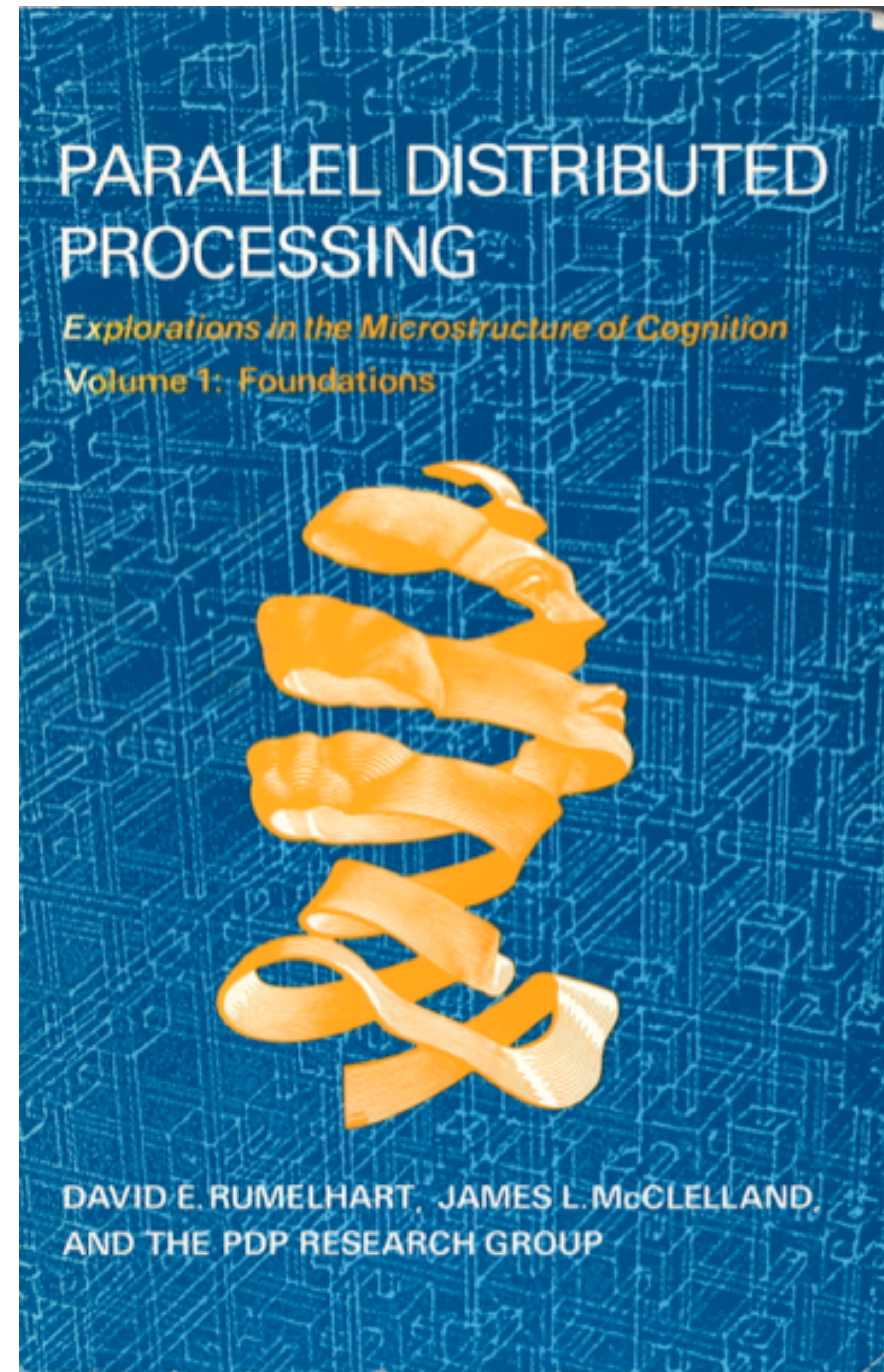
Perceptrons - the first systematic study of parallelism in computation - has remained a classical work on threshold automata networks for nearly two decades. It marked a historical turn in artificial intelligence, and it is required reading for anyone who wants to understand the connectionist counterrevolution that is going on today.

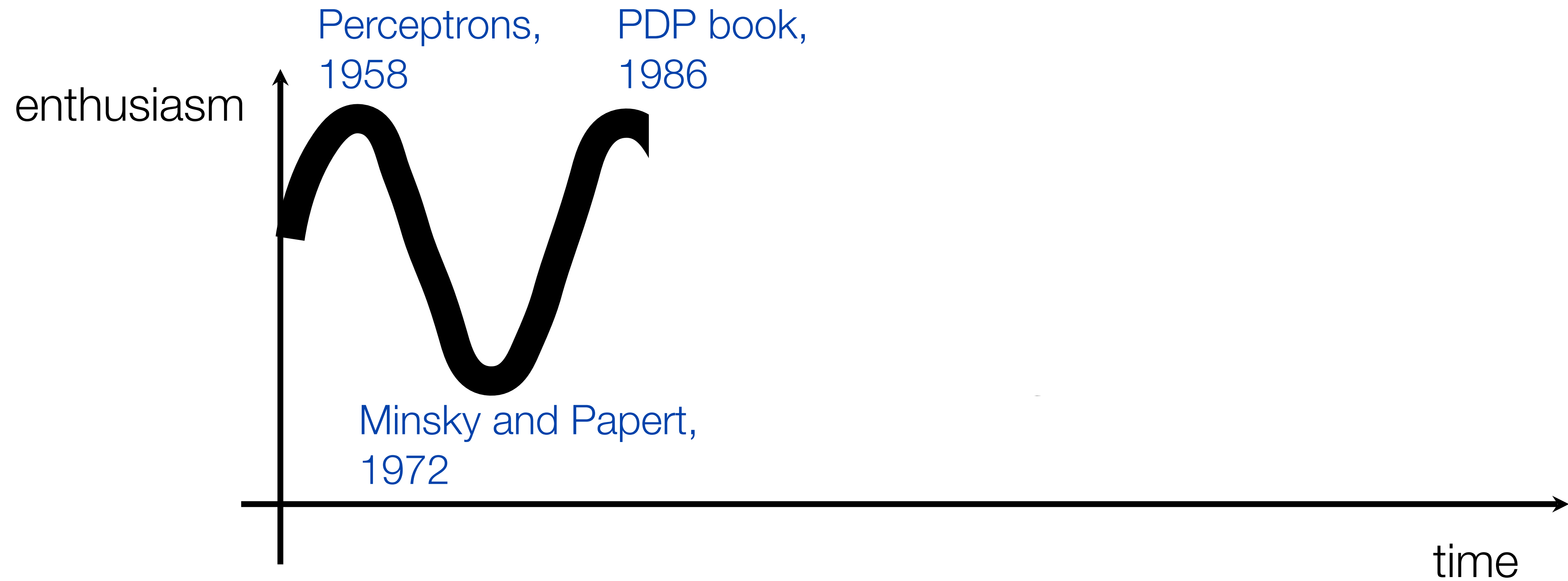
Artificial-intelligence research, which for a time concentrated on the programming of ton Neumann computers, is swinging back to the idea that intelligence might emerge from the activity of networks of neuronlike entities. Minsky and Papert's book was the first example of a mathematical analysis carried far enough to show the exact limitations of a class of computing machines that could seriously be considered as models of the brain. Now the new developments in mathematical tools, the recent interest of physicists in the theory of disordered matter, the new insights into and psychological models of how the brain works, and the evolution of fast computers that can simulate networks of automata have given *Perceptrons* new importance.

Witnessing the swing of the intellectual pendulum, Minsky and Papert have added a new chapter in which they discuss the current state of parallel computers, review developments since the appearance of the 1972 edition, and identify new research directions related to connectionism. They note a central theoretical challenge facing connectionism: the challenge to reach a deeper understanding of how "objects" or "agents" with individuality can emerge in a network. Progress in this area would link connectionism with what the authors have called "society theories of mind."



Parallel Distributed Processing (PDP), 1986





LeCun convolutional neural networks

PROC. OF THE IEEE, NOVEMBER 1998

7

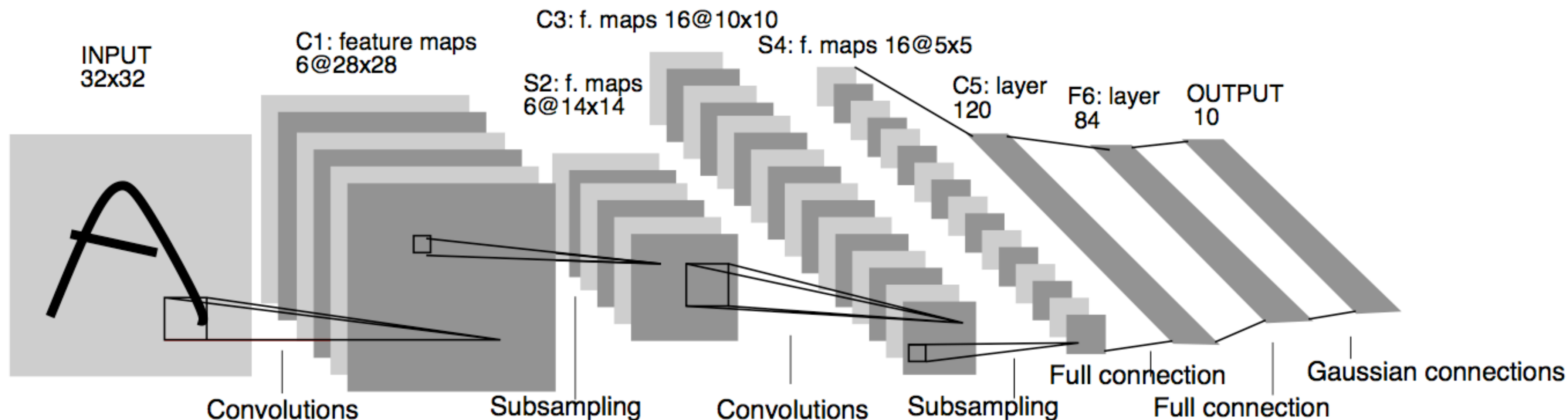


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

Demos:

<http://yann.lecun.com/exdb/lenet/index.html>

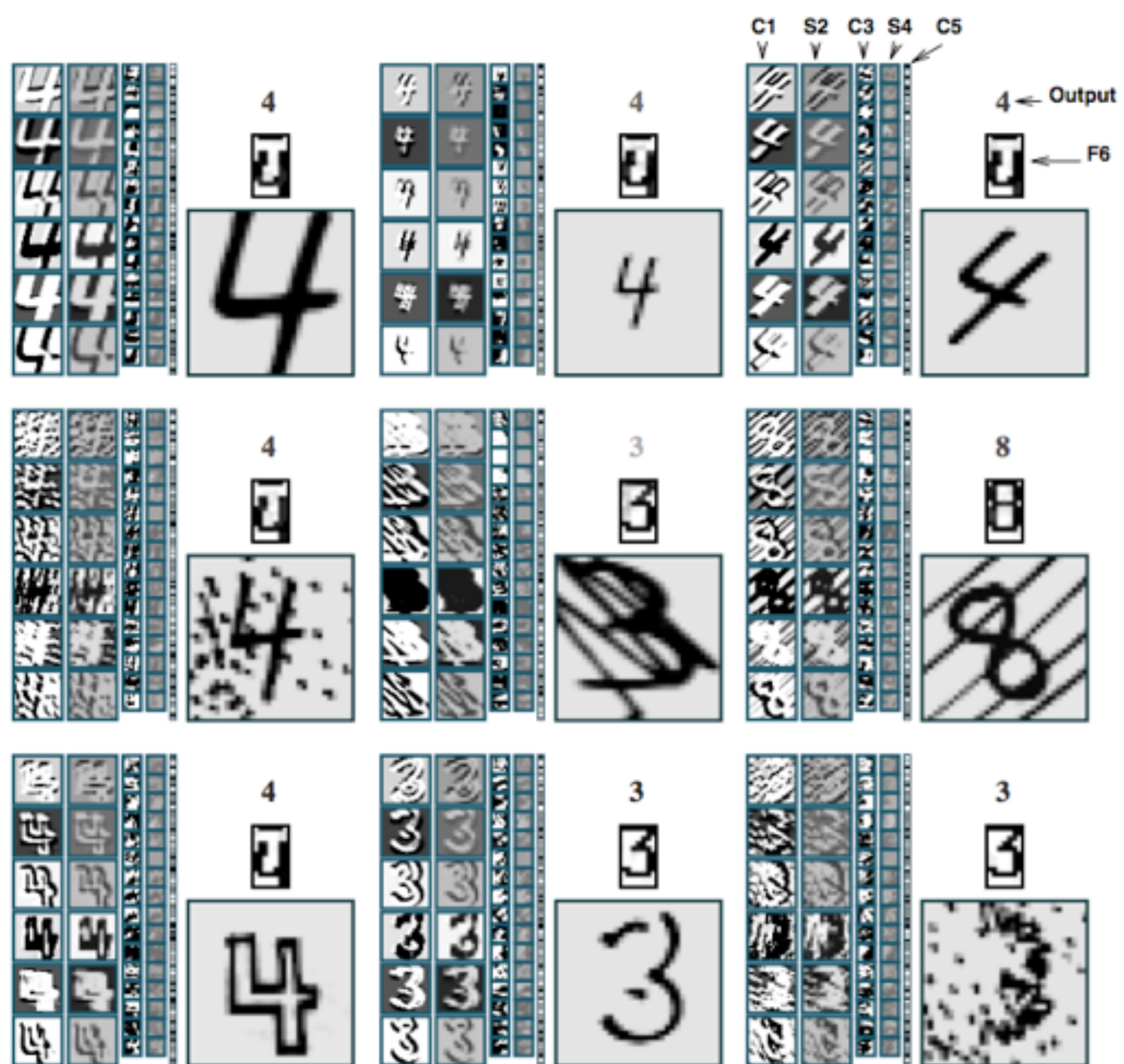
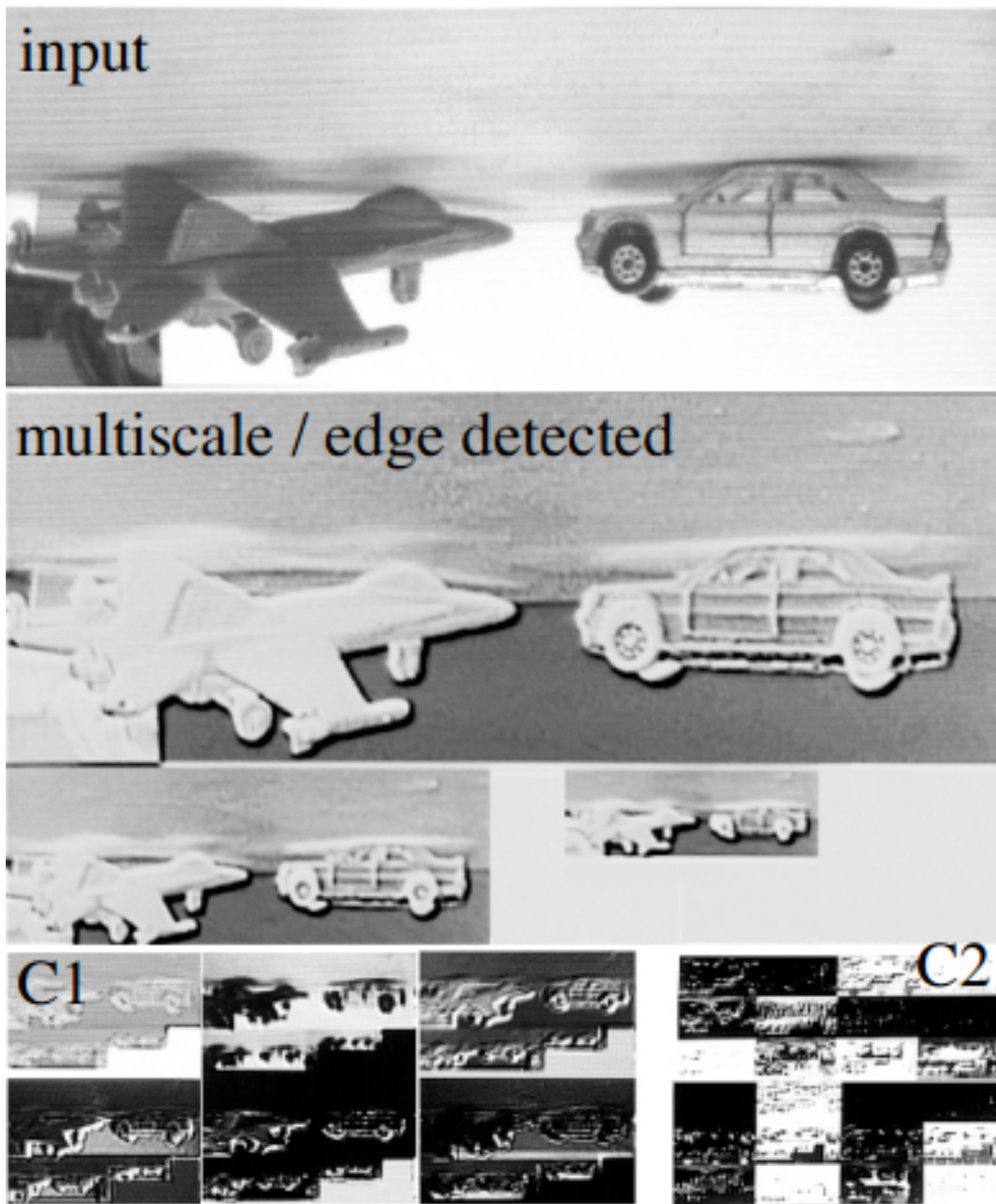


Fig. 13. Examples of unusual, distorted, and noisy characters correctly recognized by LeNet-5. The grey-level of the output label represents the penalty (lighter for higher penalties).

Source: Isola, Torralba, Freeman

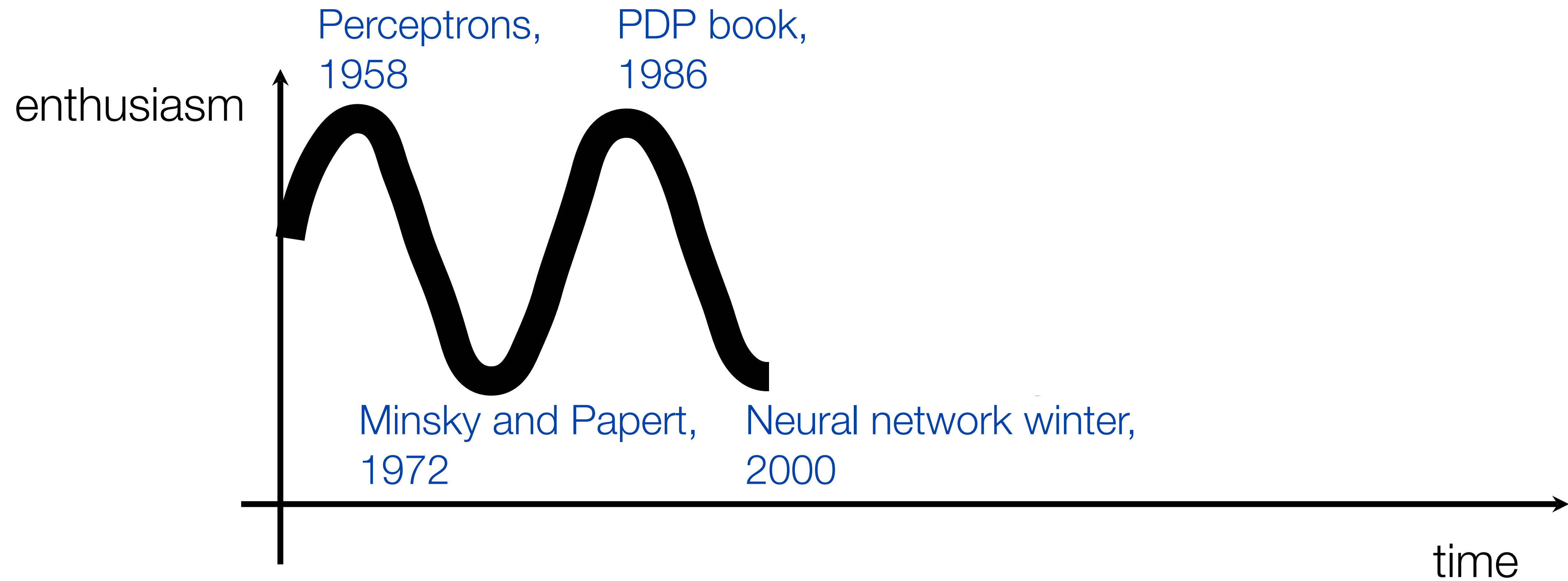


Neural networks to
recognize
handwritten digits
and human faces?
yes

Neural networks for
tougher problems?
not really

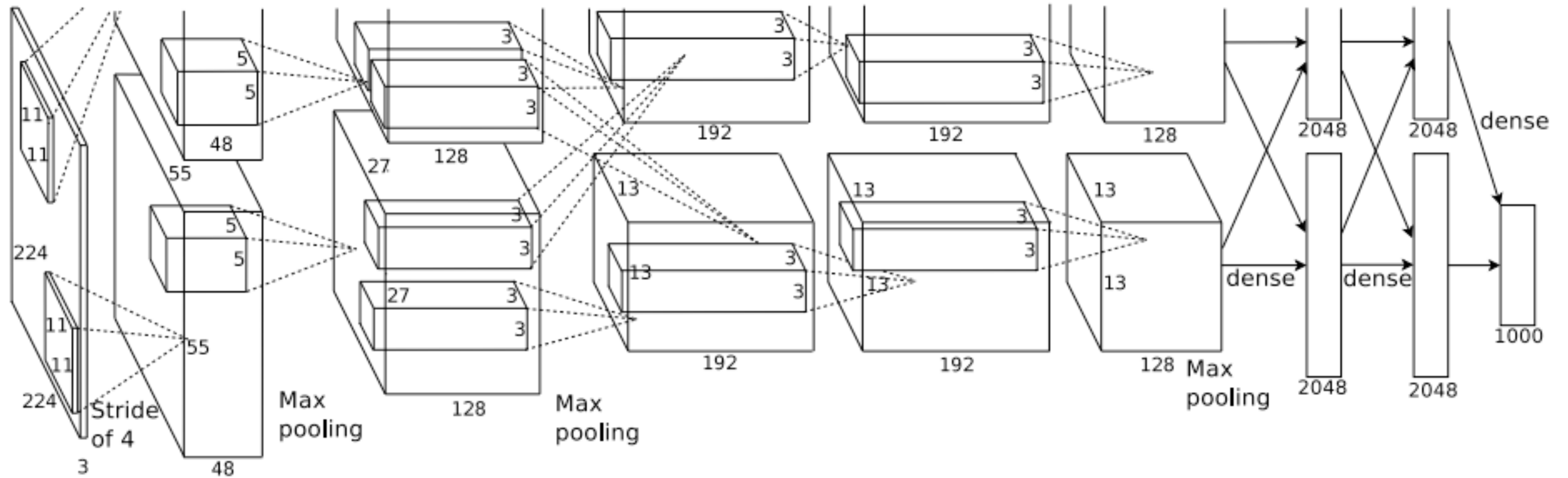
Machine learning circa 2000

- Neural Information Processing Systems (NeurIPS), is a top conference on machine learning.
- For the 2000 conference:
 - title words predictive of paper acceptance: “Belief Propagation” and “Gaussian”.
 - title words predictive of paper rejection: “Neural” and “Network”.

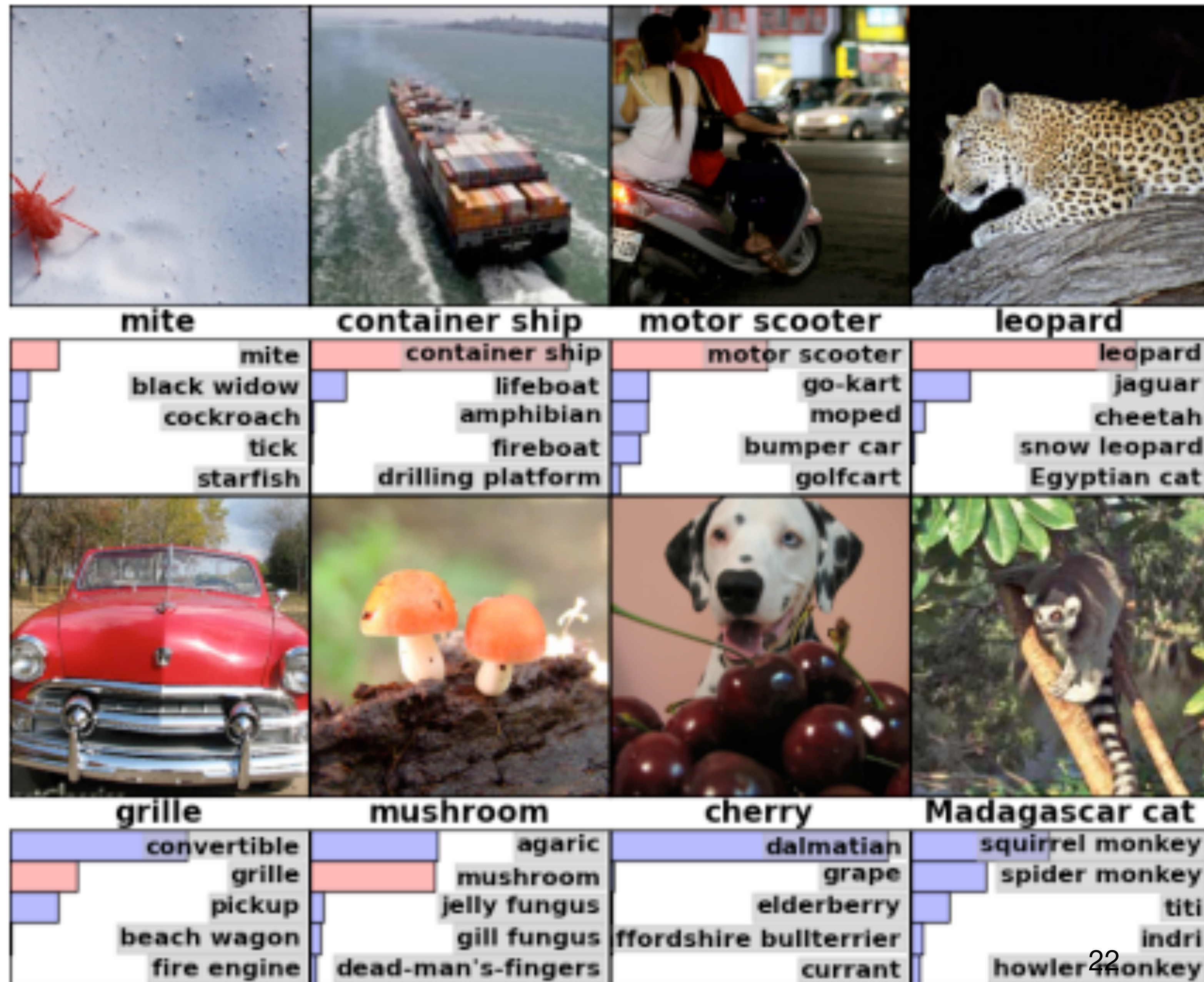


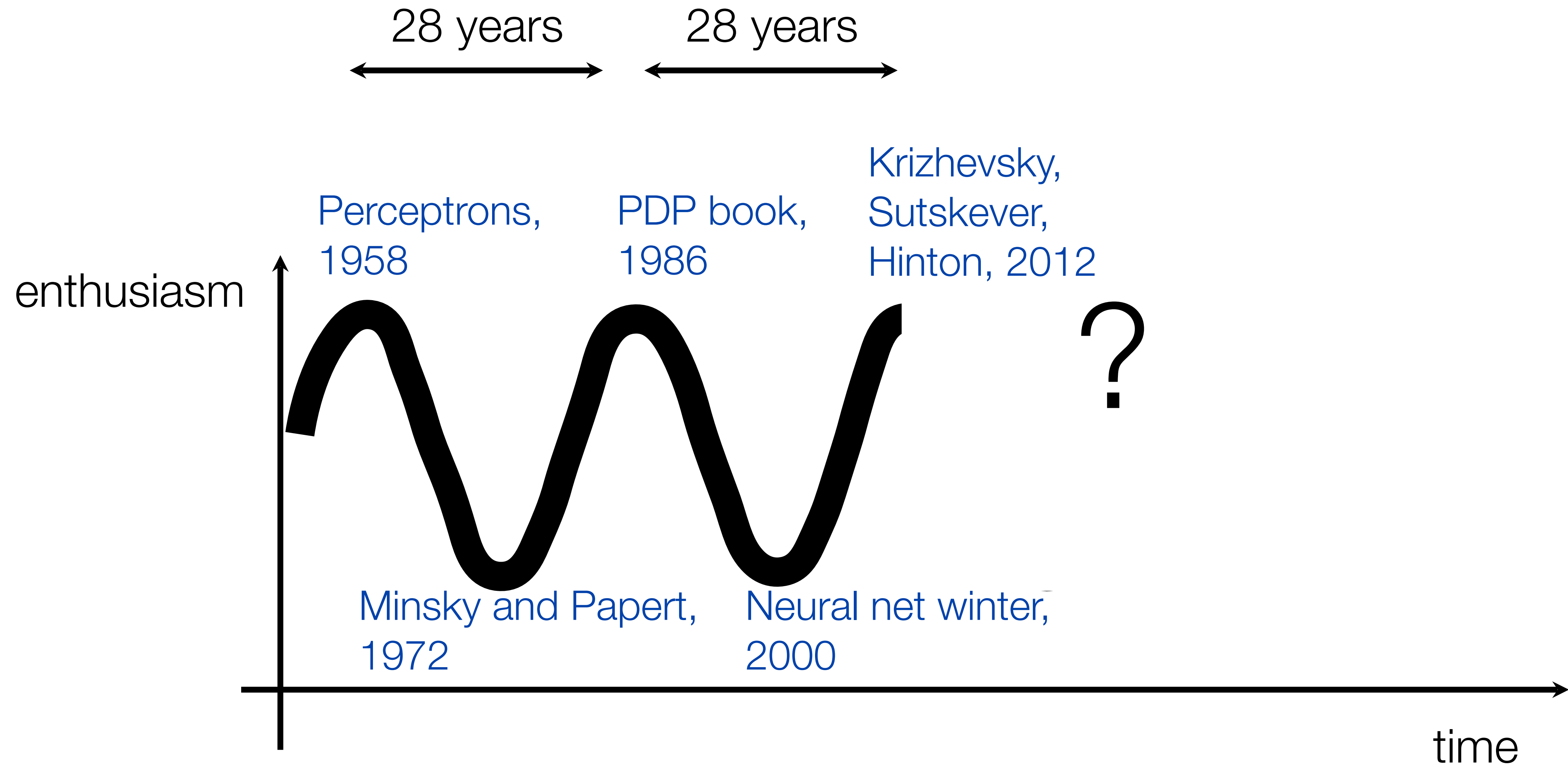
Krizhevsky, Sutskever, and Hinton, NeurIPS 2012

“AlexNet”

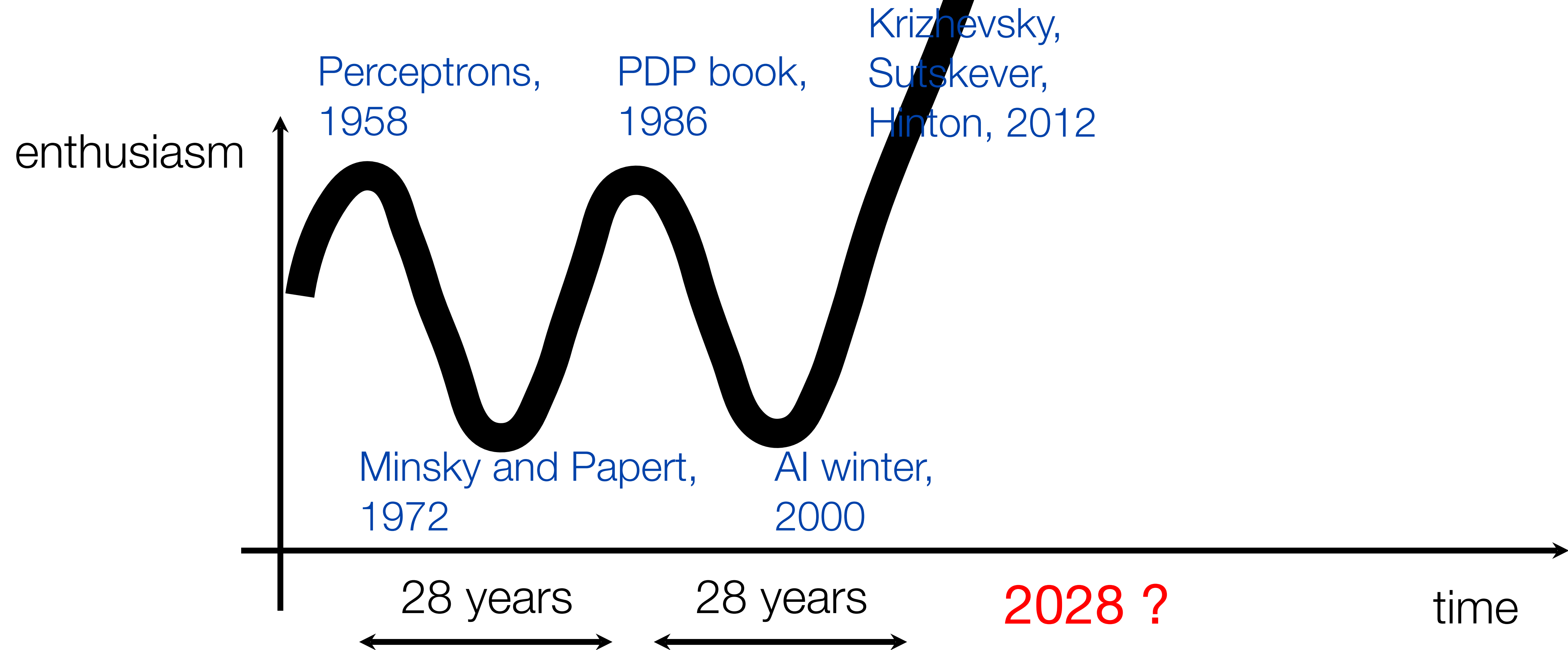


Krizhevsky, Sutskever, and Hinton, NeurIPS 2012

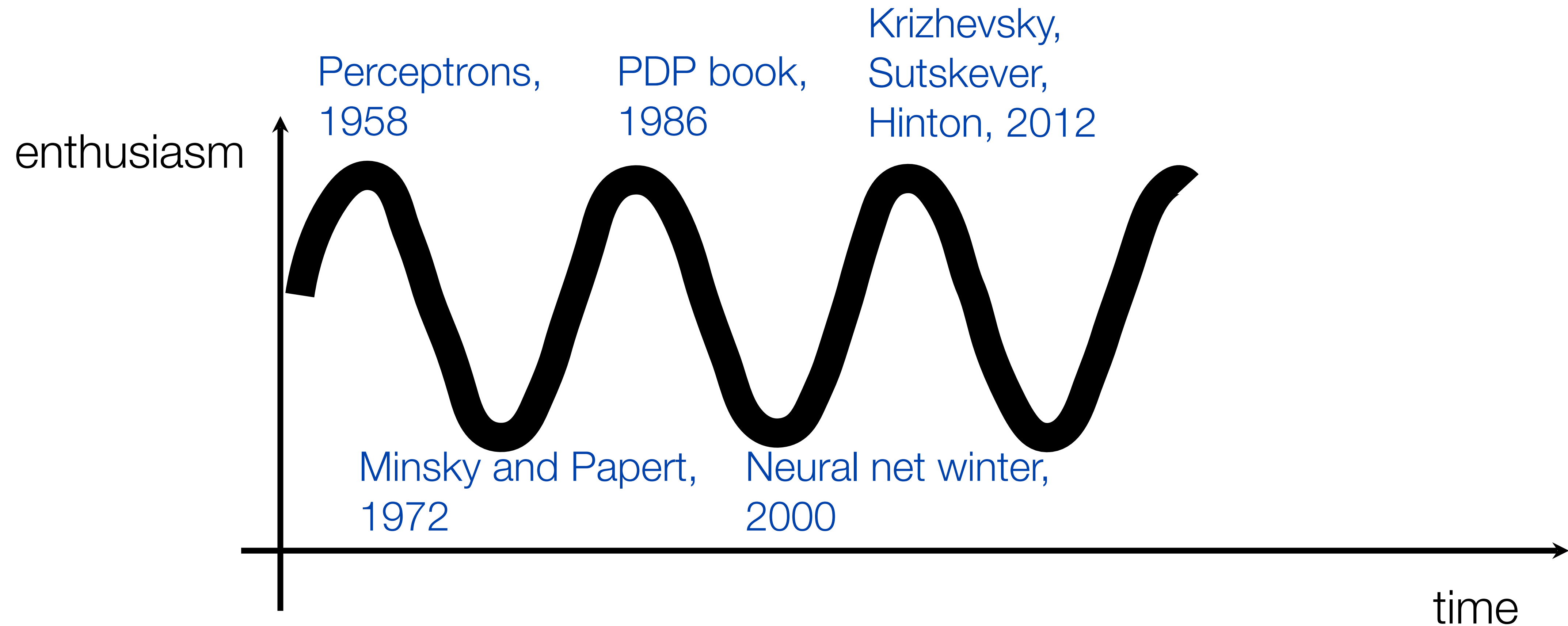




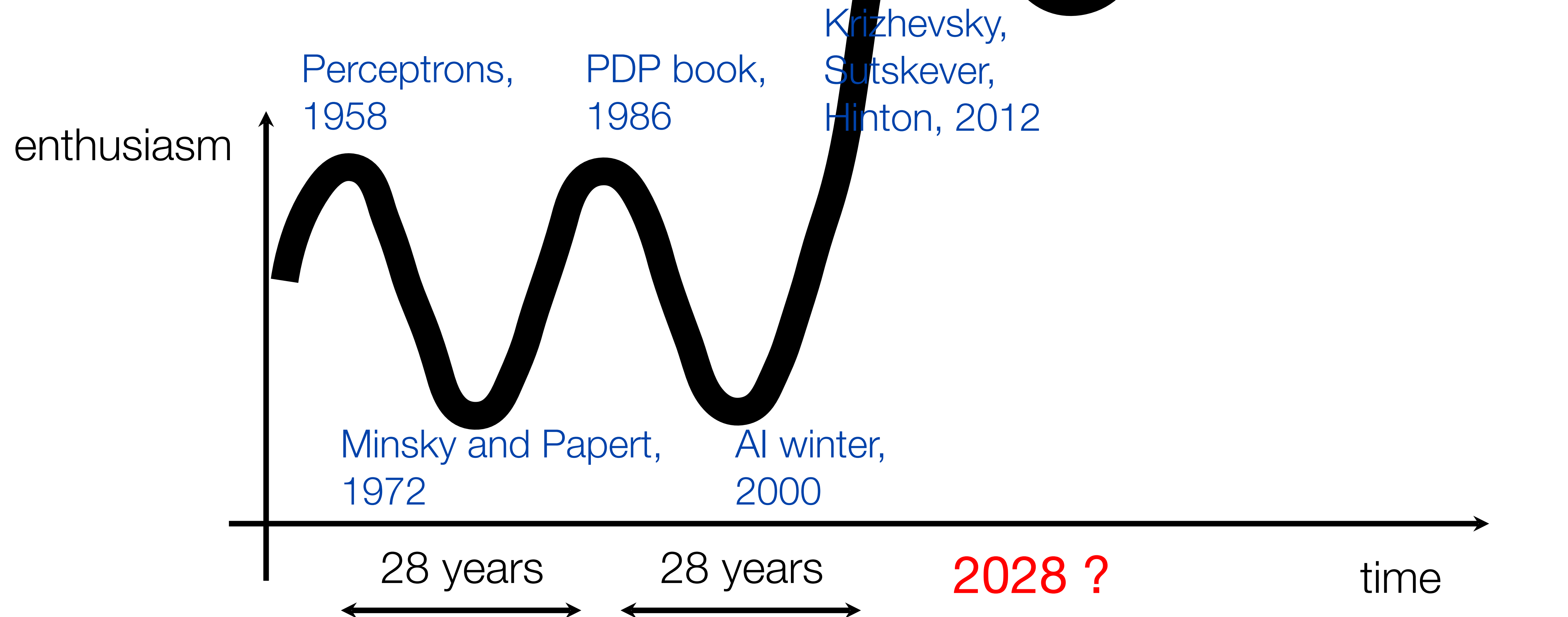
What comes next?



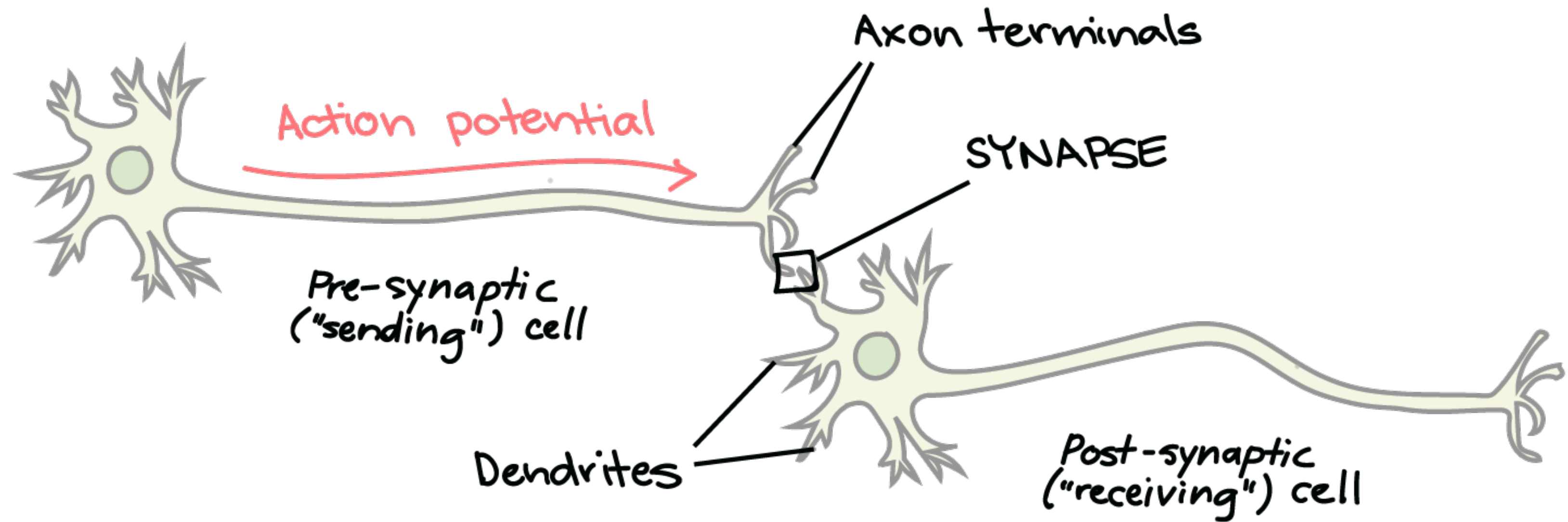
What comes next?



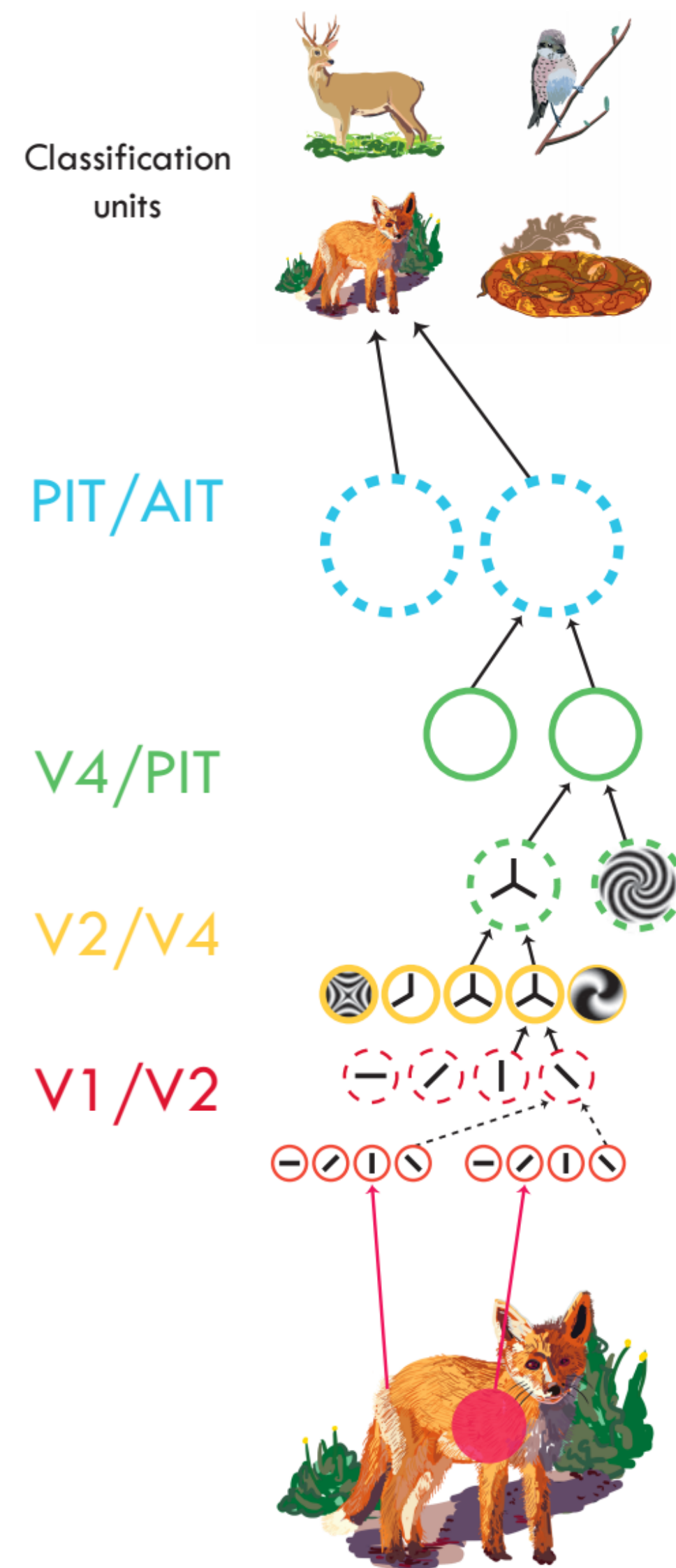
What comes next?



Inspiration: Neurons



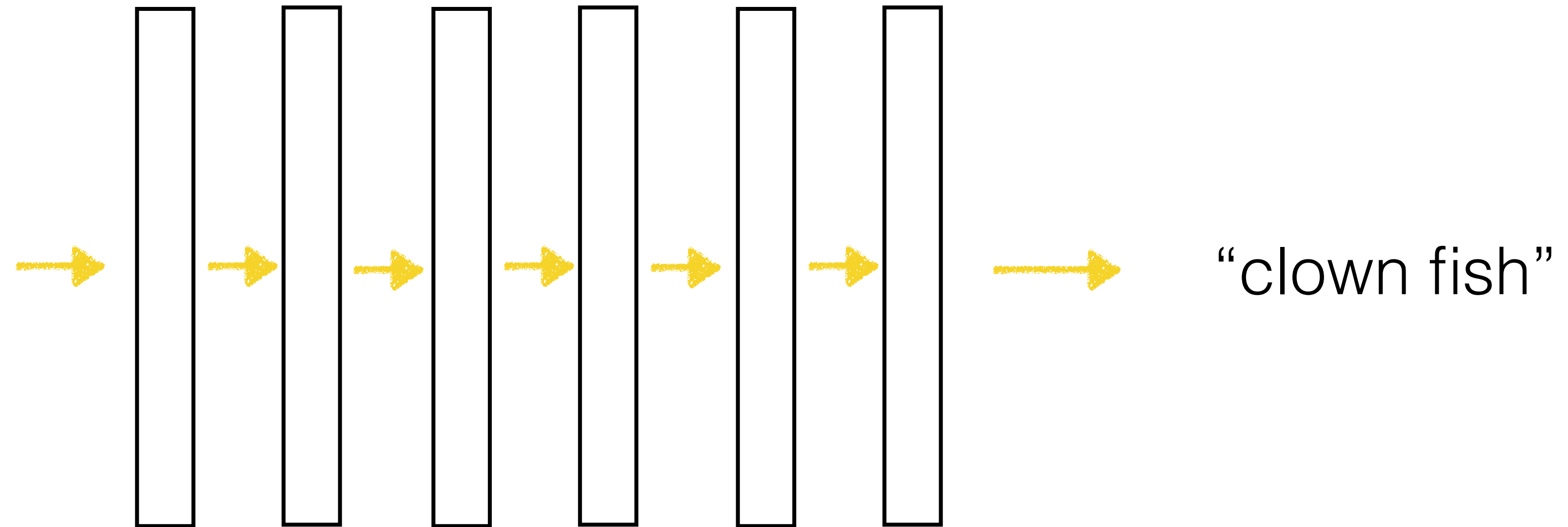
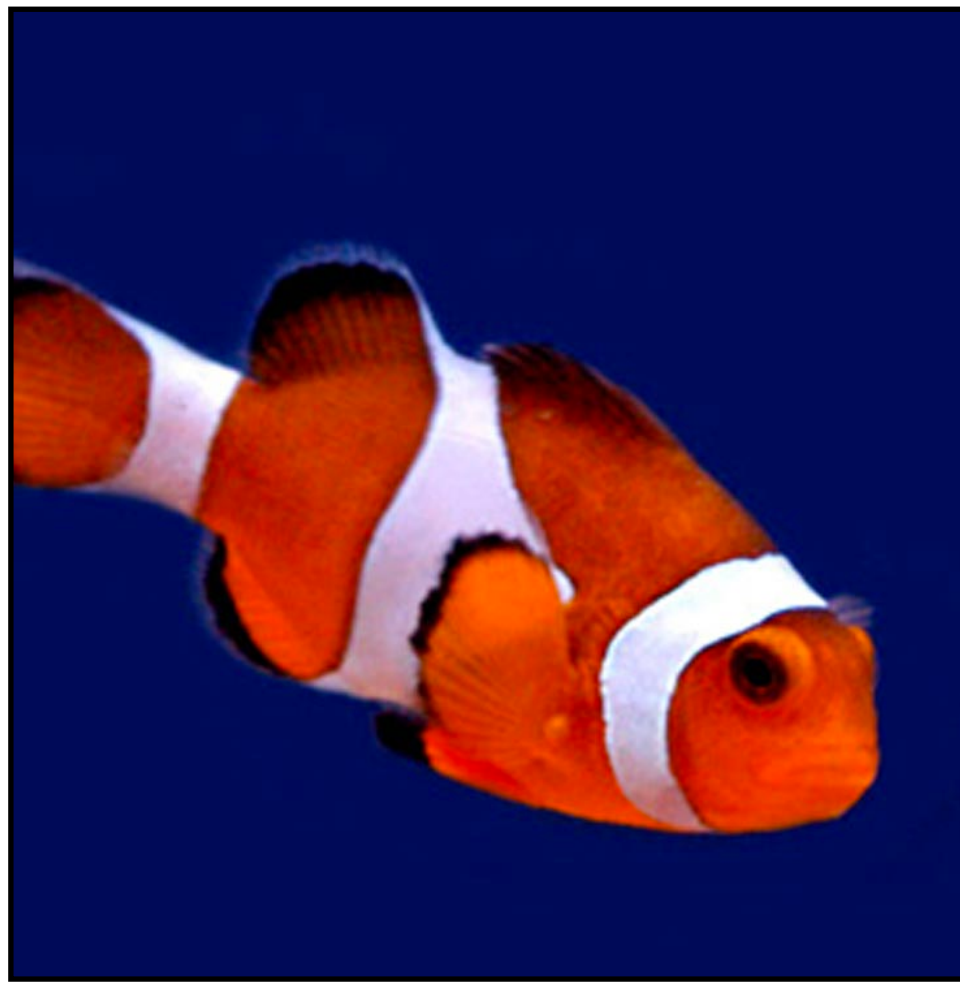
Inspiration: Hierarchical Representations



Best to treat as *inspiration*. The neural nets we'll talk about aren't very biologically plausible.

[Serre, 2014]

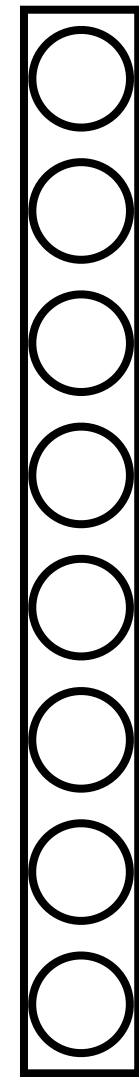
Object recognition



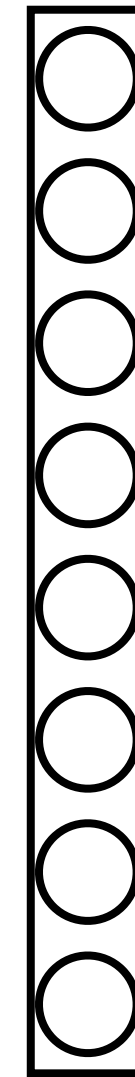
Neural network

Computation in a neural net

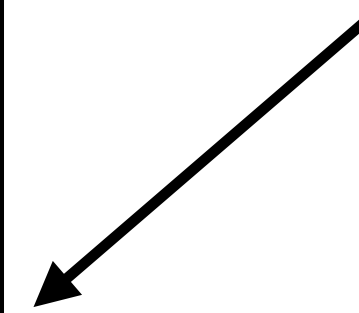
Input vector



Output vector

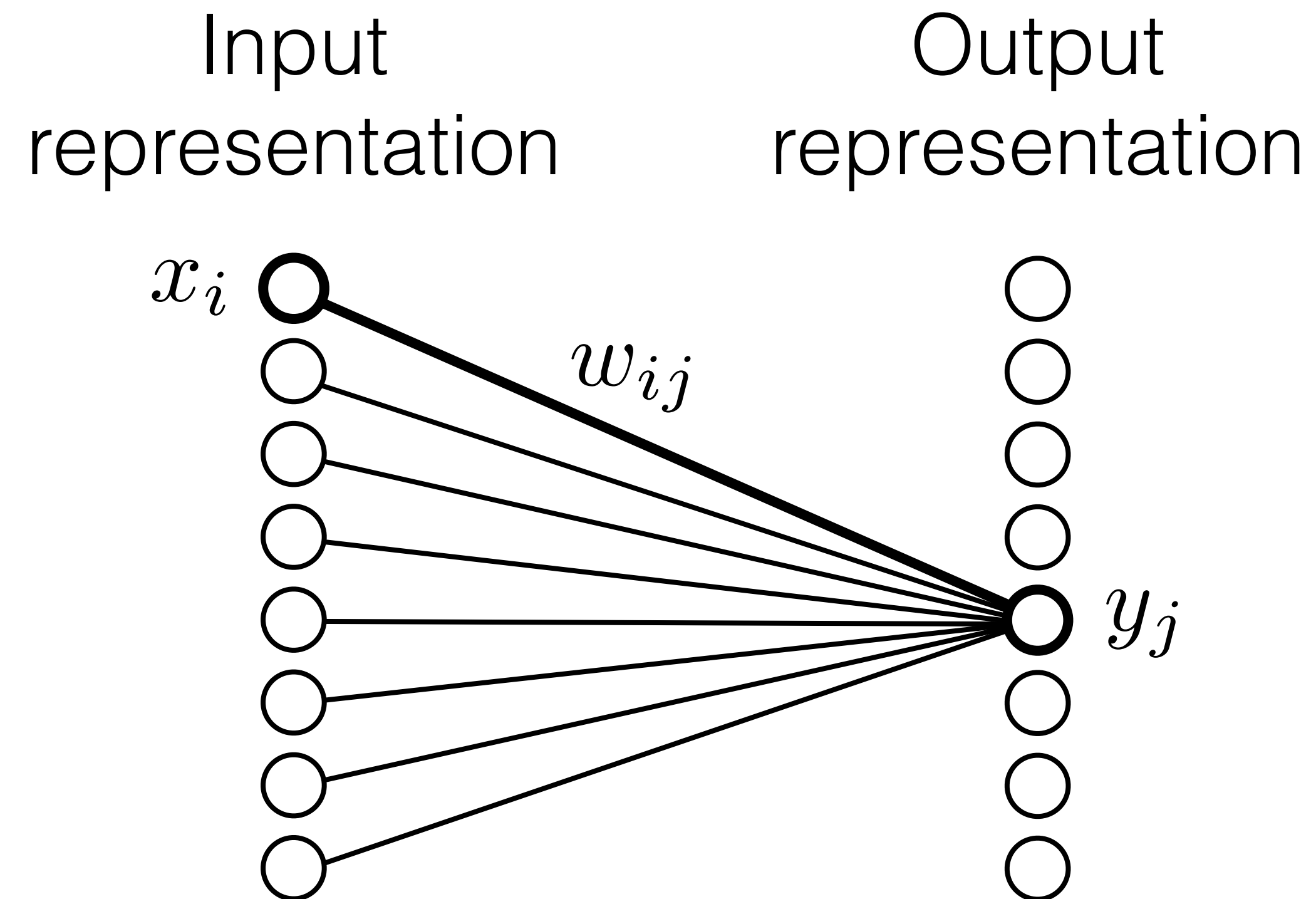


Neuron
(a.k.a unit)



Computation in a neural net

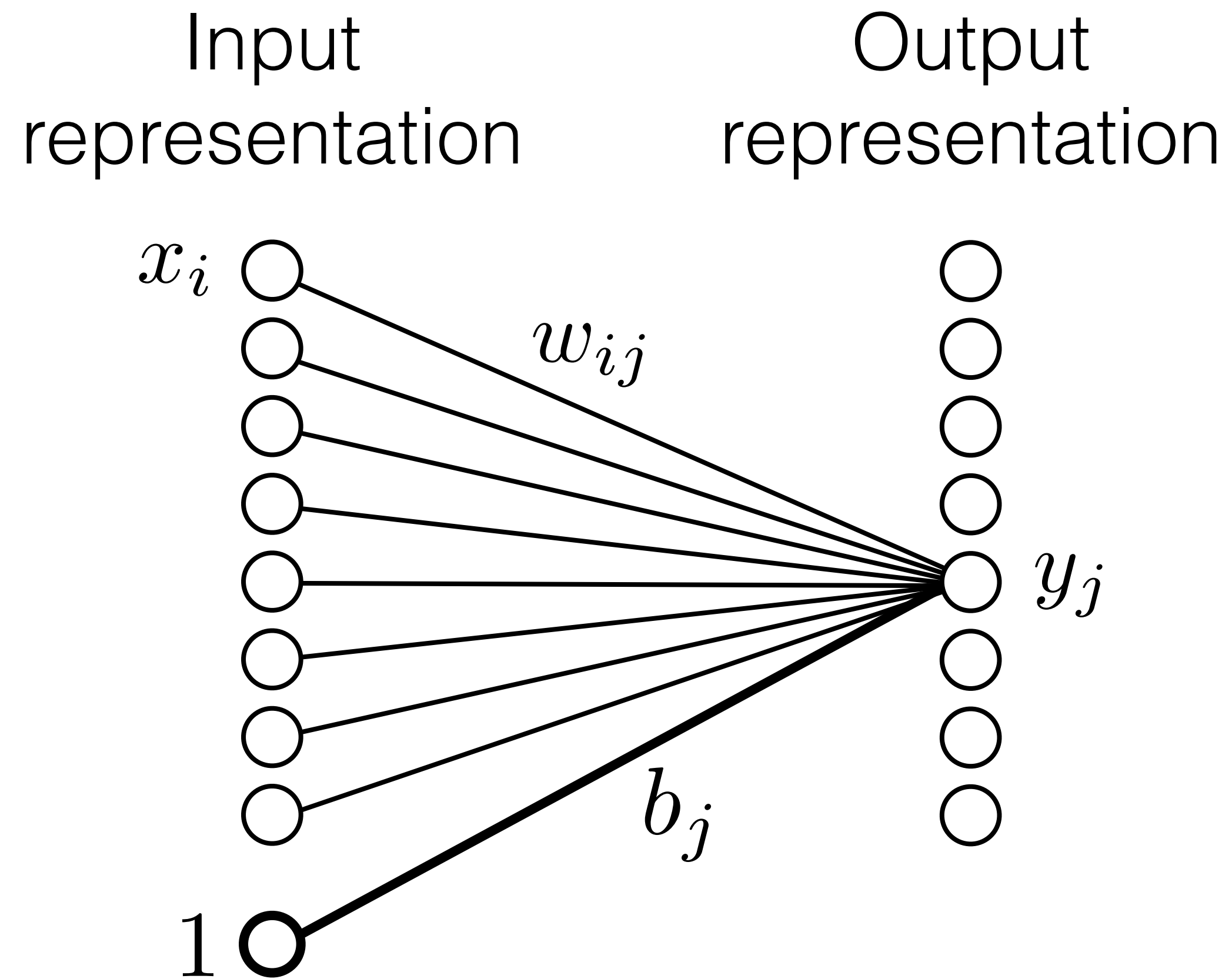
Linear layer



$$y_j = \sum_i w_{ij} x_i$$

Computation in a neural net

Linear layer



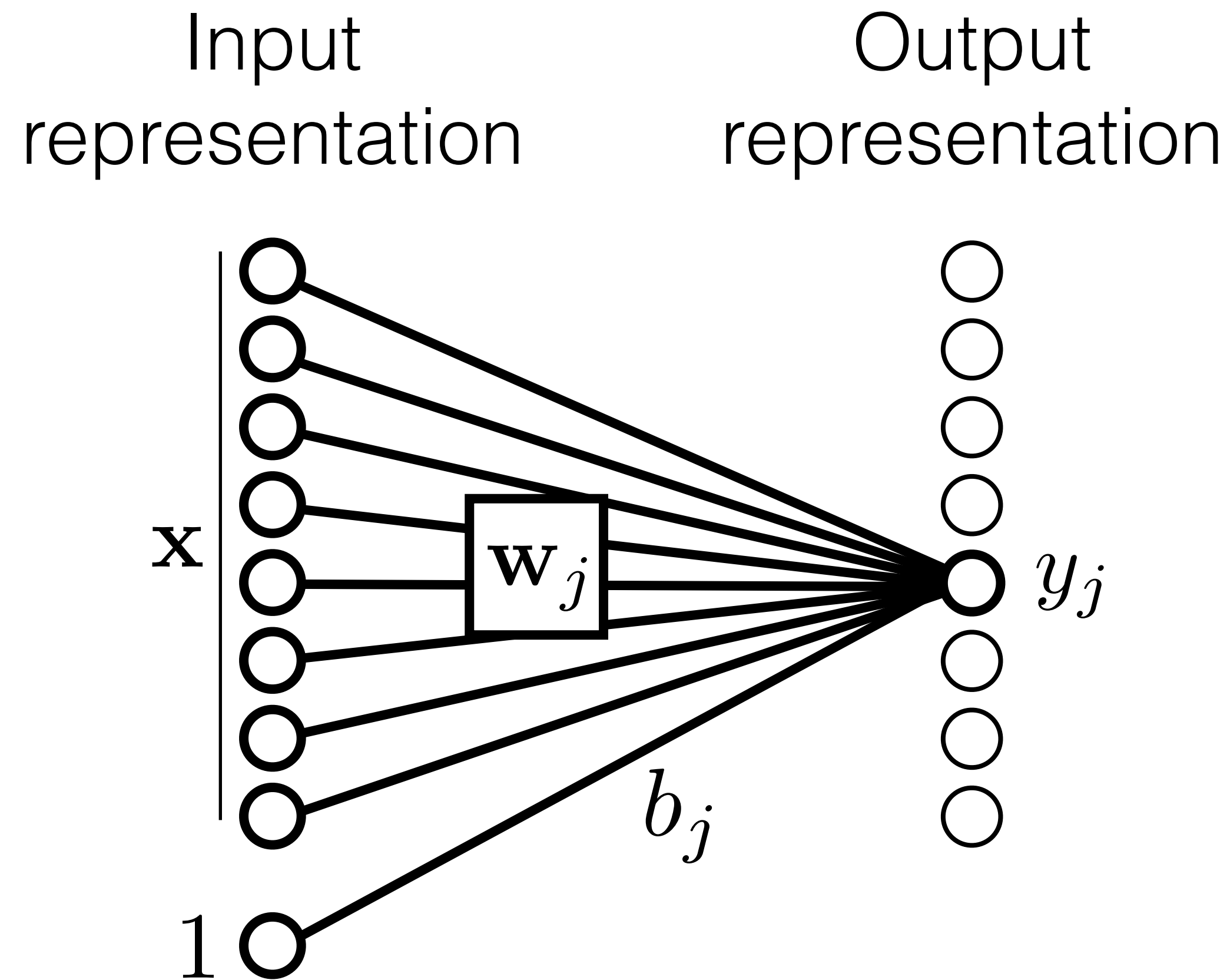
$$y_j = \sum_i w_{ij} x_i + b_j$$

weights

bias

Computation in a neural net

Linear layer



$$y_j = \mathbf{x}^T \mathbf{w}_j + b_j$$

weights

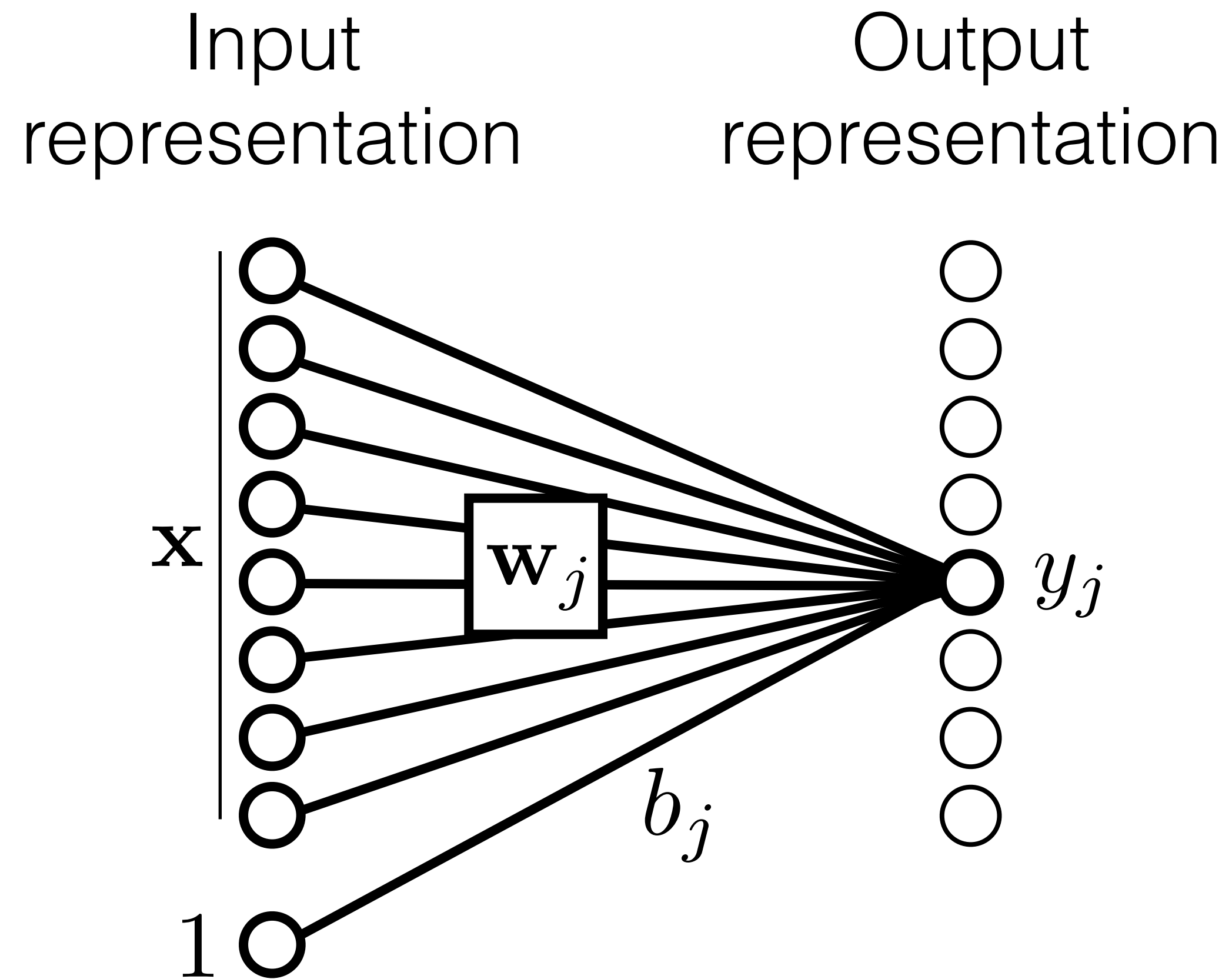
bias

$$\theta = \{\mathbf{W}, \mathbf{b}\}$$

parameters of the model

Computation in a neural net

Linear layer



Full layer

weights

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

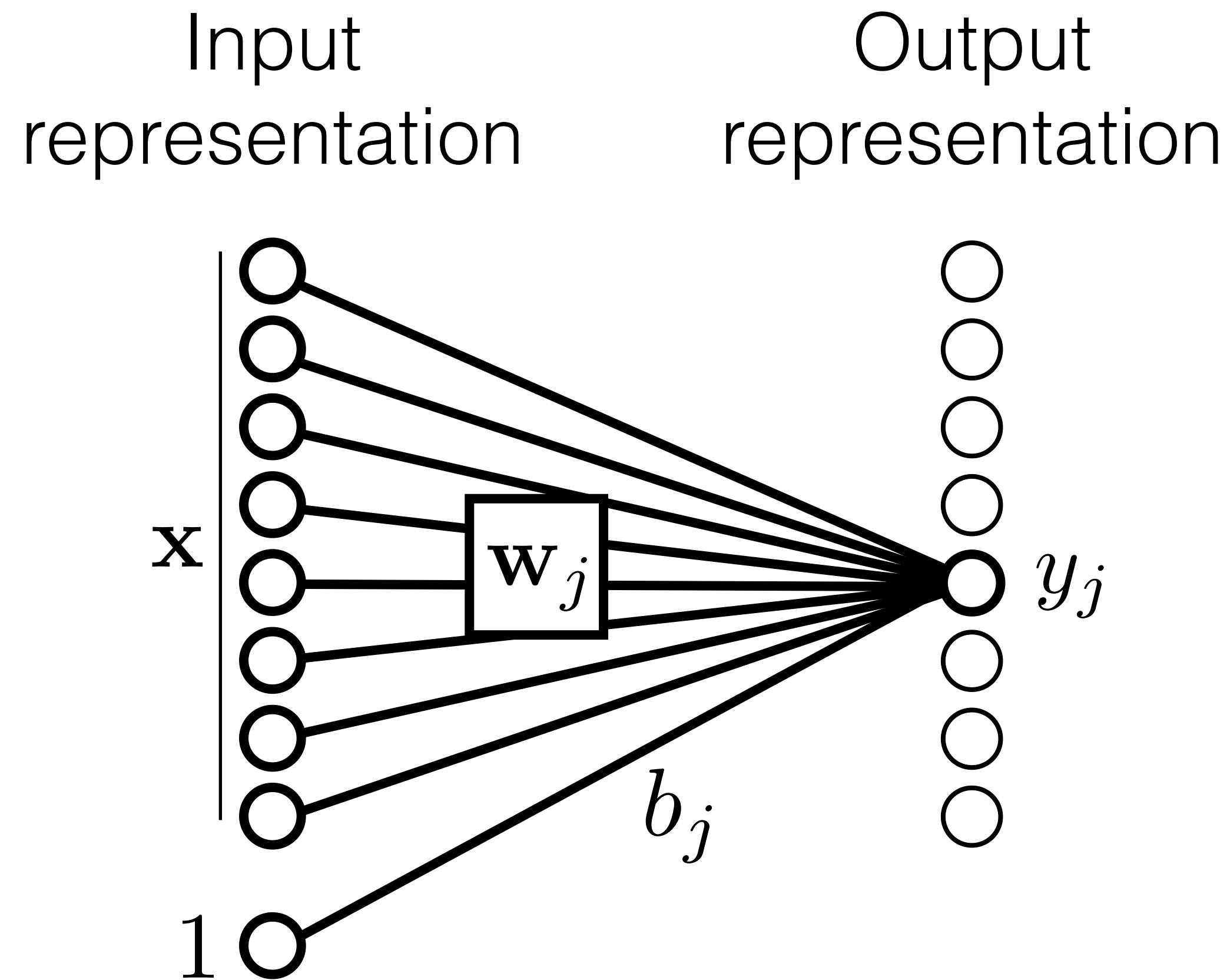
bias

$$\theta = \{\mathbf{W}, \mathbf{b}\}$$

parameters of the model

Computation in a neural net

Linear layer



Full layer

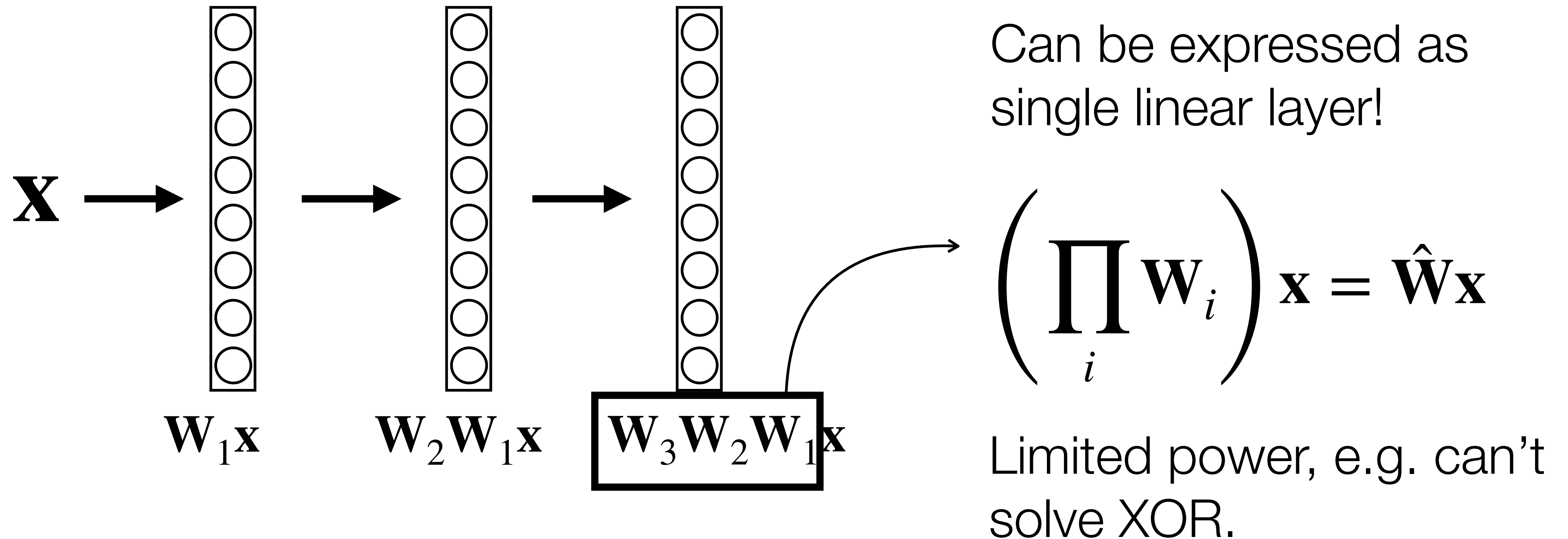
weights + bias

$$\mathbf{y} = \mathbf{W}\mathbf{x}$$

Can again simplify notation by appending a 1 to $\mathbf{x}$

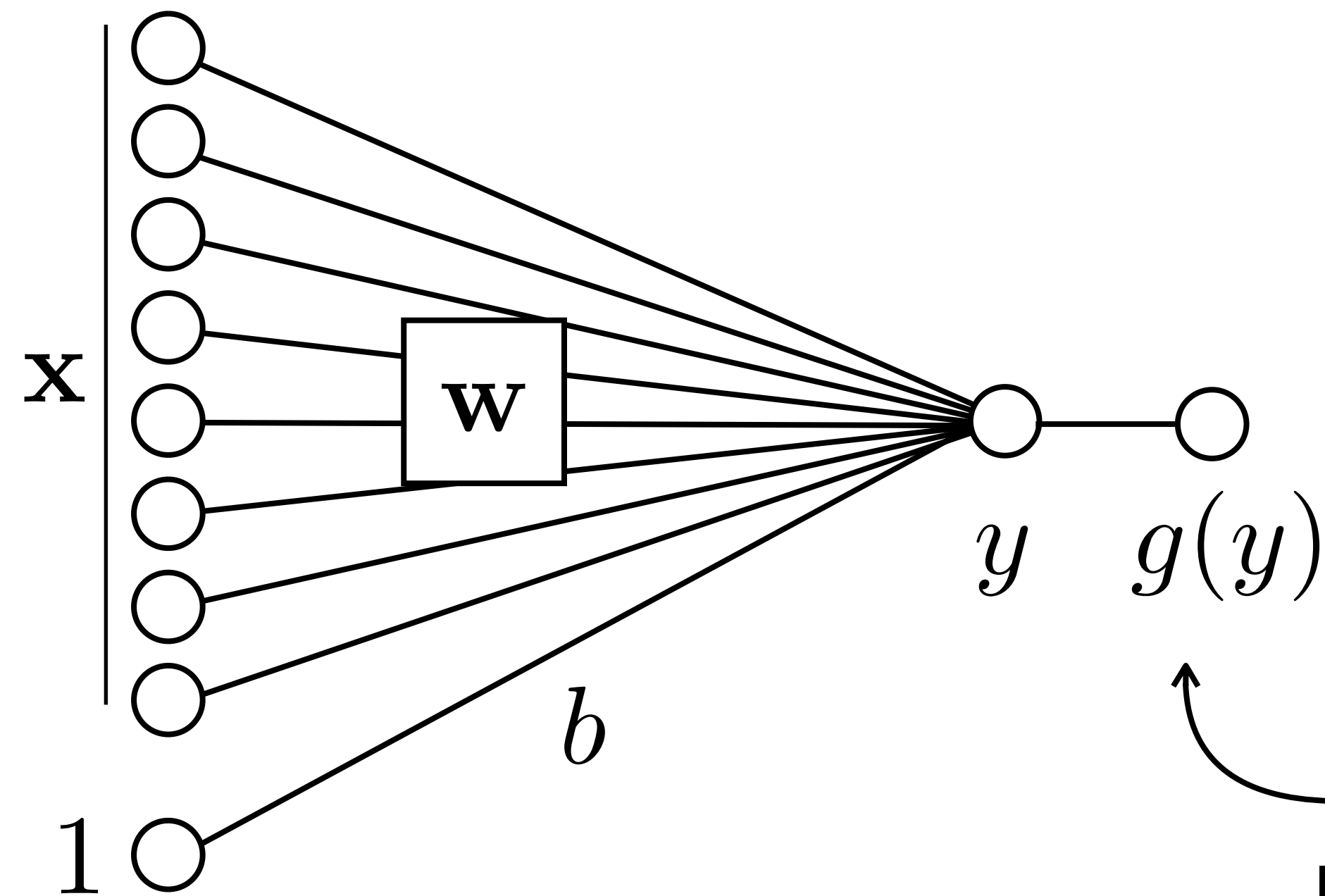
What's the problem with this idea?

Consider stacking multiple layers:



Solution: simple nonlinearity

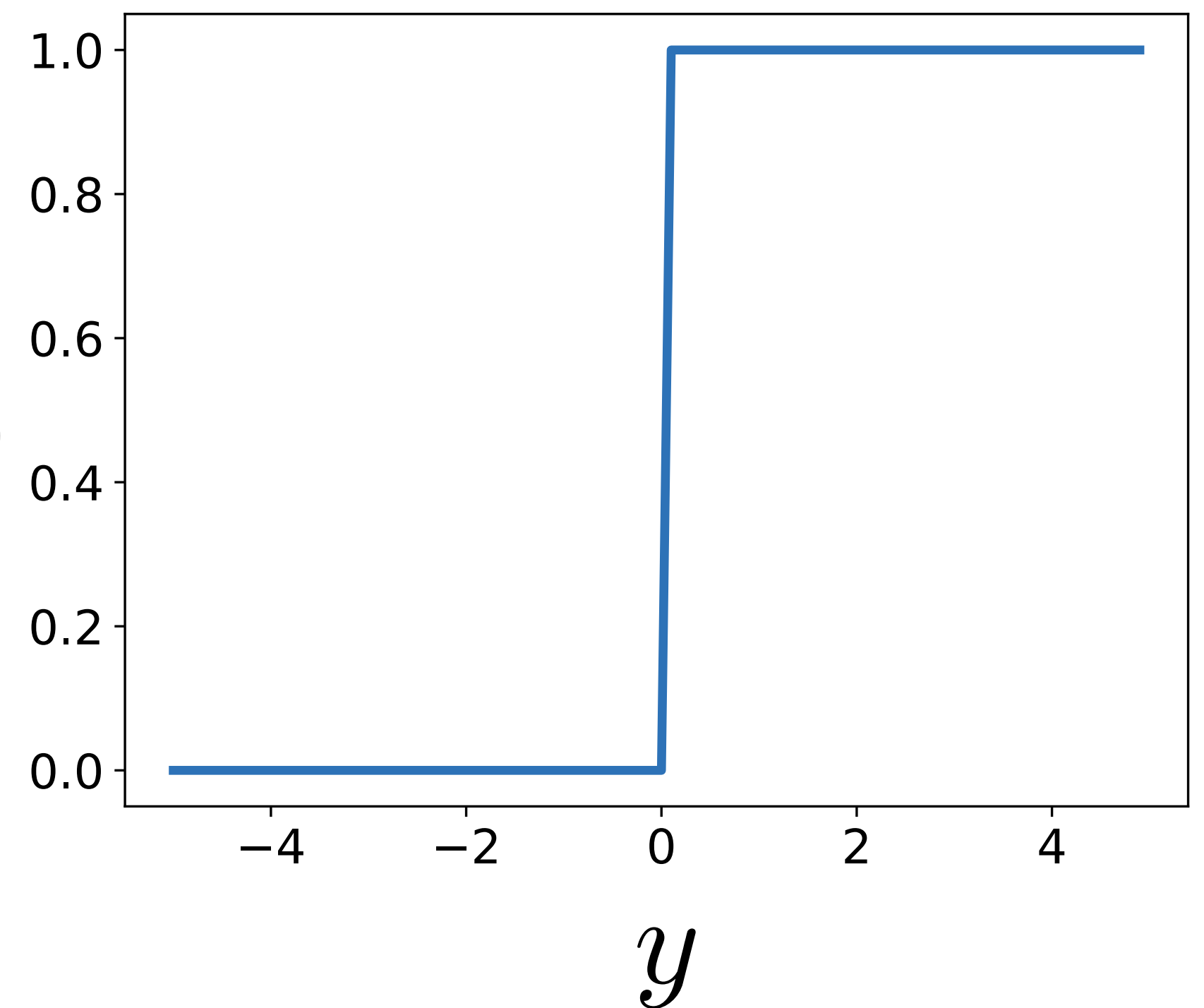
Input
representation



Output
representation

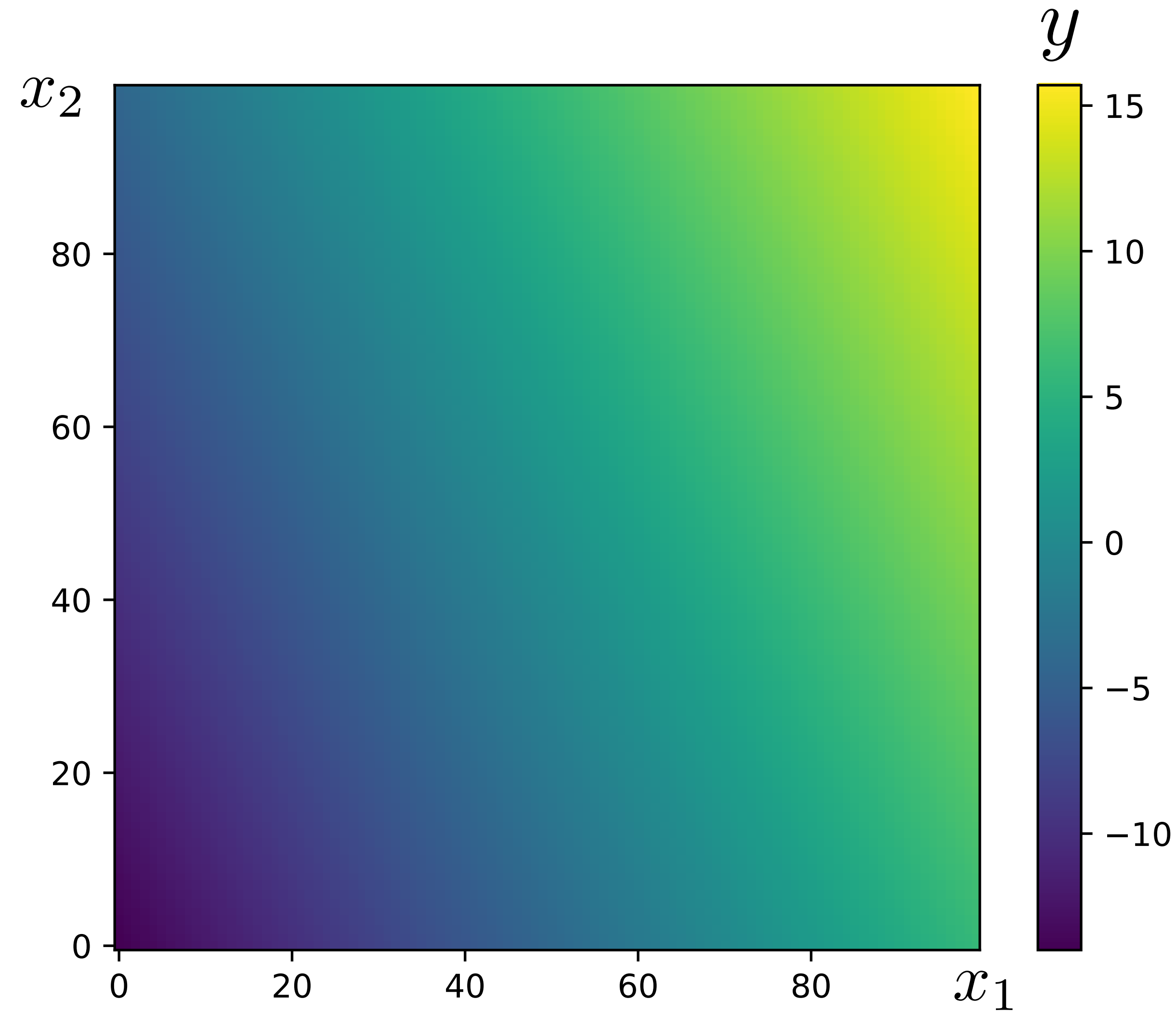
$$g(y) = \begin{cases} 1, & \text{if } y > 0 \\ 0, & \text{otherwise} \end{cases}$$

$g(y)$



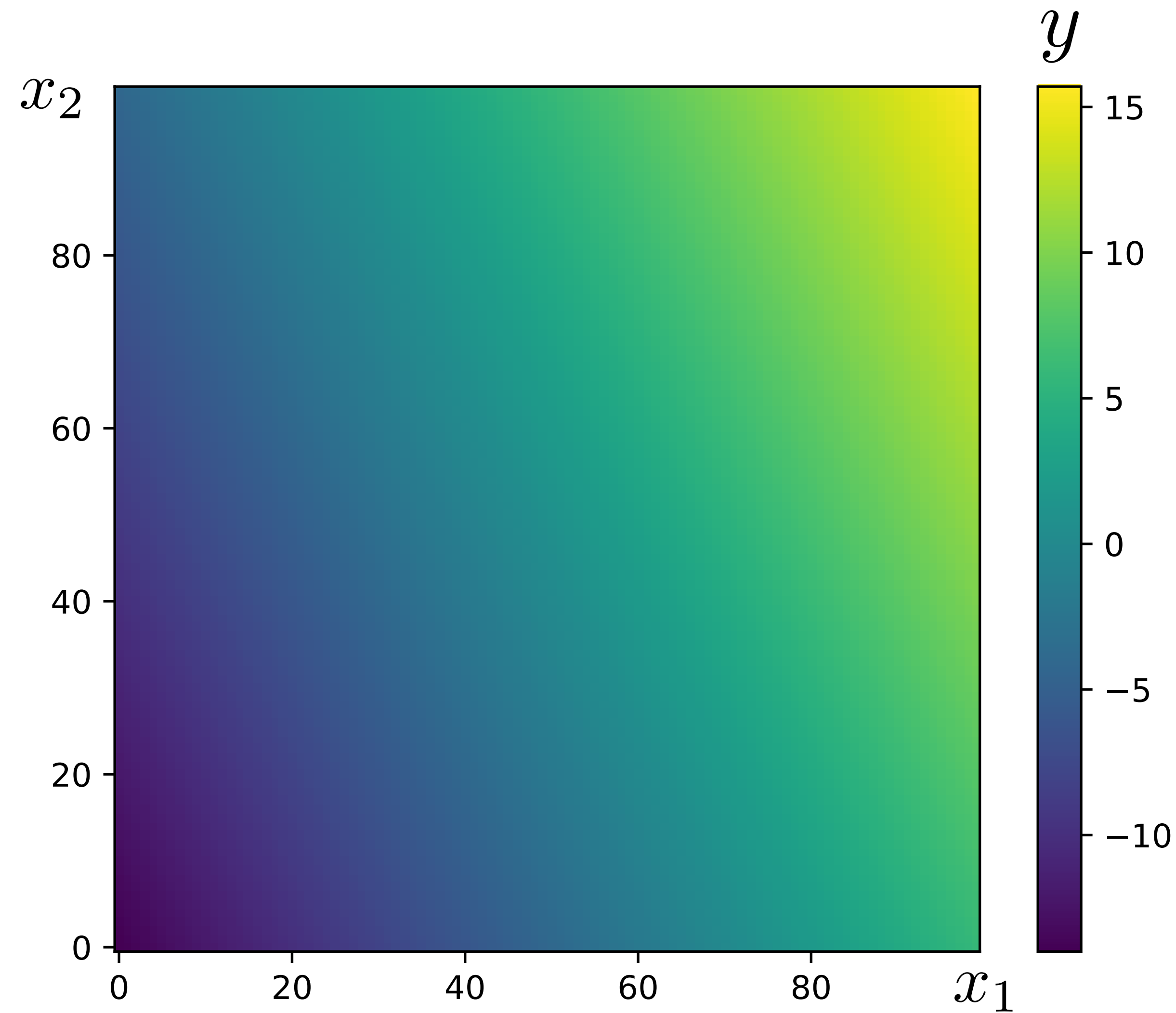
**Pointwise
Non-linearity**

Example: linear classification with a perceptron



$$y = \mathbf{x}^T \mathbf{w} + b$$

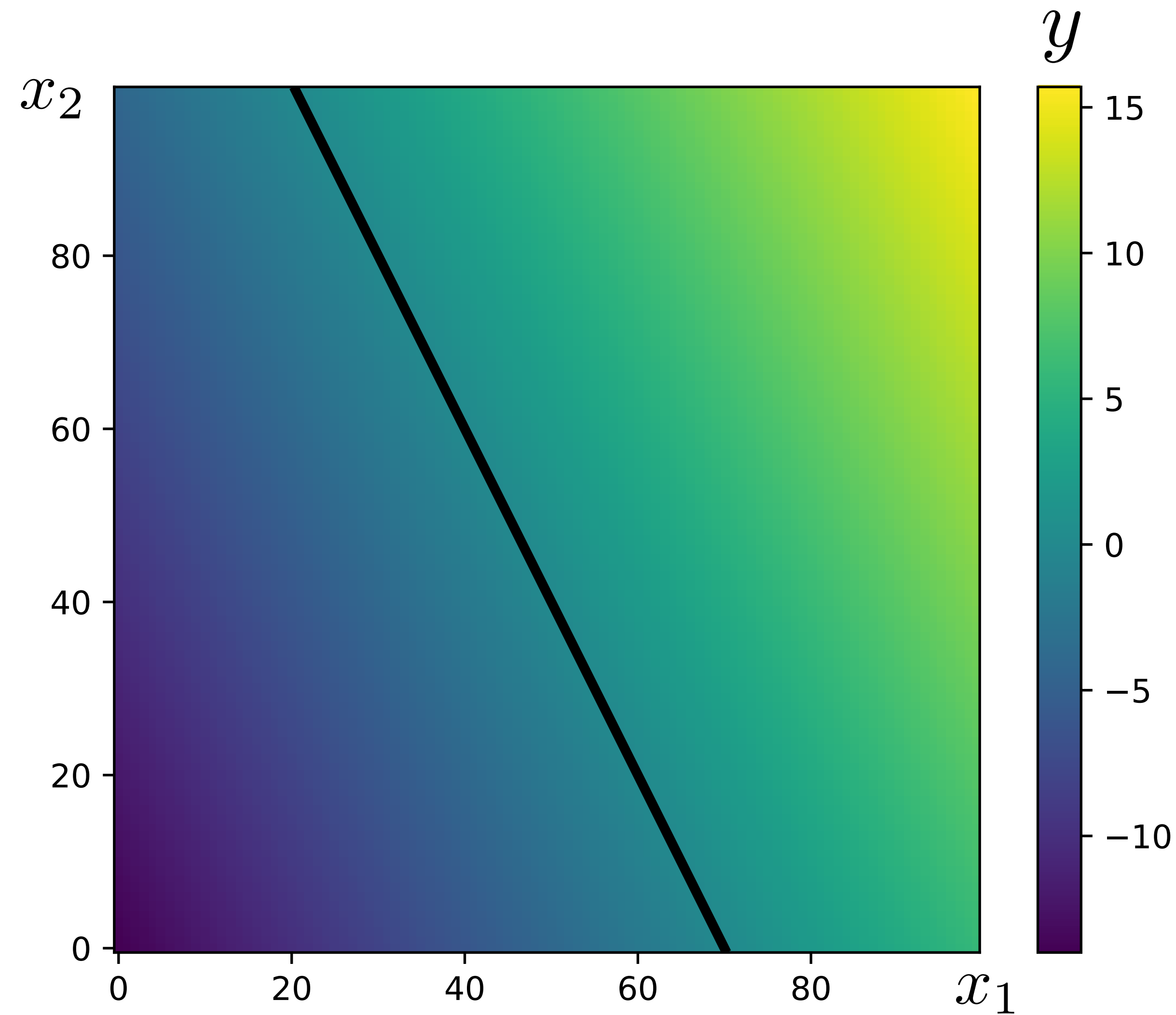
Example: linear classification with a perceptron



$$y = \mathbf{x}^T \mathbf{w} + b$$

$$g(y) = \begin{cases} 1, & \text{if } y > 0 \\ 0, & \text{otherwise} \end{cases}$$

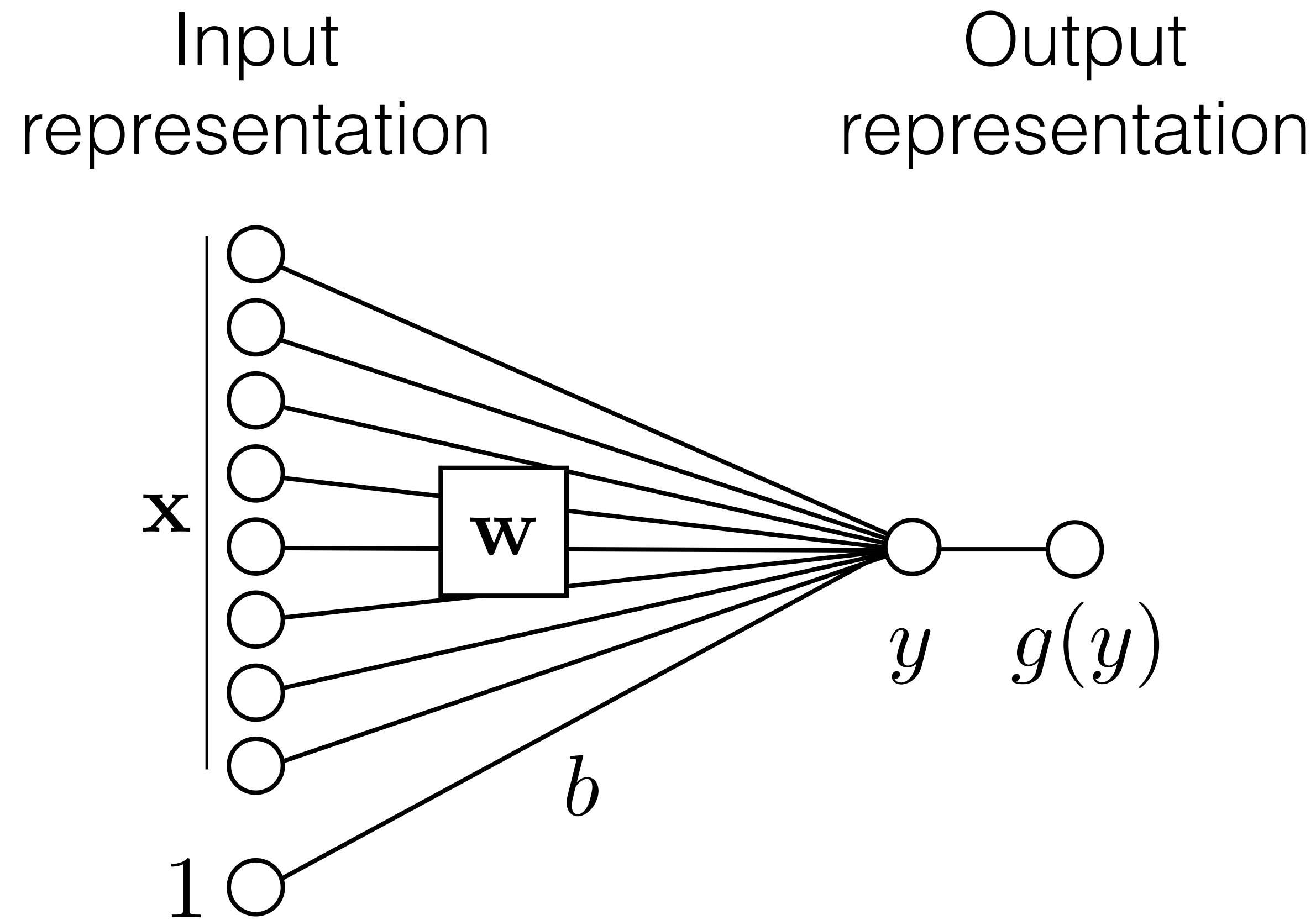
Example: linear classification with a perceptron



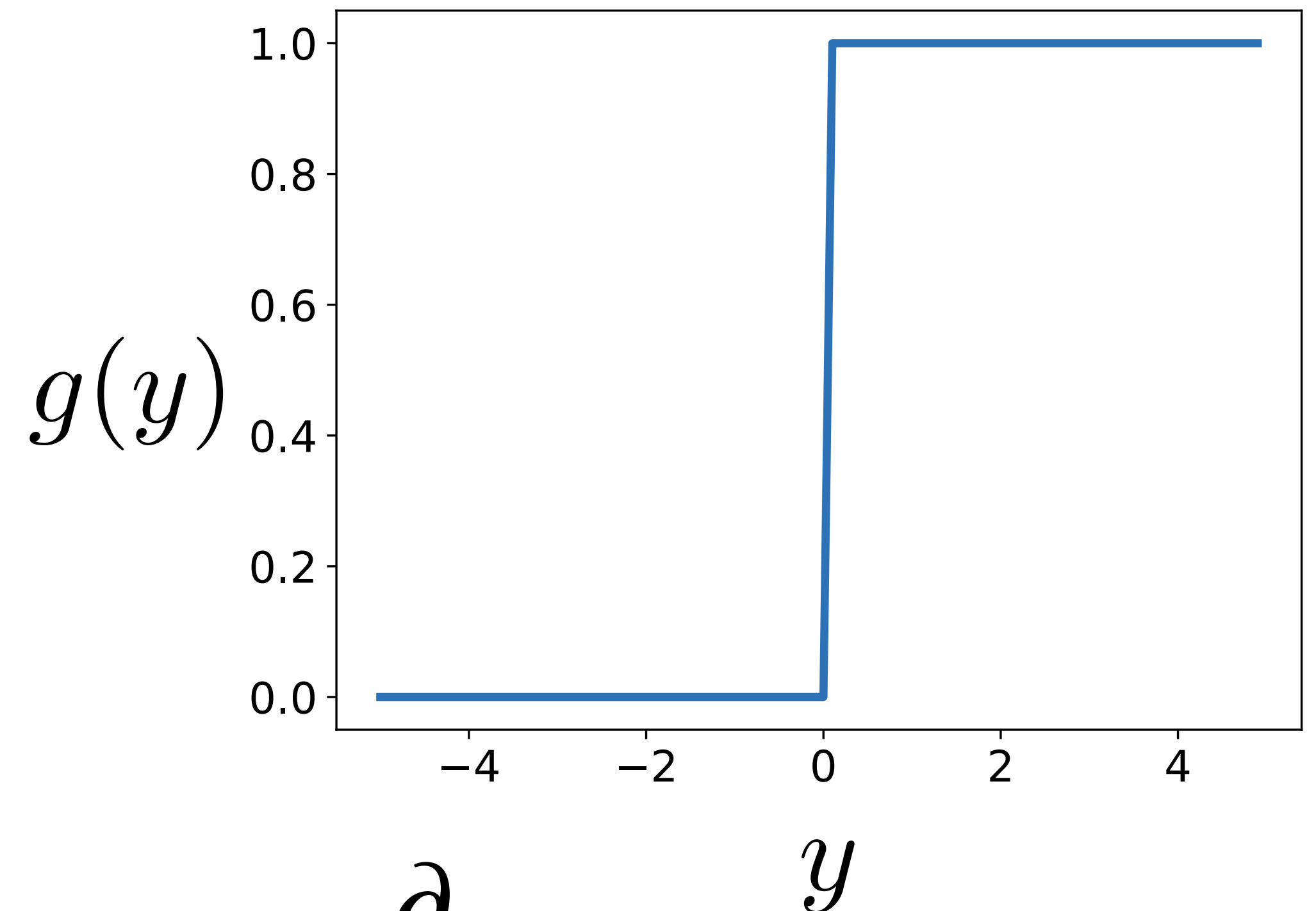
$$y = \mathbf{x}^T \mathbf{w} + b$$

$$g(y) = \begin{cases} 1, & \text{if } y > 0 \\ 0, & \text{otherwise} \end{cases}$$

Computation in a neural net — nonlinearity

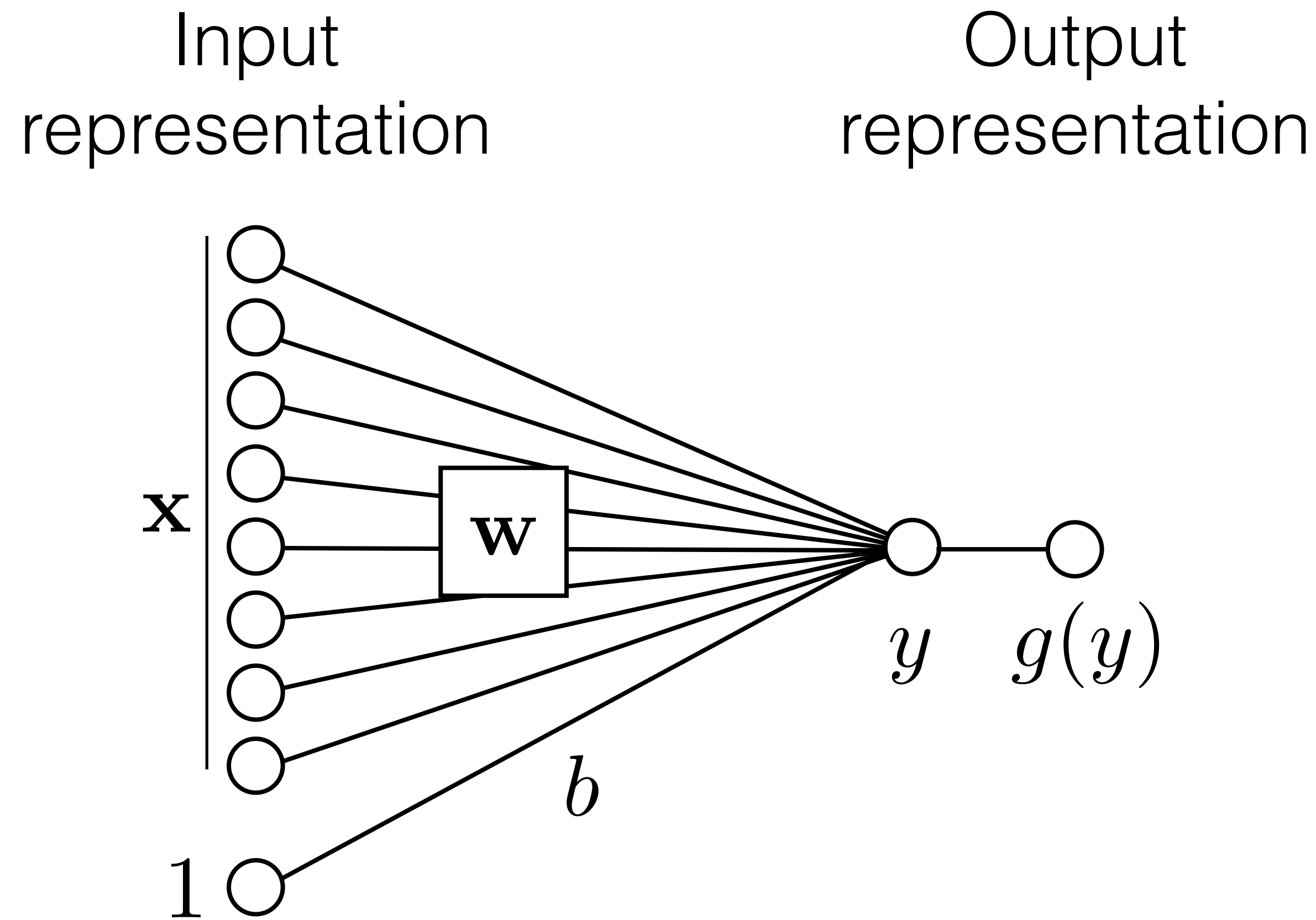


$$g(y) = \begin{cases} 1, & \text{if } y > 0 \\ 0, & \text{otherwise} \end{cases}$$



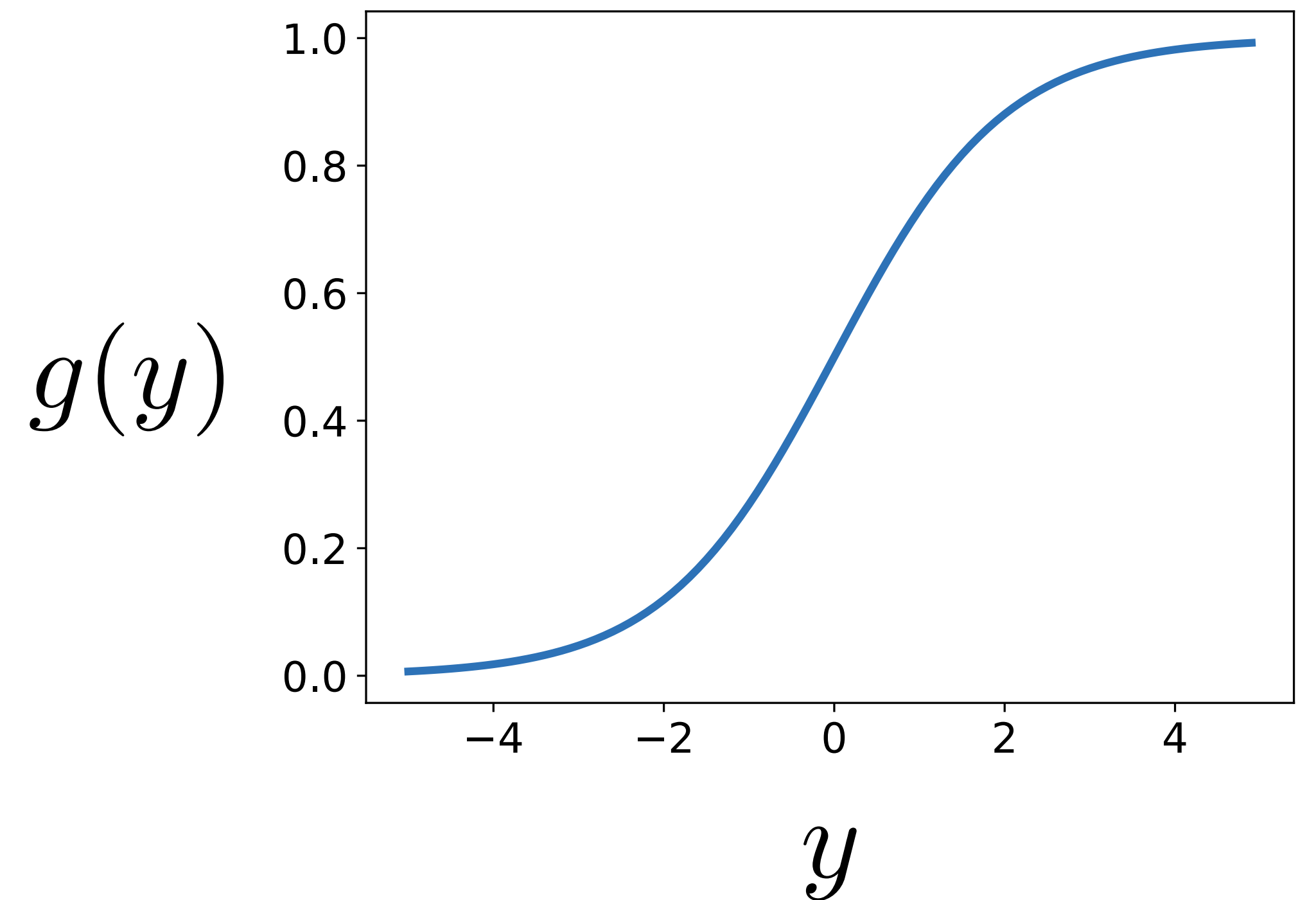
Can't use with gradient descent, $\frac{\partial}{\partial y} g = 0$

Computation in a neural net — nonlinearity



Sigmoid

$$g(y) = \sigma(y) = \frac{1}{1 + e^{-y}}$$

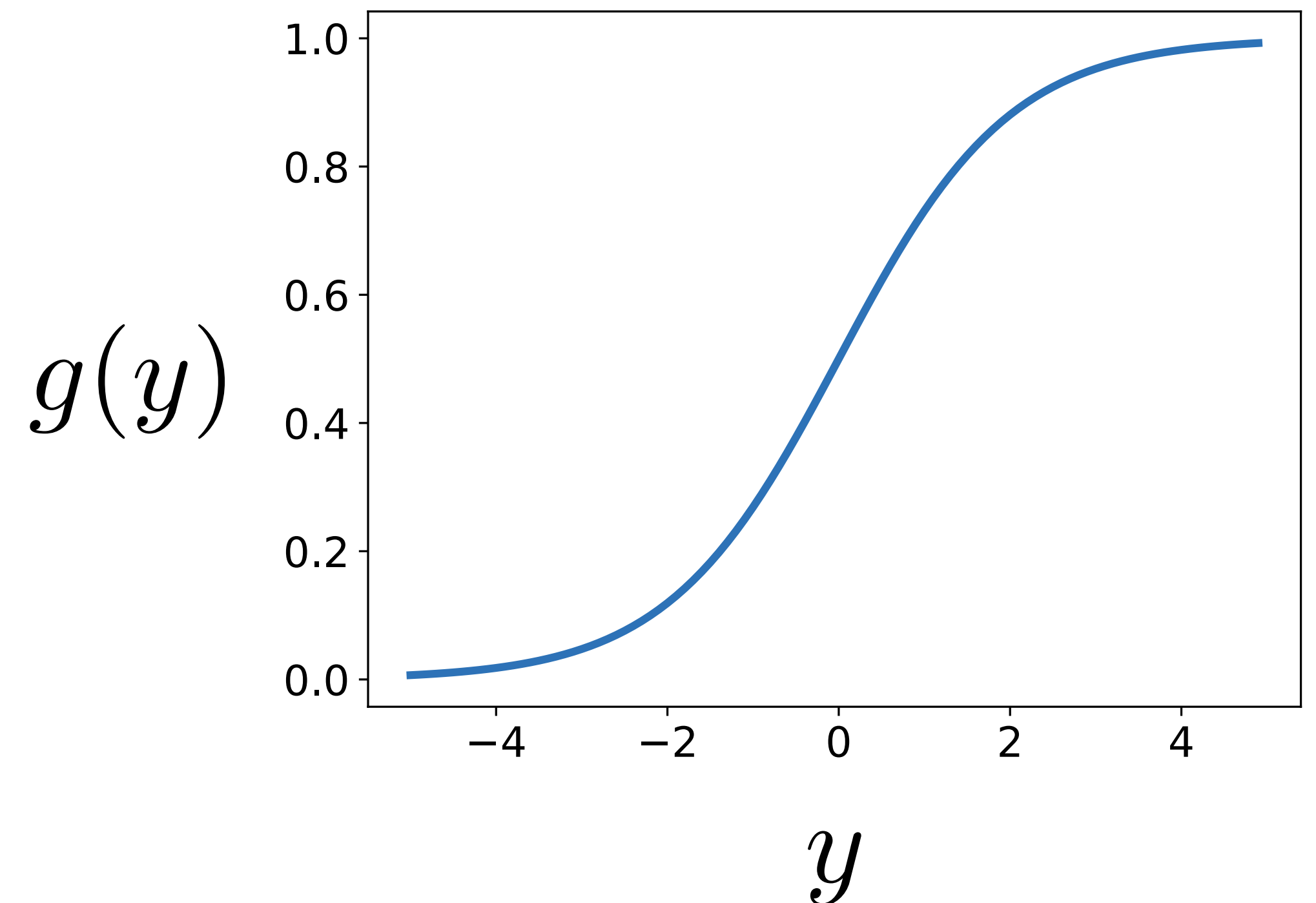


Computation in a neural net — nonlinearity

- Interpretation as firing rate of neuron
- Bounded between $[0,1]$
- Saturation for large +/- inputs
- Gradients go to zero
- Centered at 0.5. Better in practice to use: $\tanh(y) = 2g(y) - 1$

Sigmoid

$$g(y) = \sigma(y) = \frac{1}{1 + e^{-y}}$$



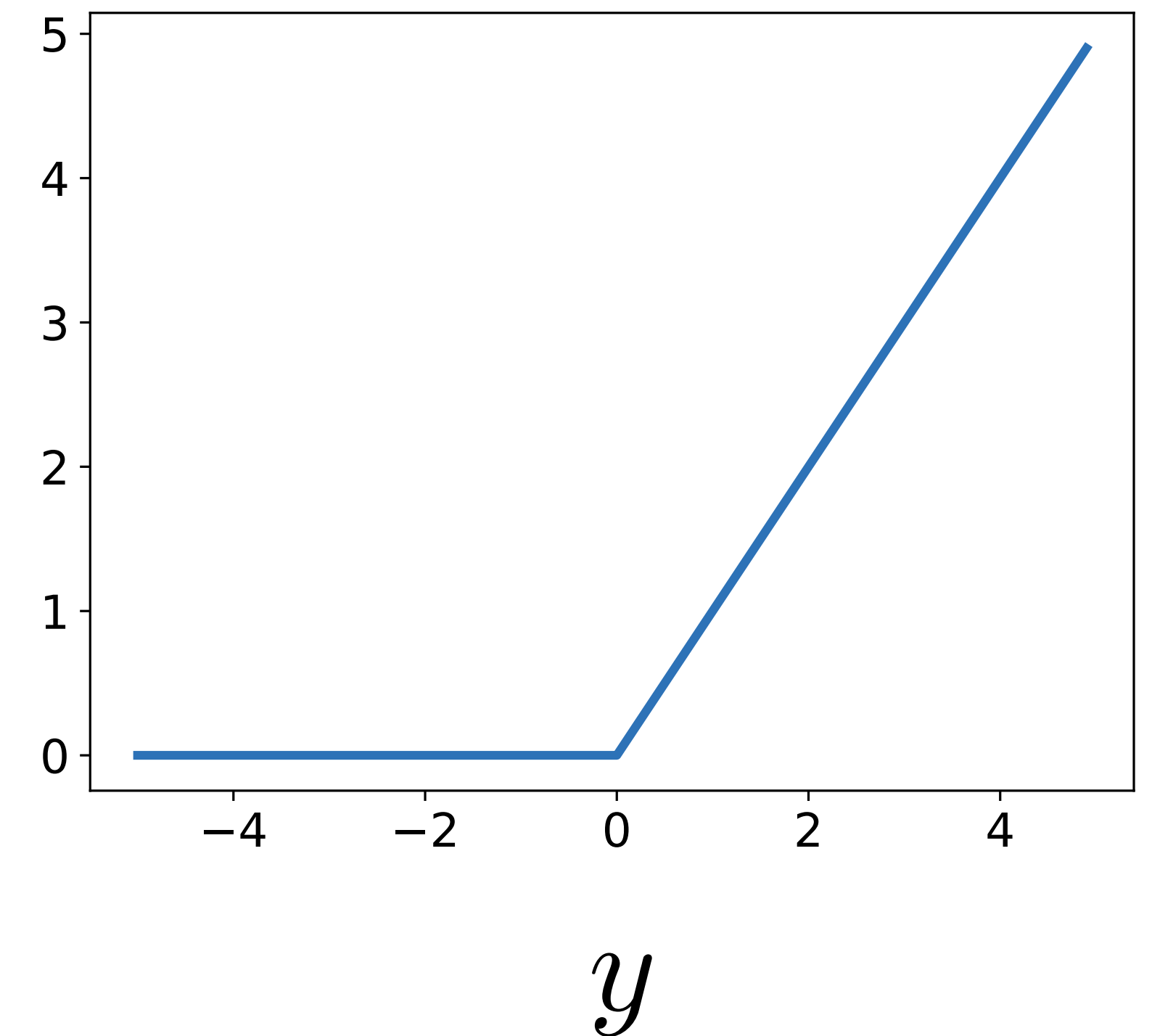
Computation in a neural net — nonlinearity

- Unbounded output (on positive side)
- Efficient to implement: $\frac{\partial g}{\partial y} = \begin{cases} 0, & \text{if } y < 0 \\ 1, & \text{if } y \geq 0 \end{cases}$
- Also seems to help convergence (6x speedup vs. tanh in [Krizhevsky et al. 2012])
- Drawback: if strongly in negative region, unit is dead forever (no gradient).
- Default choice: widely used in current models!

Rectified linear unit (ReLU)

$$g(y) = \max(0, y)$$

$g(y)$



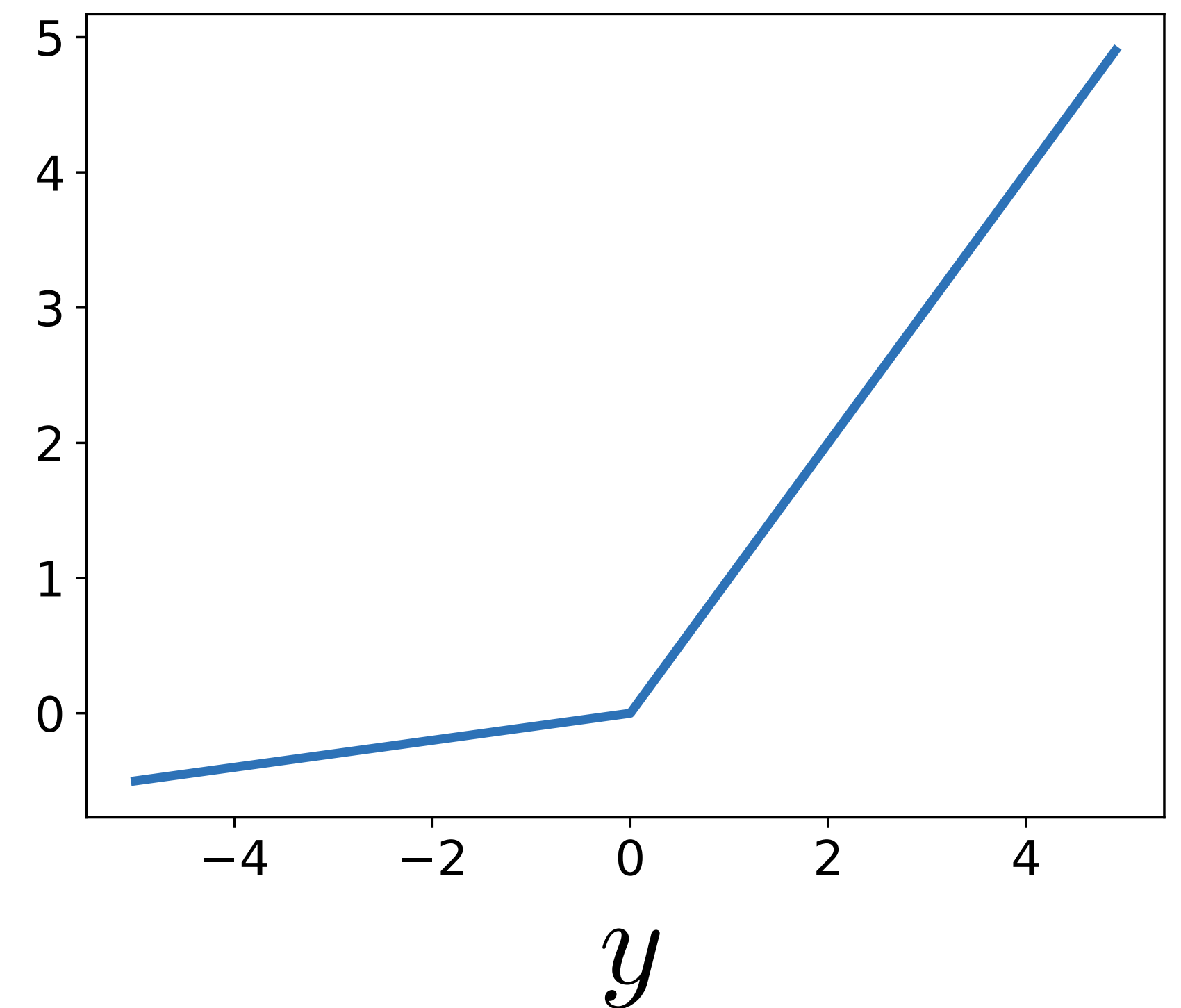
Computation in a neural net — nonlinearity

- where a is small (e.g., 0.02)
- Efficient to implement: $\frac{\partial g}{\partial y} = \begin{cases} -a, & \text{if } y < 0 \\ 1, & \text{if } y \geq 0 \end{cases}$
- Has non-zero gradients everywhere (unlike ReLU)

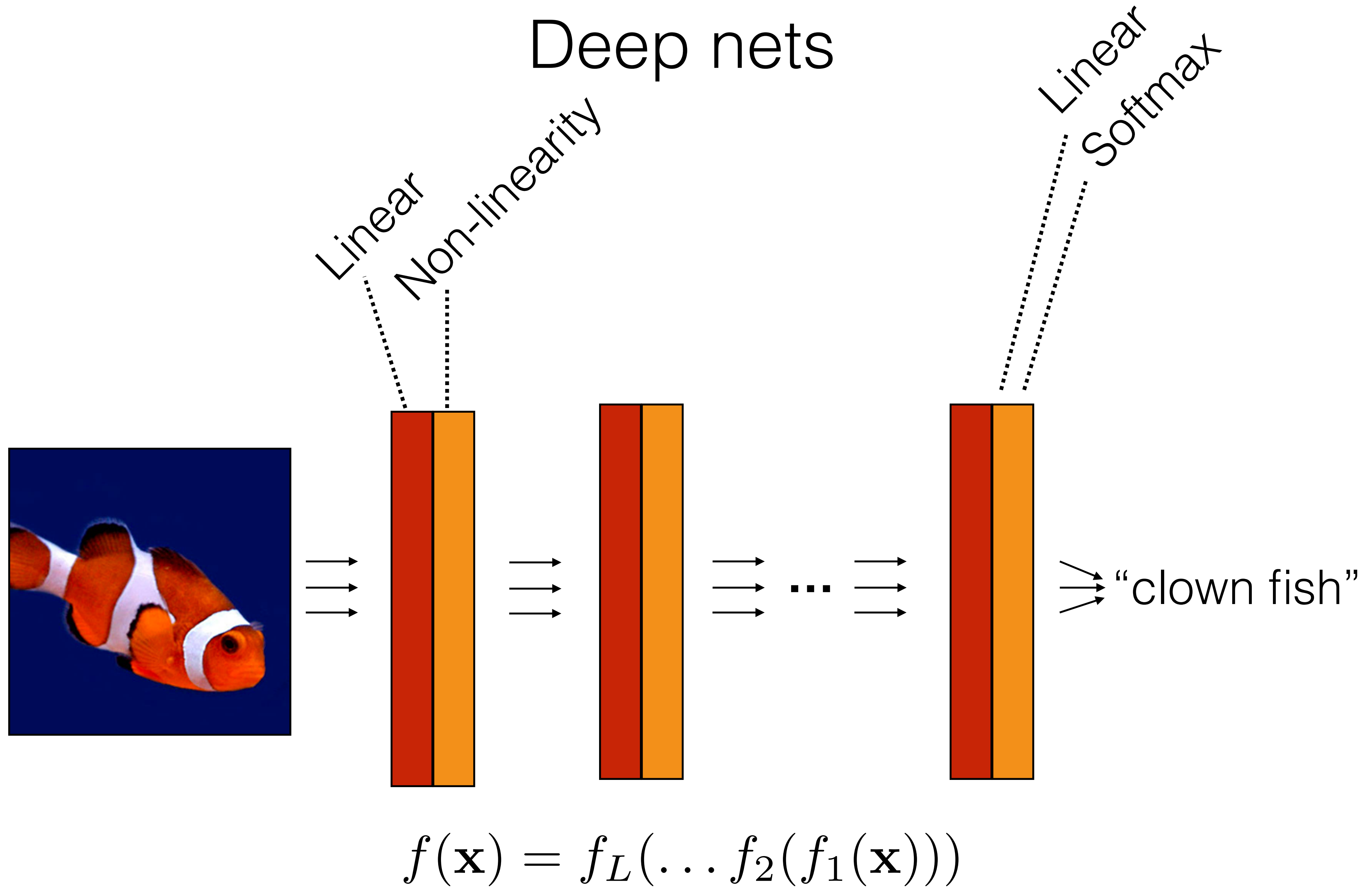
Leaky ReLU

$$g(y) = \begin{cases} \max(0, y), & \text{if } y \geq 0 \\ a \min(0, y), & \text{if } y < 0 \end{cases}$$

$g(y)$



Deep nets

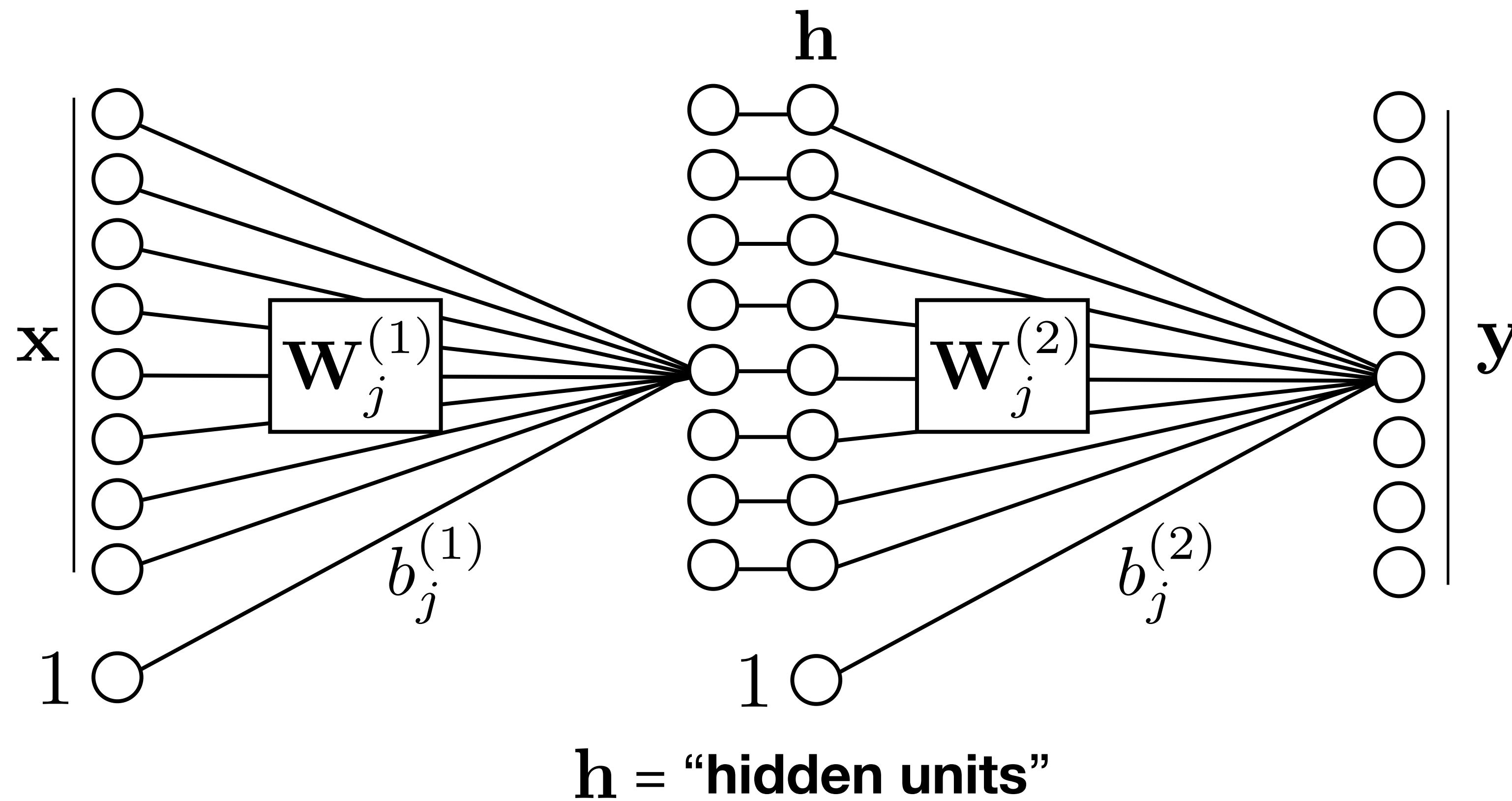


Stacking layers

Input
representation

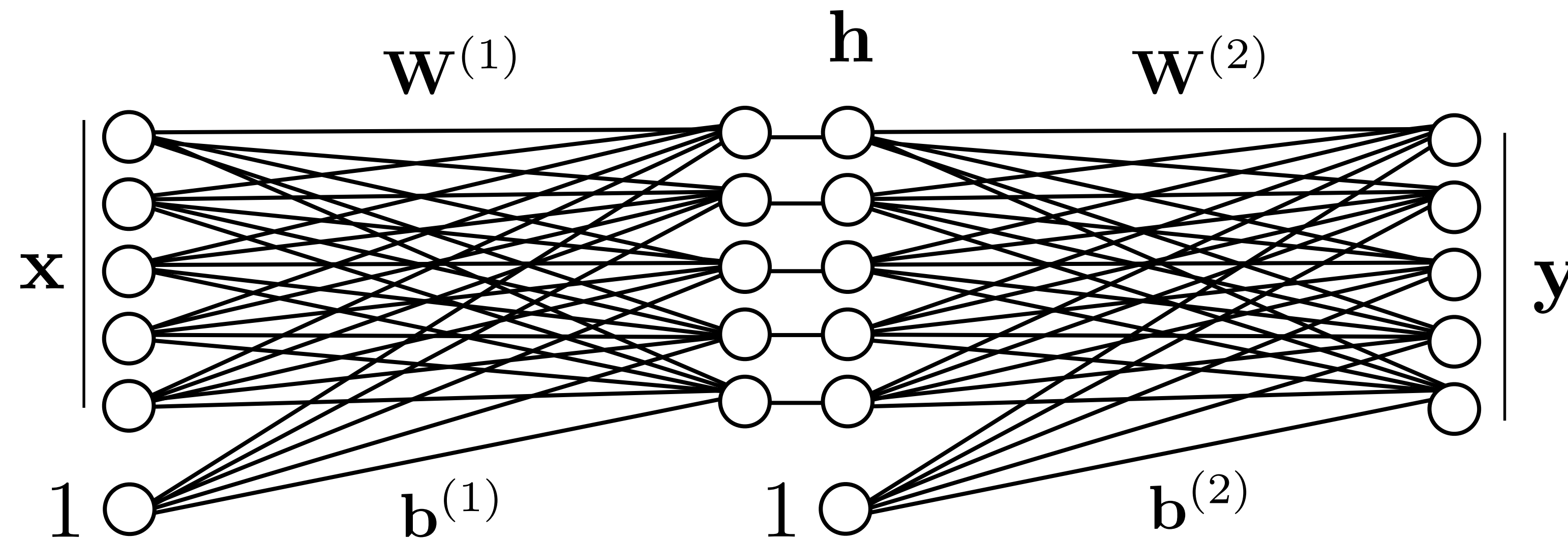
Intermediate
representation

Output
representation



Stacking layers

Input representation Intermediate representation Output representation

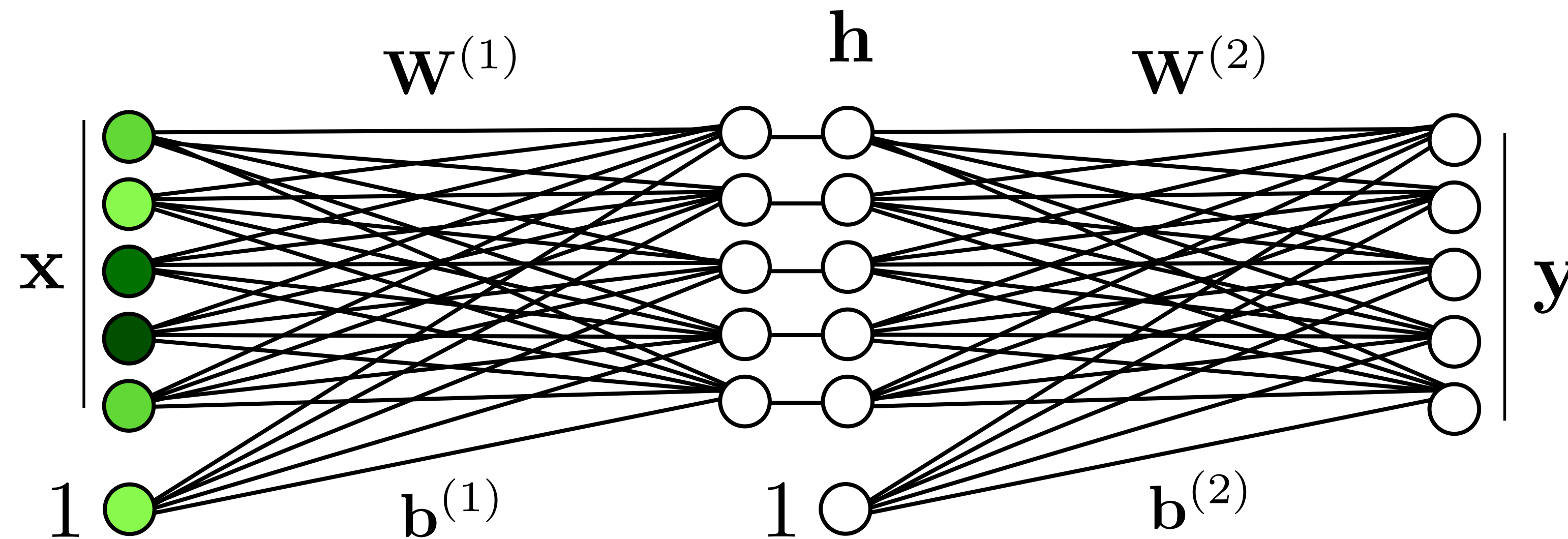


$$\mathbf{h} = g(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) \quad \mathbf{y} = \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}$$

$$\theta = \{\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(L)}, \mathbf{b}^{(1)}, \dots, \mathbf{b}^{(L)}\}$$

Stacking layers

Input representation Intermediate representation Output representation



positive

negative

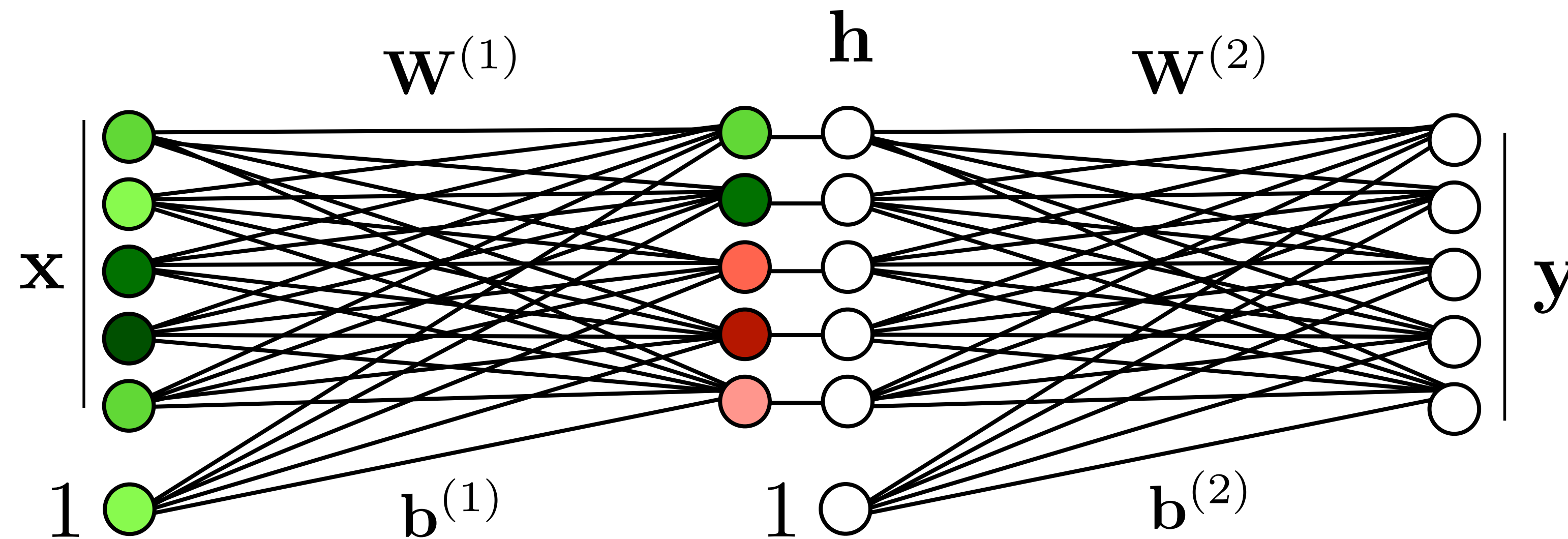
$$\mathbf{h} = g(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$\mathbf{y} = \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}$$

$$\theta = \{\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(L)}, \mathbf{b}^{(1)}, \dots, \mathbf{b}^{(L)}\}$$

Stacking layers

Input representation Intermediate representation Output representation



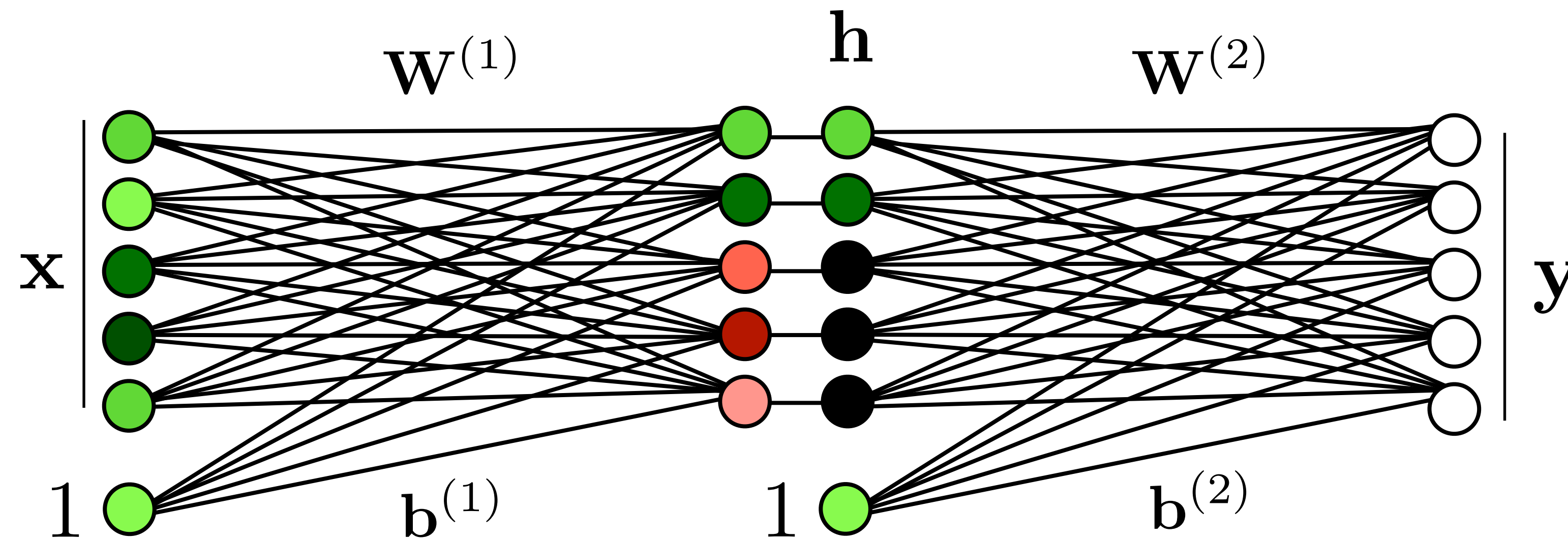
$$\mathbf{h} = g(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$\mathbf{y} = \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}$$

$$\theta = \{\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(L)}, \mathbf{b}^{(1)}, \dots, \mathbf{b}^{(L)}\}$$

Stacking layers

Input representation Intermediate representation Output representation



positive
negative

$$\mathbf{h} = g(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$\mathbf{y} = \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}$$

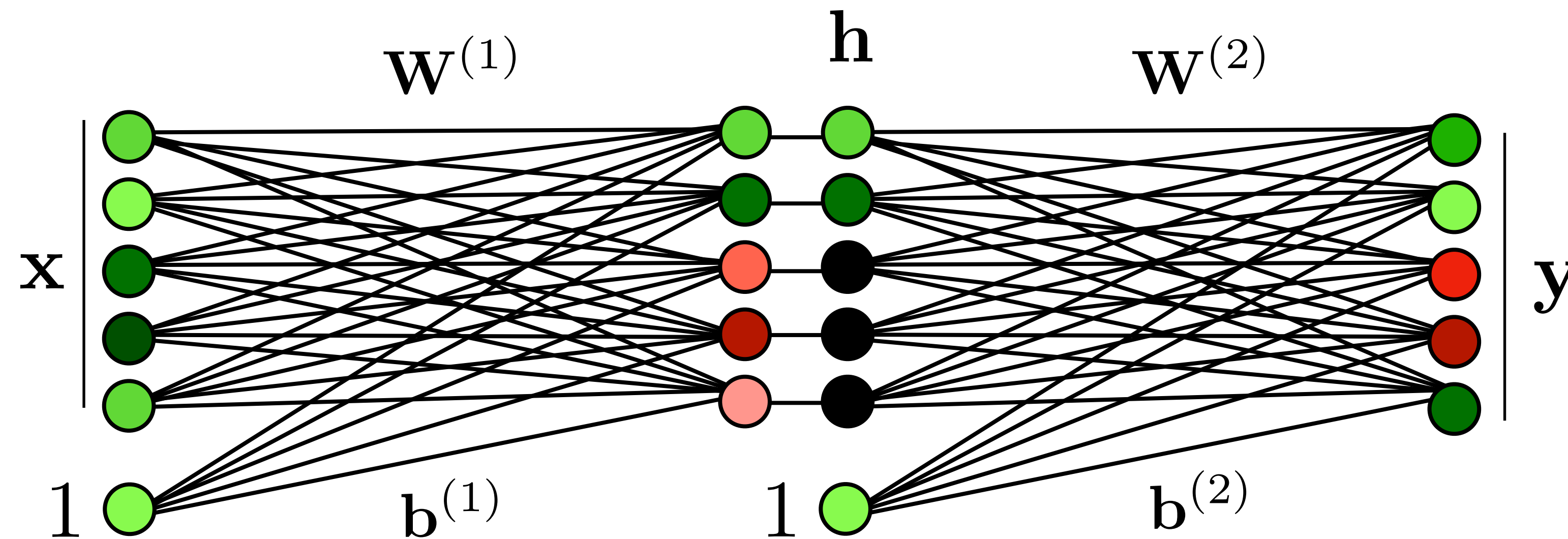
$$\theta = \{\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(L)}, \mathbf{b}^{(1)}, \dots, \mathbf{b}^{(L)}\}$$

Stacking layers

Input
representation

Intermediate
representation

Output
representation



positive

negative

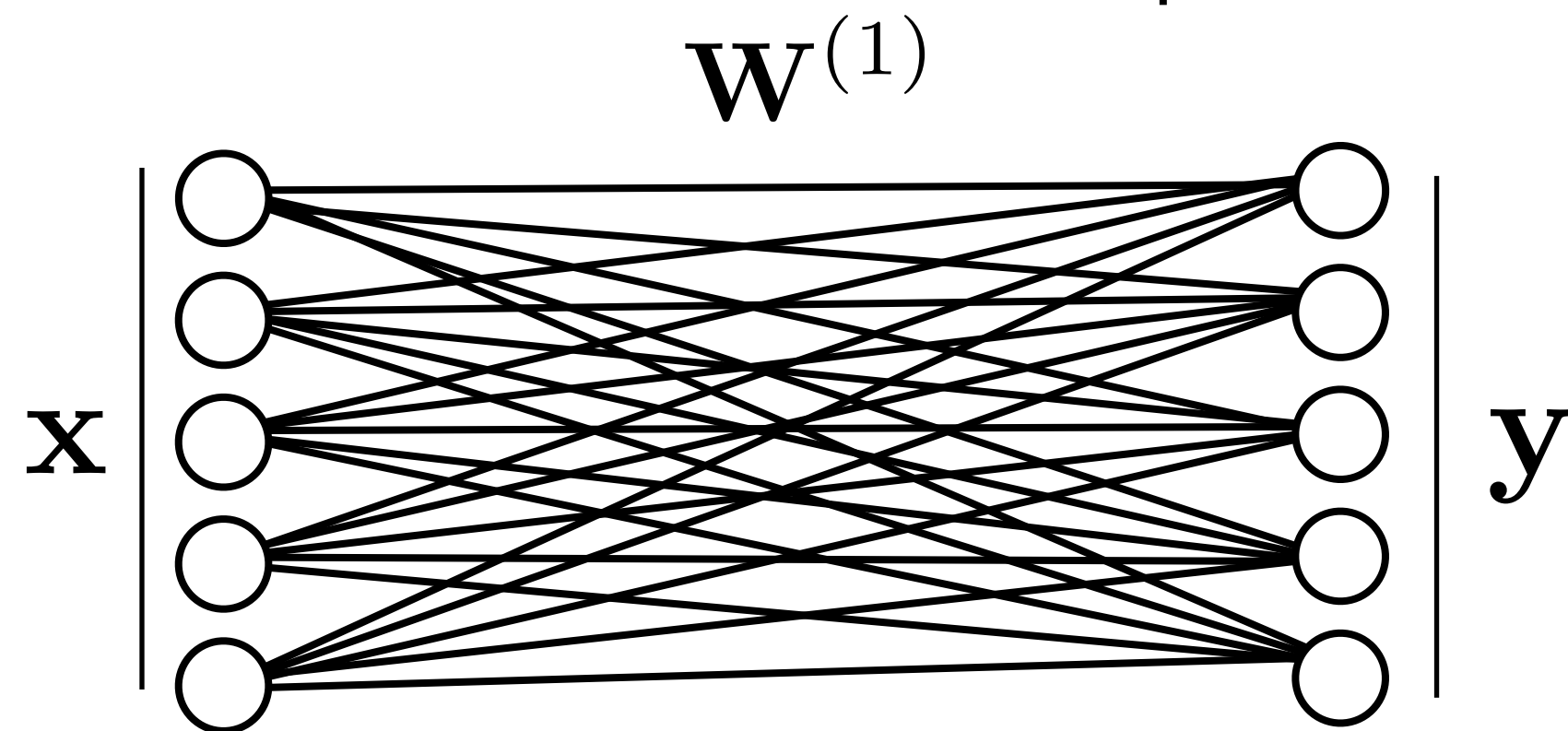
$$\mathbf{h} = g(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$\mathbf{y} = \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}$$

$$\theta = \{\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(L)}, \mathbf{b}^{(1)}, \dots, \mathbf{b}^{(L)}\}$$

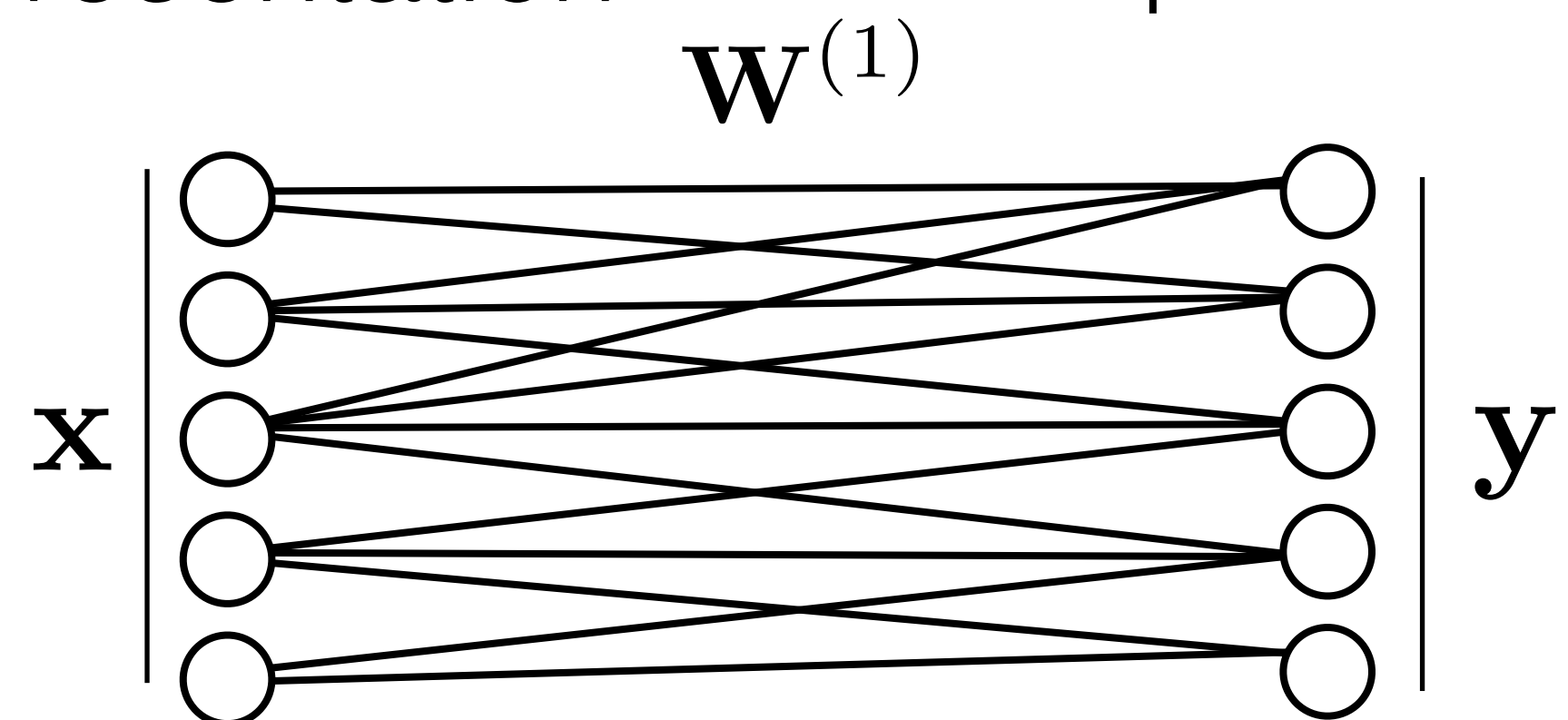
Connectivity patterns

Input representation Output representation



Fully connected layer

Input representation Output representation



*Locally connected layer
(Sparse $\mathbf{W}$)*

Computation has a simple form

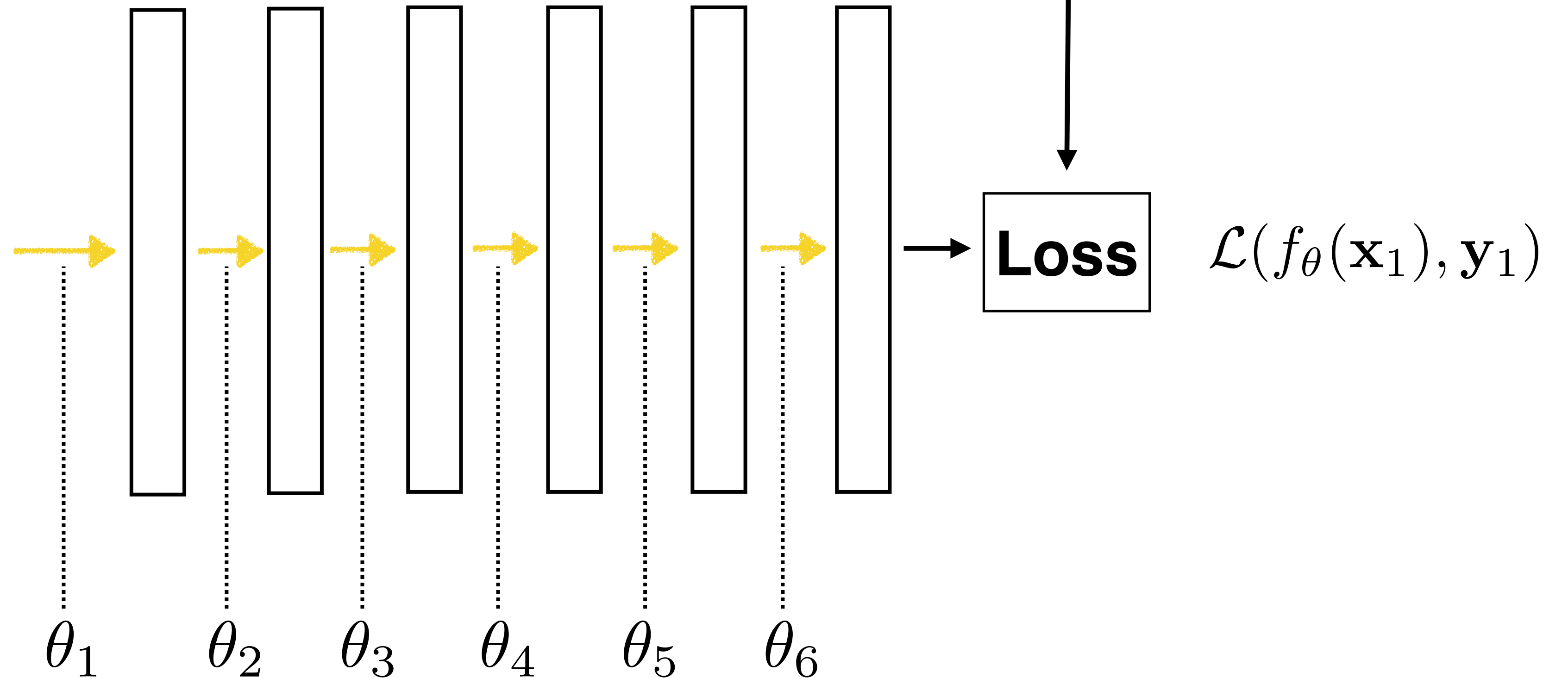
$$y = \mathbf{W}^{(n)} g(\mathbf{W}^{(n-1)} \dots g(\mathbf{W}^{(3)} g(\mathbf{W}^{(2)} (g(\mathbf{W}^{(1)} \mathbf{x}))))))$$

- Composition of linear functions with nonlinearities in between
- For example, matrix multiplications with ReLU, $\max(0, \mathbf{x})$ afterwards
- Do a matrix multiplication, set all negative values to 0, repeat

Deep learning

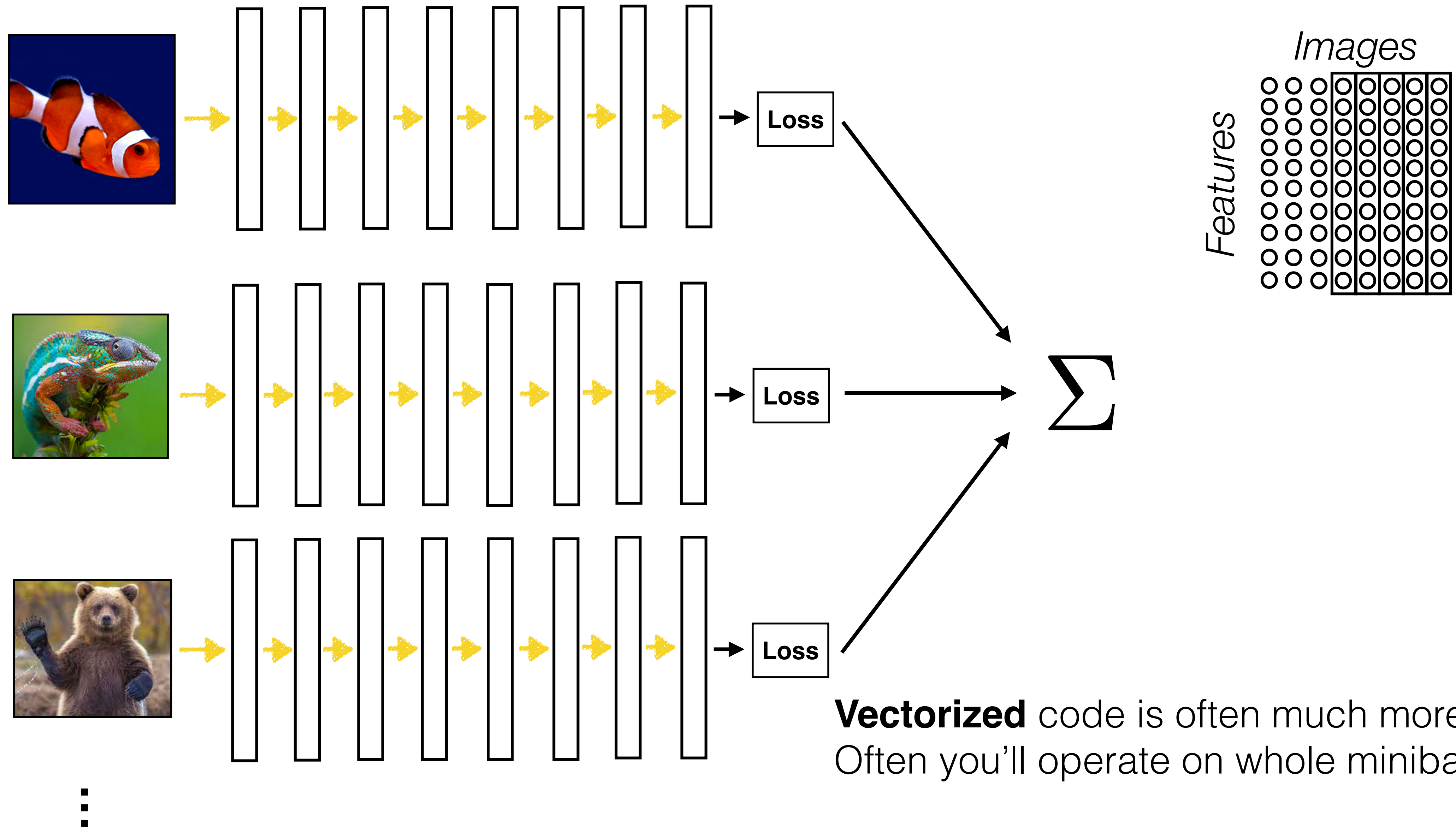
$\mathbf{y}_1$
“clown fish”

Learned



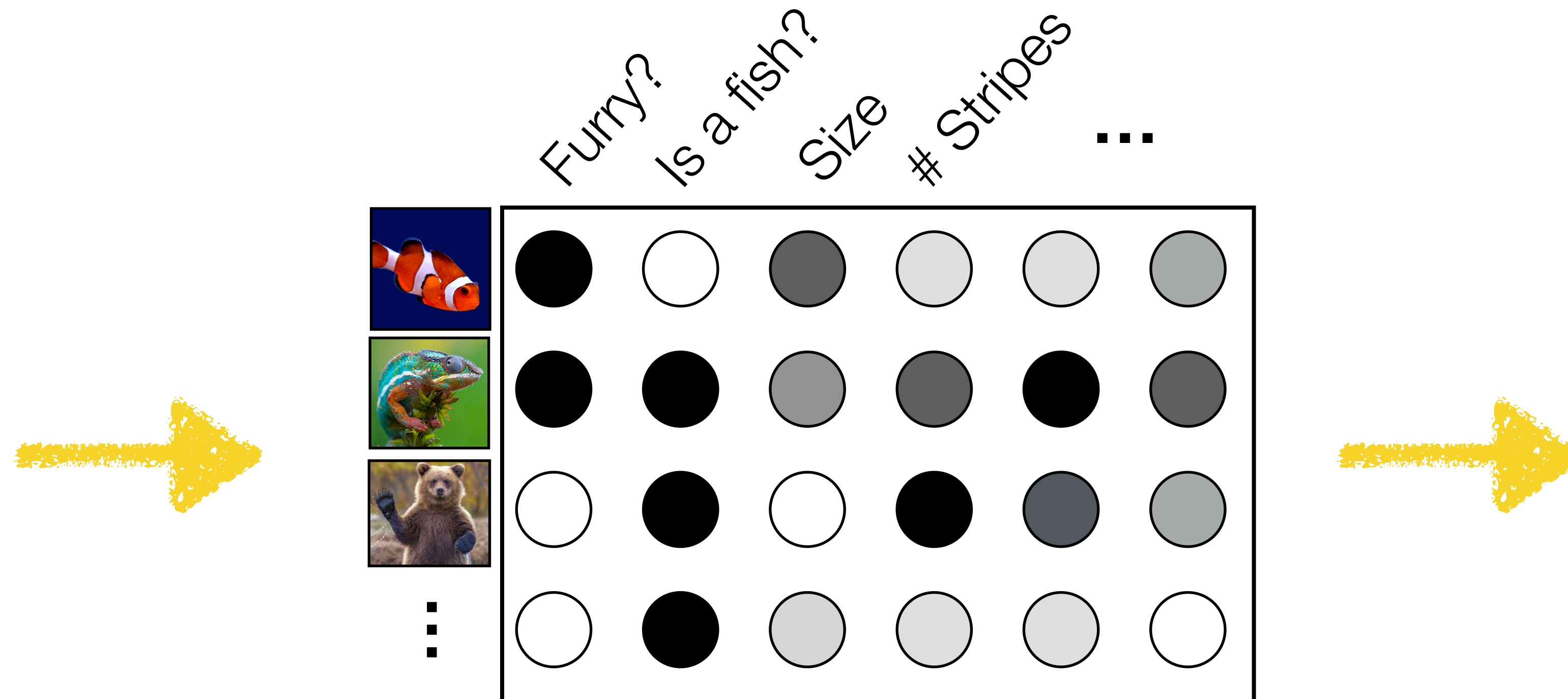
$$\theta^* = \arg \min_{\theta} \sum_{i=1}^N \mathcal{L}(f_{\theta}(\mathbf{x}_i), \mathbf{y}_i)$$

Batch processing



Tensors

(multi-dimensional arrays)



Each layer is a representation of the data

Tensors

(multi-dimensional arrays)

$$\mathbf{h}^{(1)} \in \mathbb{R}^{N_{\text{batch}} \times C^{(1)}}$$

neurons

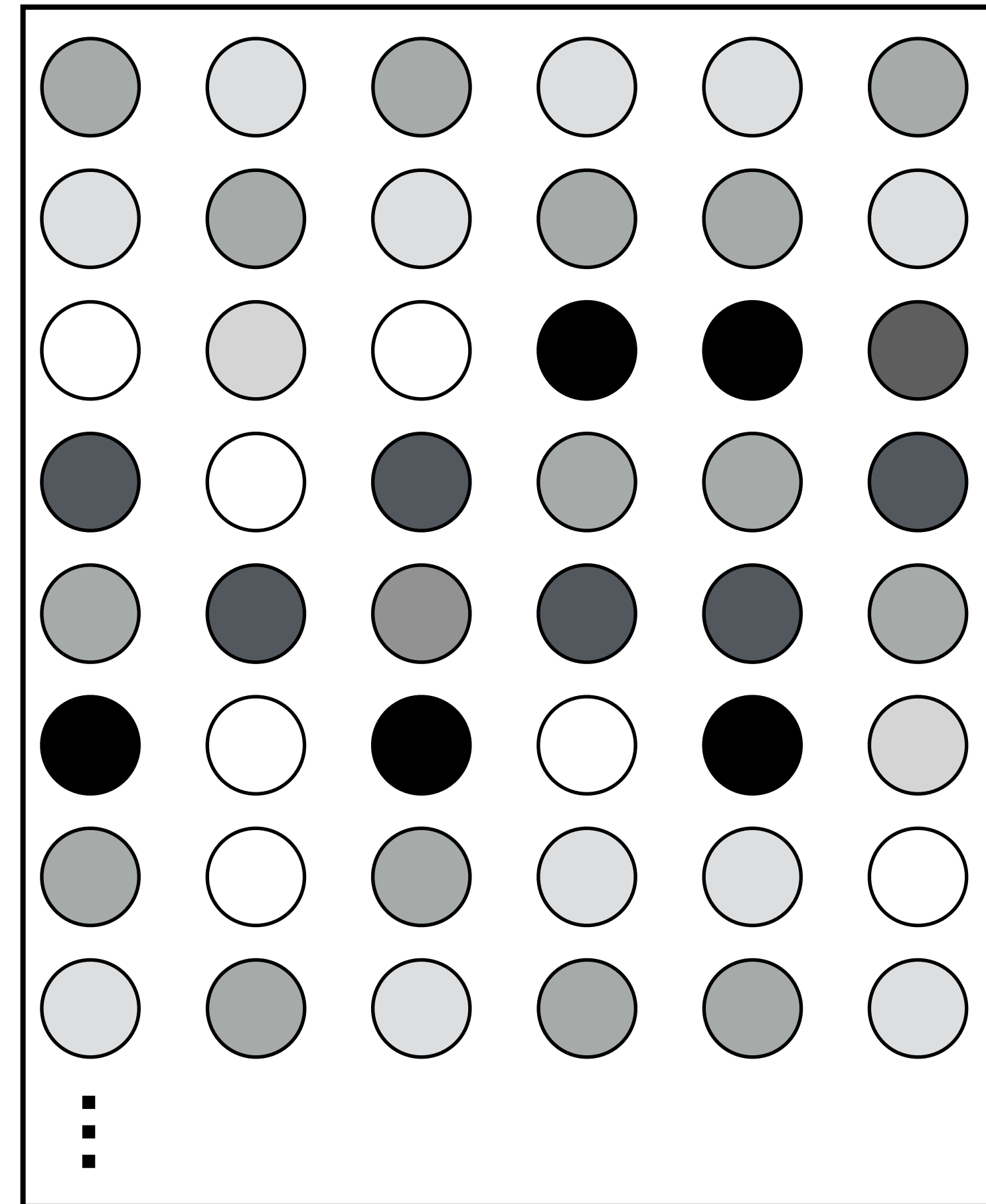
features

units

“channels”

N_{batch}

$C^{(1)}$



Tensors

(multi-dimensional arrays)

$$\mathbf{h}^{(1)} \in \mathbb{R}^{N_{\text{batch}} \times C^{(1)}}$$

neurons

features

units

“channels”

N_{batch}

$C^{(1)}$

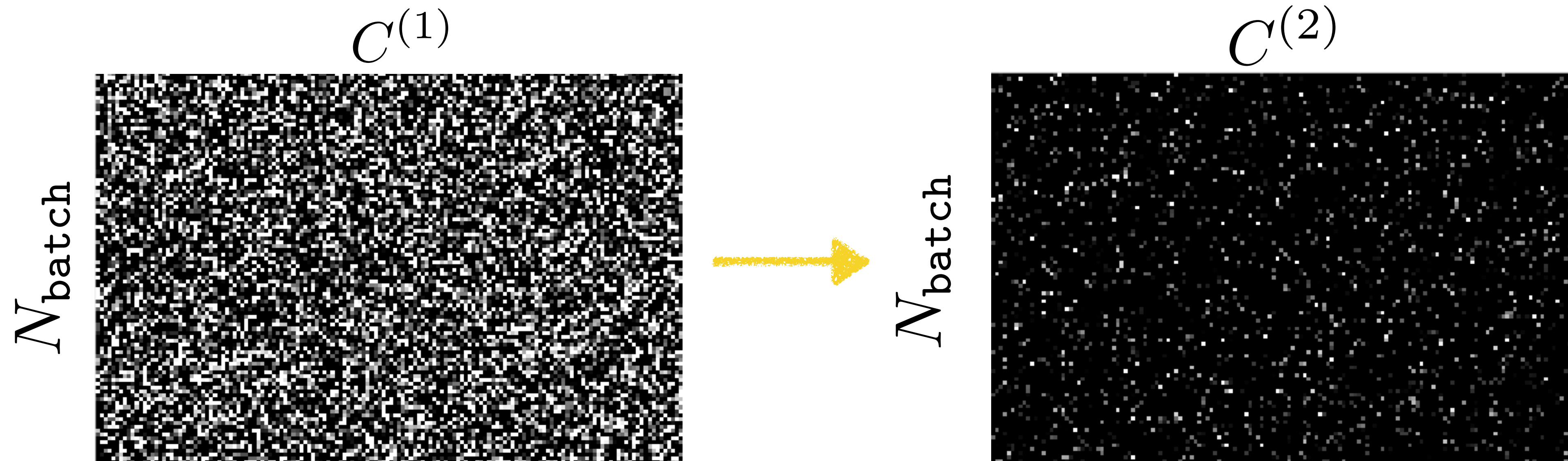


Tensors

(multi-dimensional arrays)

$$\mathbf{h}^{(1)} \in \mathbb{R}^{N_{\text{batch}} \times C^{(1)}}$$

$$\mathbf{h}^{(2)} \in \mathbb{R}^{N_{\text{batch}} \times C^{(2)}}$$

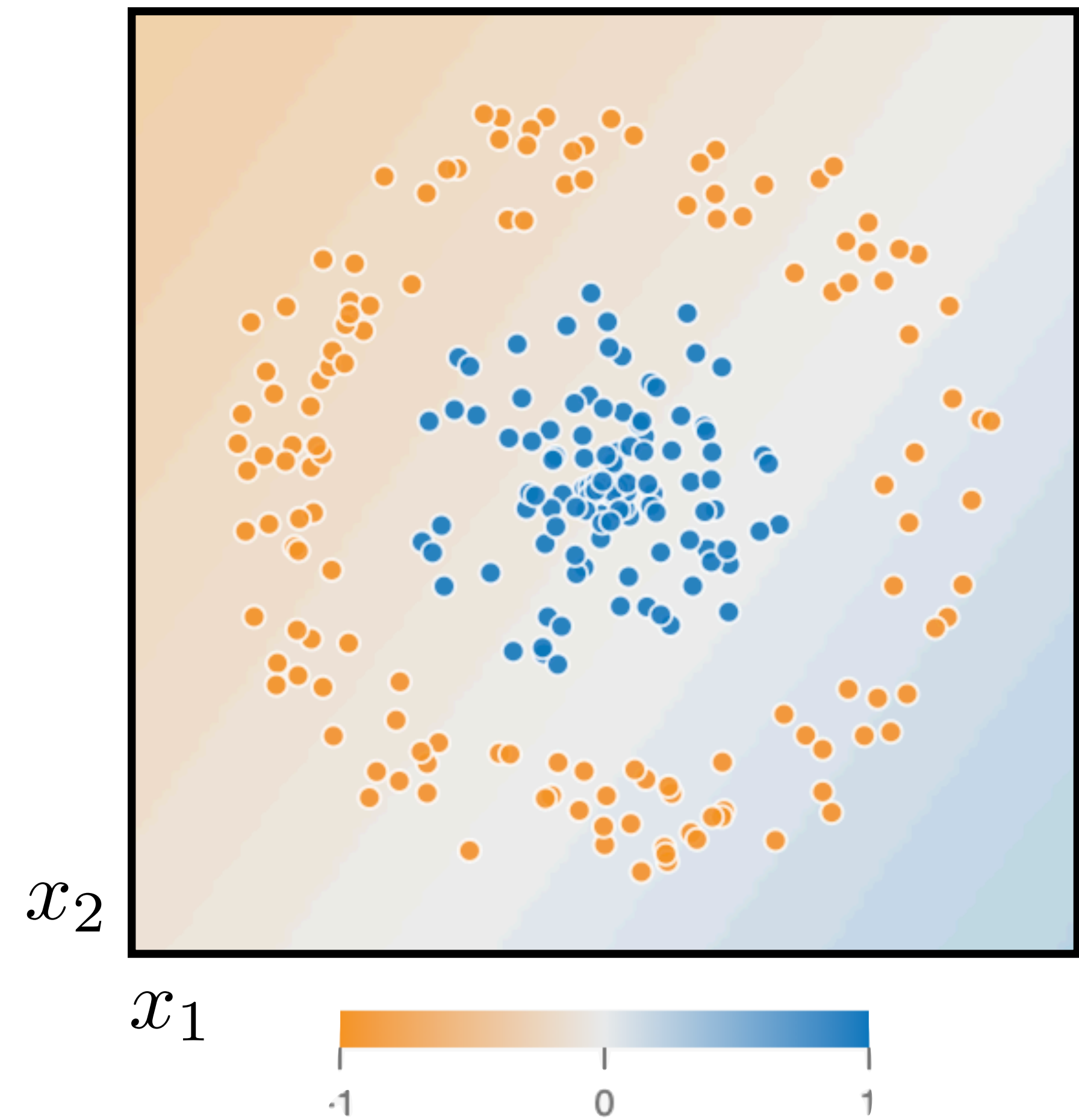
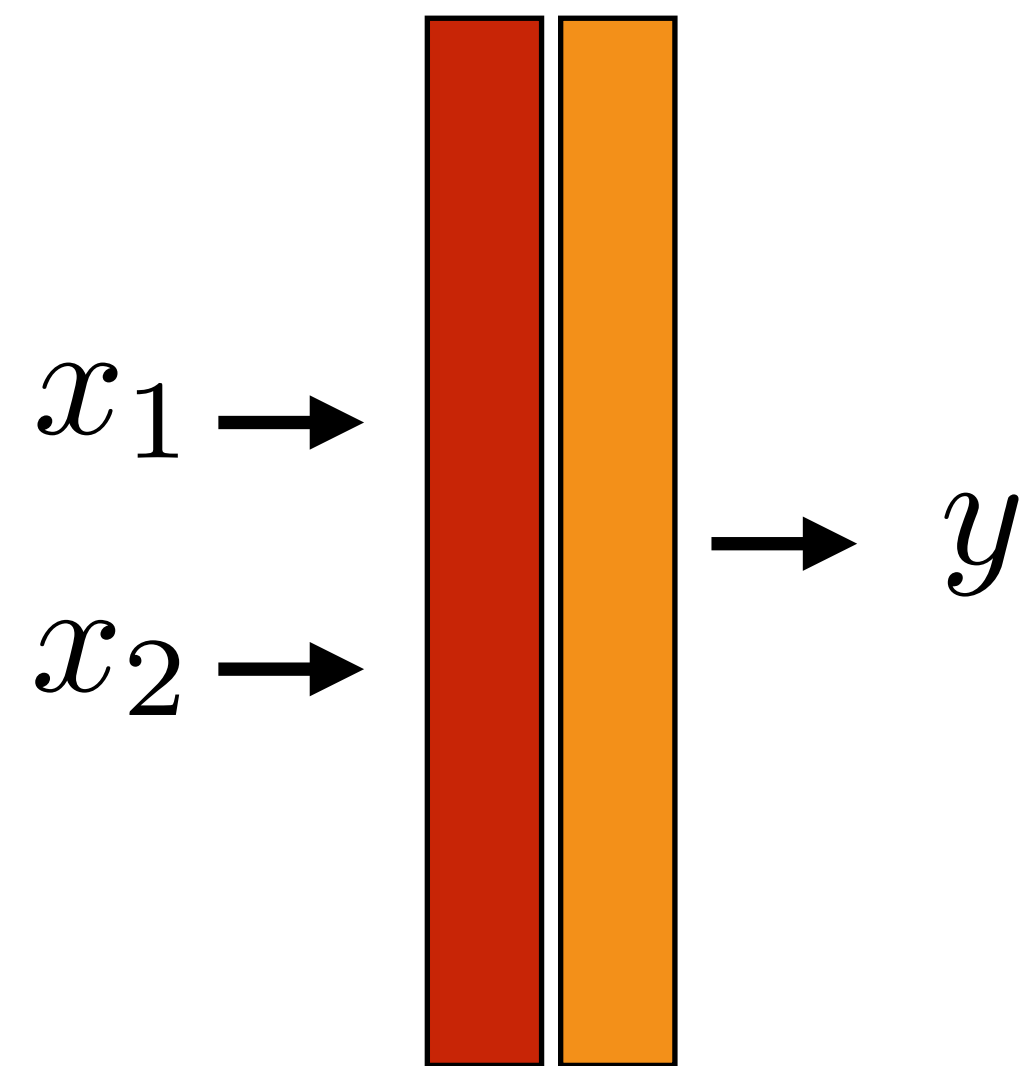


Representational power

- 1 layer? Linear decision surface.
- 2+ layers? In theory, can represent any function!
 - Simple proof by M. Nielsen
<http://neuralnetworksanddeeplearning.com/chap4.html>
- But issue is efficiency: very wide two layers vs narrow deep model? In practice, more layers helps.

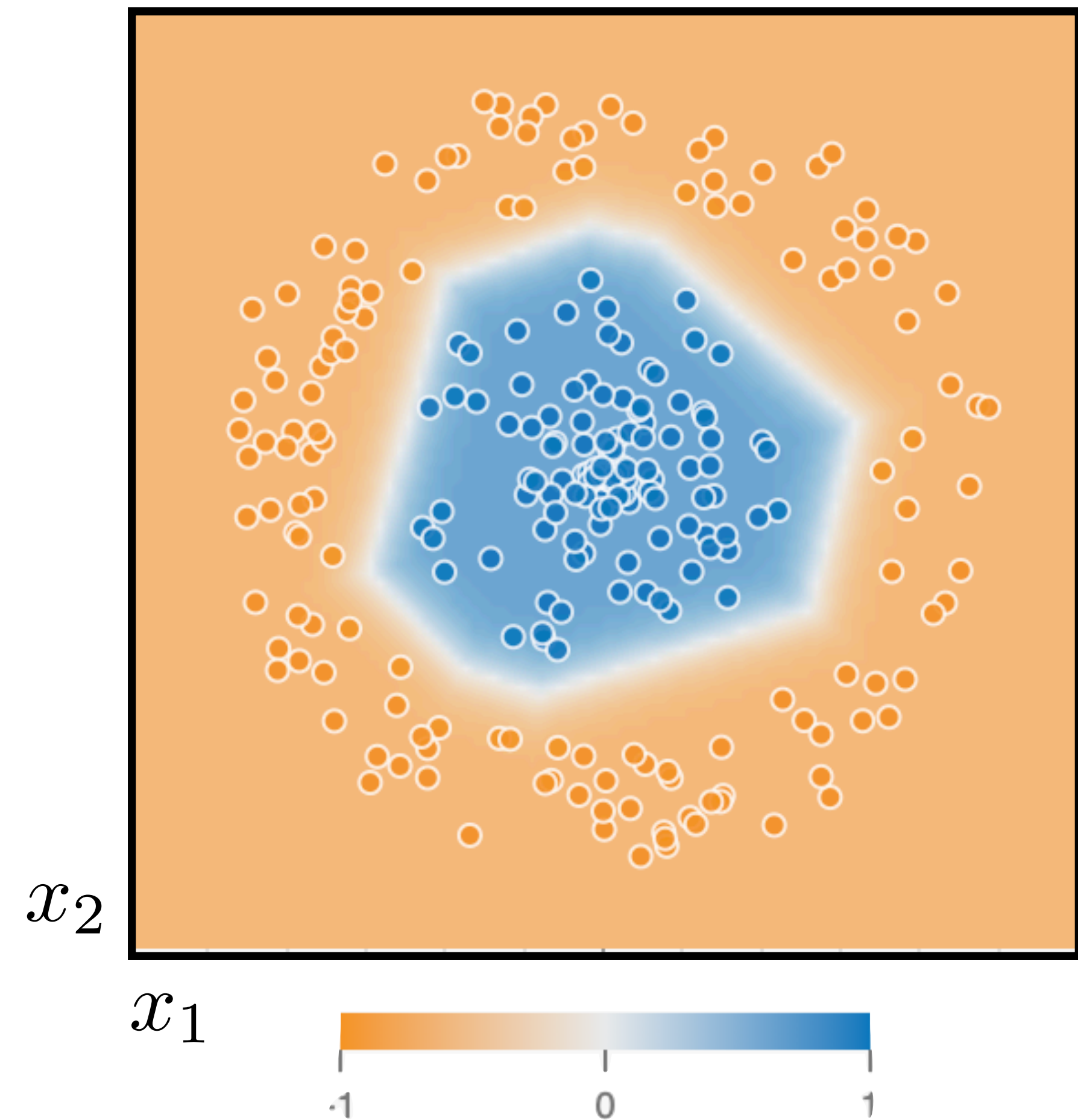
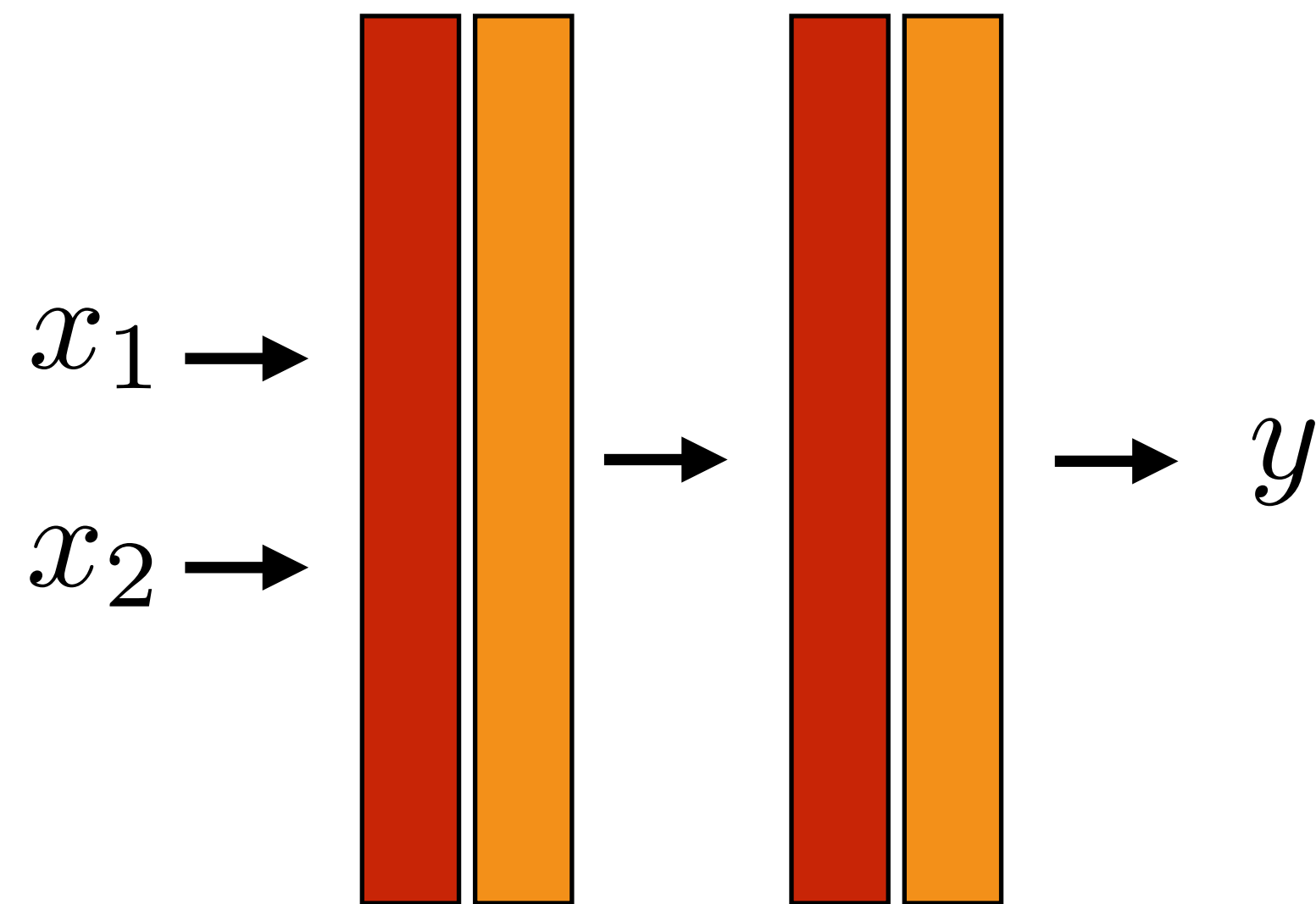
Example: perceptron

$$\mathbf{y} = \sigma(\mathbf{W}^{(1)}\mathbf{x})$$

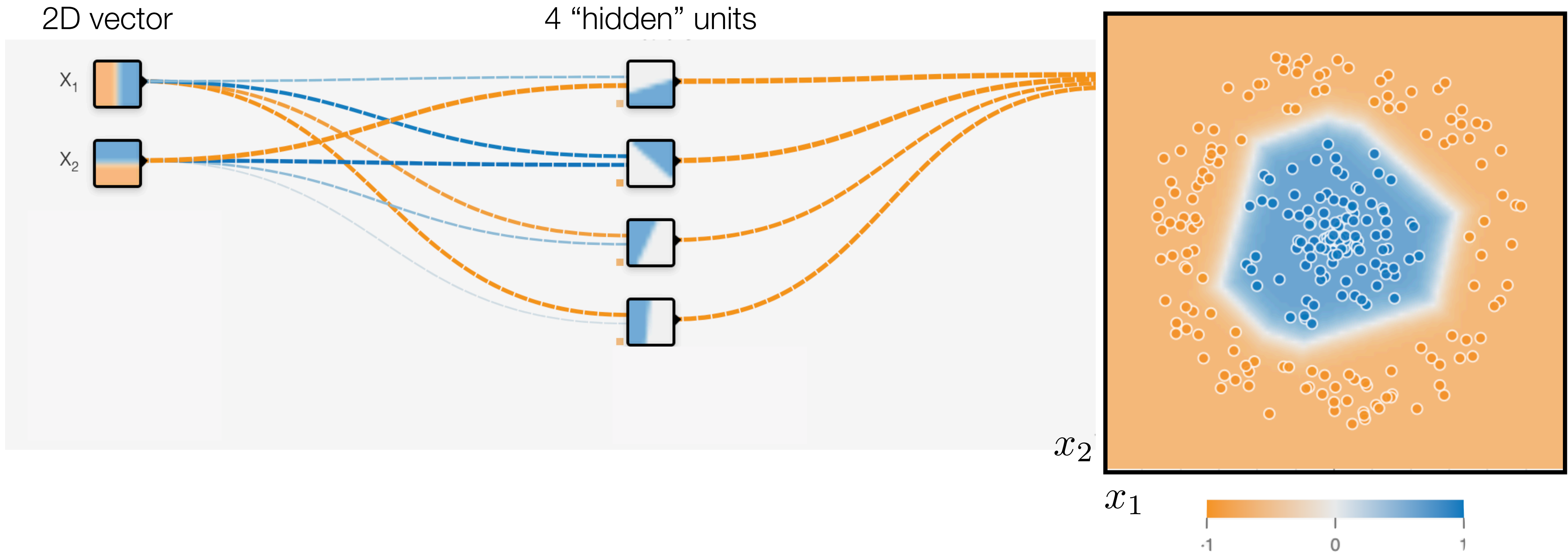


Example: multilayer perceptron (MLP)

$$y = \sigma(\mathbf{W}^{(2)} \max(0, \mathbf{W}^{(1)} \mathbf{x}))$$

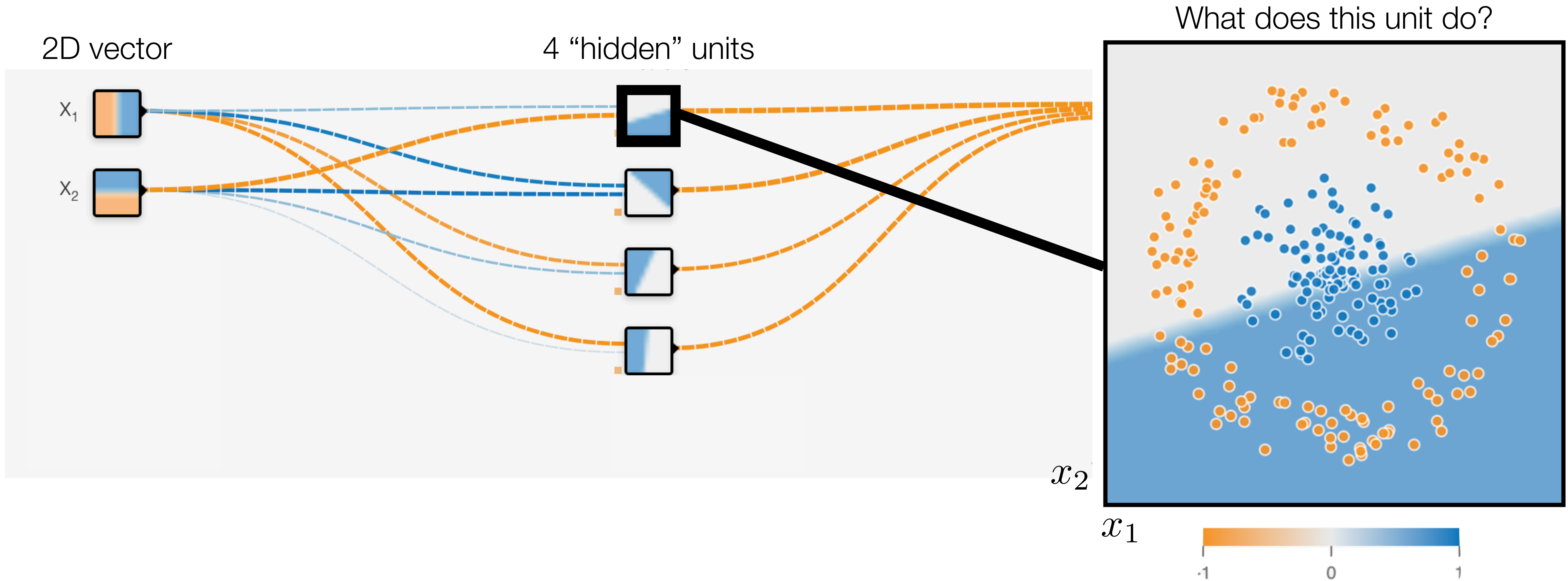


Example: multilayer perceptron (MLP)

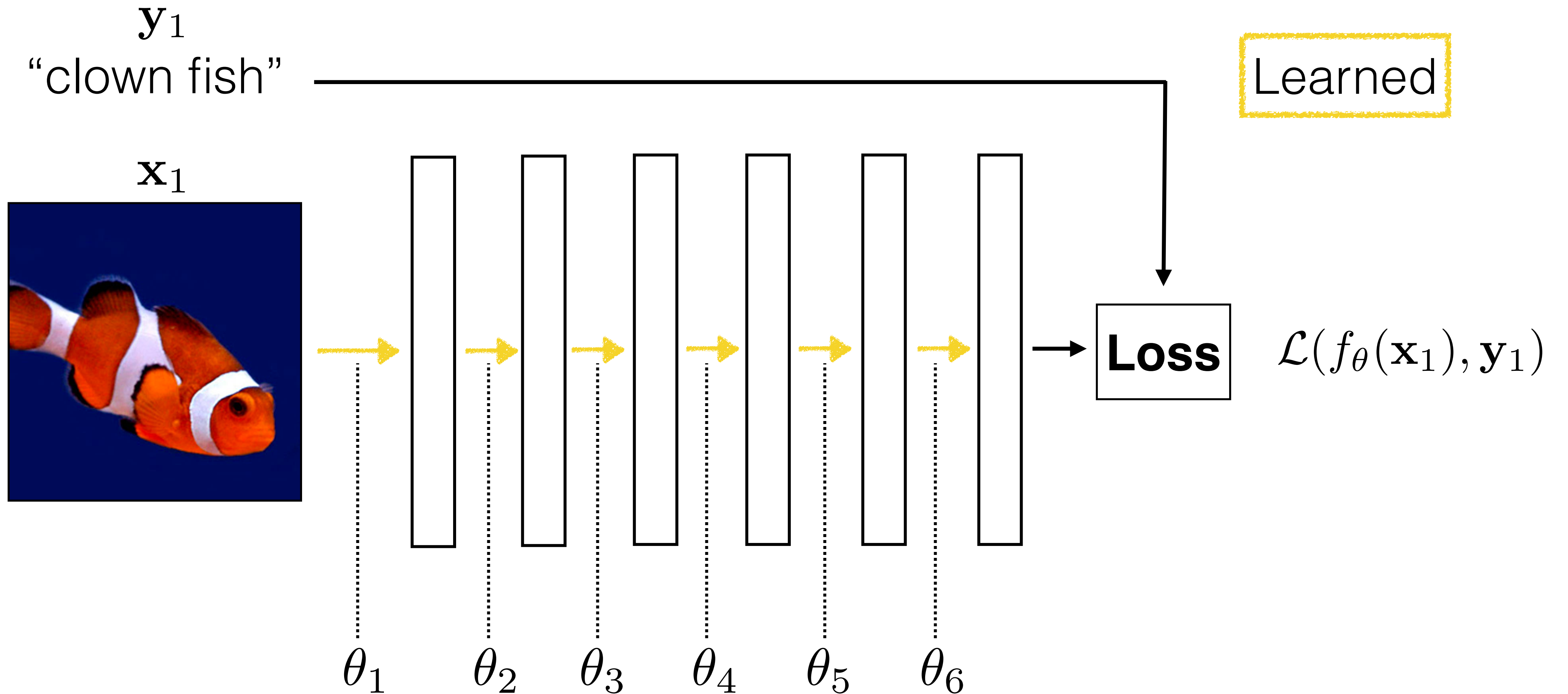


Example source: <http://playground.tensorflow.org>

Example: multilayer perceptron (MLP)



How do we learn the parameters?



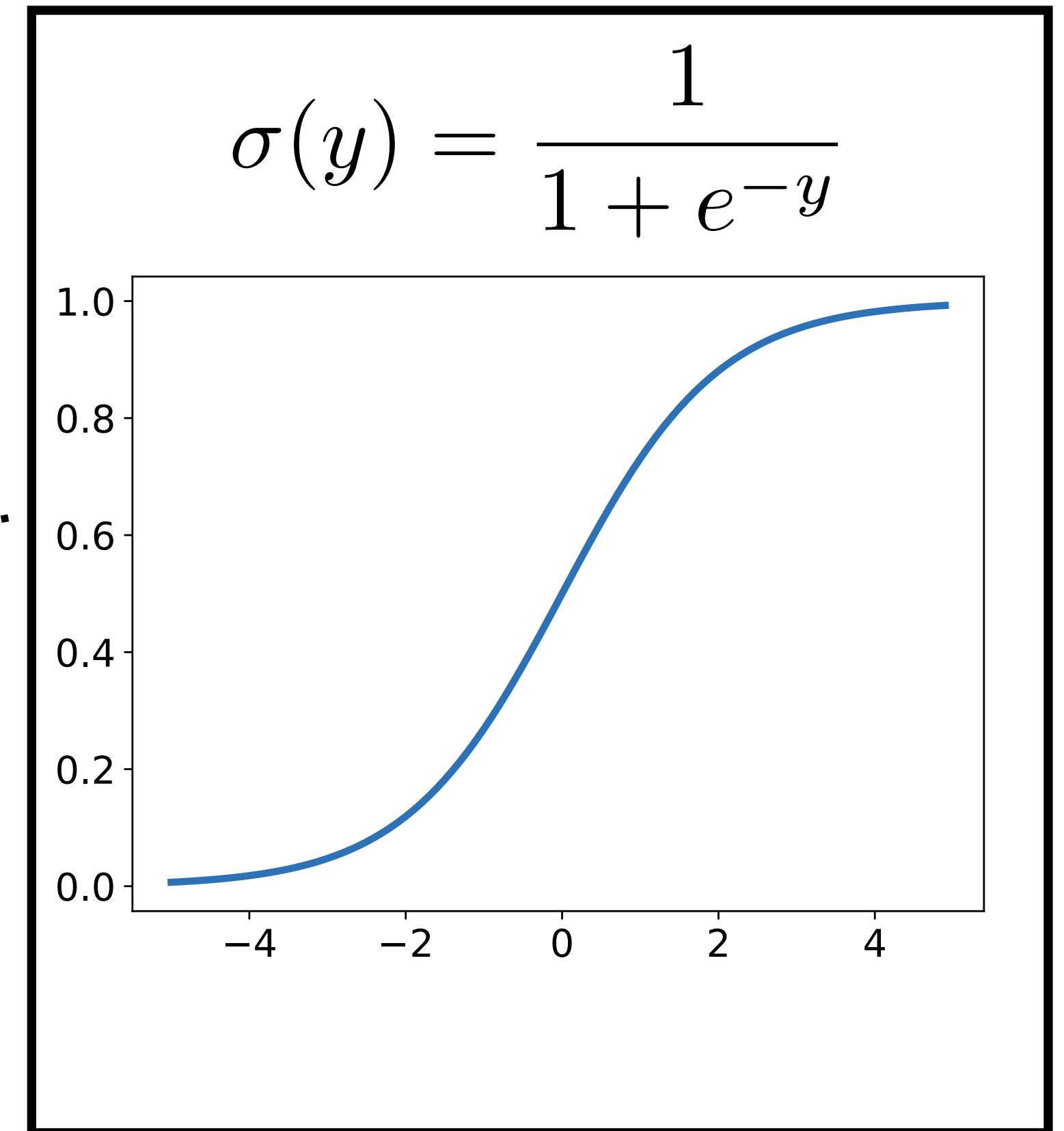
$$\theta^* = \arg \min_{\theta} \sum_{i=1}^N \mathcal{L}(f_{\theta}(x_i), y_i)$$

Learning parameters

Squared loss with single-variable network:

$$L = \frac{1}{2}(y - f(x))^2$$

$$L = \frac{1}{2}(y - \sigma(wx + b))^2$$



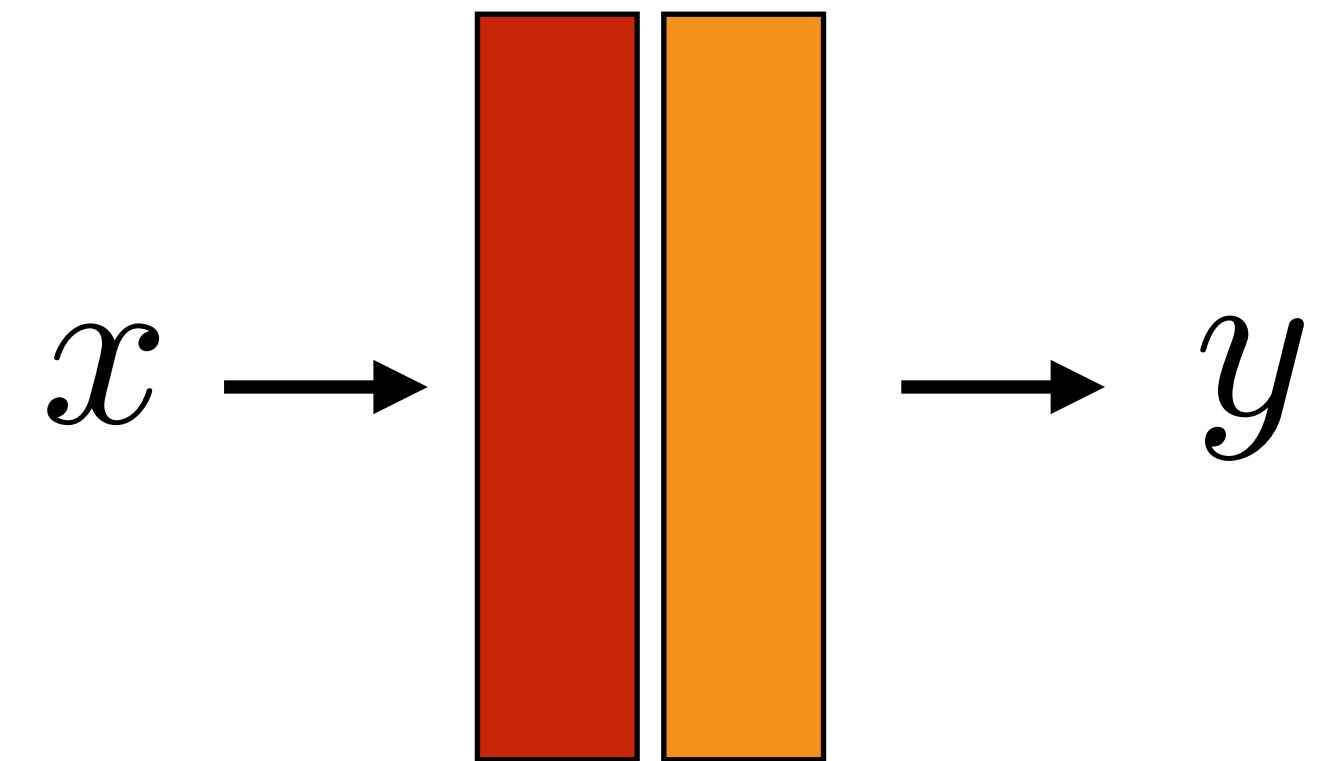
Learning parameters

Squared loss with single-variable network:

$$L = \frac{1}{2}(y - f(x))^2$$

$$L = \frac{1}{2}(y - \sigma(wx + b))^2$$

Want: derivatives $\frac{\partial L}{\partial w}$, $\frac{\partial L}{\partial b}$



Learning parameters

Writing out the layers explicitly:

$$z = wx + b$$

$$t = \sigma(z)$$

$$L = \frac{1}{2}(y - t)^2$$

How do we do this?
Use the chain rule!

$$\frac{\partial}{\partial x} f(g(x)) = \frac{\partial f}{\partial g} \frac{\partial g}{\partial x}$$

Computing derivatives with the chain rule

$$\frac{\partial L}{\partial w} = \frac{\partial}{\partial w} \left[\frac{1}{2} (y - \sigma(wx + b))^2 \right]$$

$$= (y - \sigma(wx + b)) \frac{\partial}{\partial w} (y - \sigma(wx + b))$$

$$= (y - \sigma(wx + b)) \sigma'(wx + b) \frac{\partial}{\partial w} (wx + b)$$

$$= (y - \sigma(wx + b)) \sigma'(wx + b) x$$

$$\frac{\partial L}{\partial b} = \frac{\partial}{\partial b} \left[\frac{1}{2} (y - \sigma(wx + b))^2 \right]$$

$$= (y - \sigma(wx + b)) \frac{\partial}{\partial b} (y - \sigma(wx + b))$$

$$= (y - \sigma(wx + b)) \sigma'(wx + b) \frac{\partial}{\partial b} (wx + b)$$

$$= (y - \sigma(wx + b)) \sigma'(wx + b)$$

Limitations to this approach

- Inefficient! Lots of redundant computation
- We'll also need to extend this to multivariable functions
- **Next lecture:** backpropagation